

Técnico em
desenvolvimento de sistemas

Levantamento de requisitos

Sumário

Fundamentos de requisitos	3
Definição	4
Modelos de documentação	8
Regras de negócio	9
Restrições	12
Superando as restrições	14
Tipos de requisitos	16
Requisitos funcionais	17
Não funcionais	19
Recapitulando	24
Técnicas e processos de levantamento de requisitos	25
Definição	26
Técnicas	27
Briefing	27
Levantamento Orientado a Pontos de Vista	31
Etnografia (observação)	35
Entrevistas	37
Brainstorming	40
Prototipação	43
Fases	46
Priorização de Requisitos	47
Técnicas de Priorização de Requisitos	48
Recapitulando	53
Encerramento	53
Design thinking e solução de problemas	54
Definição	55
Etapas	58
Encerramento	76
Gerenciamento e validação de requisitos	77
Gerenciamento e validação de requisitos	78

Sumário

Definição	78
Gestão de mudanças	82
Processo de gestão de mudanças	84
Validação de requisitos	86
Técnicas	86
Ferramentas	88
Documentação de requisitos	93
Normas técnicas	93
Estrutura-padrão (modelos de documentação)	102
Recapitulando	122
Metodologias ágeis e gestão de projetos	126
Metodologia SCRUM	127
Definição	127
Papéis e responsabilidades	127
Aplicação da gestão de projetos	129
Metodologia Kanban	131
Definição	131
Criação do quadro-base	132
Identificação do trabalho	134
Prioridades do trabalho	137
Gerenciamento do progresso e do desempenho	138
Encerramento	145

Fundamentos de requisitos

Apresentação

Seja bem-vindo(a) à Unidade Curricular de Levantamento de Requisitos!

Com ela, você descobrirá como entender profundamente as necessidades do cliente e traduzi-las em soluções práticas, transformando ideias em resultados concretos.

Dominar esses conceitos é essencial para qualquer profissional – que deseje criar produtos de qualidade, economizar recursos, evitar retrabalhos e oferecer valor real aos clientes – venha dar o próximo passo rumo ao sucesso na criação de projetos e soluções que realmente façam a diferença!

Definição

Um **requisito**, no contexto de desenvolvimento de sistemas, é uma **condição** ou **capacidade** as quais um sistema de software deve atender ou possuir para ser considerado útil e satisfatório para seus usuários e partes interessadas.

Pode-se entender requisito como:



Uma necessidade

Algo que o sistema precisa fazer ou possuir para resolver um problema ou atingir a um objetivo específico.



Uma expectativa

Uma característica ou funcionalidade que os usuários esperam que o sistema ofereça.



Uma restrição

Uma limitação ou condição que o sistema deve seguir, como requisitos de desempenho, segurança ou orçamento.

Em outras palavras, requisitos são as “regras do jogo” as quais definem o que o sistema deve ser e fazer. Eles são a base para todo o processo de desenvolvimento, guiando as decisões de design, implementação e testes.

É importante lembrar que os requisitos podem variar muito em natureza e granularidade.



Saiba mais

Granularidade é um termo usado para descrever o nível de detalhamento ou divisão de uma informação, processo ou estrutura. A palavra “grão” (ou “grânulo”) está na origem do termo e ajuda a entender: quanto mais “granular” algo, menor será a unidade considerada, ou seja, mais detalhada..

Alguns podem ser amplos e abstratos, como “o sistema deve ser fácil de usar”, enquanto outros podem ser detalhados e específicos, como “o sistema deve permitir o cadastro de clientes com nome, endereço e CPF”.

Por que entender a definição de requisito é tão importante?



Comunicação eficaz

Permite que todos os envolvidos no projeto (usuários, desenvolvedores, gerentes etc.) tenham um entendimento comum da função de um sistema.



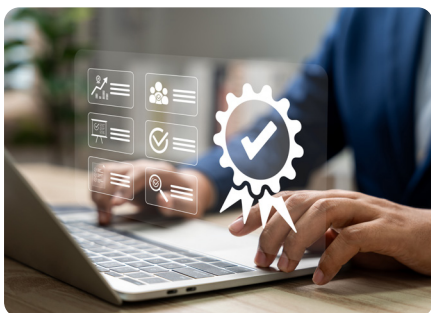
Planejamento preciso

Facilita a estimativa de tempo, recursos e custos necessários para o desenvolvimento do sistema.



Redução de retrabalho

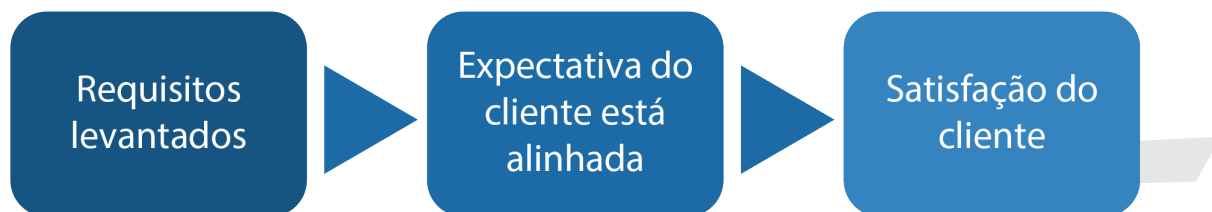
Evita que funcionalidades sejam desenvolvidas de forma inadequada ou que não atendam às necessidades dos usuários.



Garantia de qualidade

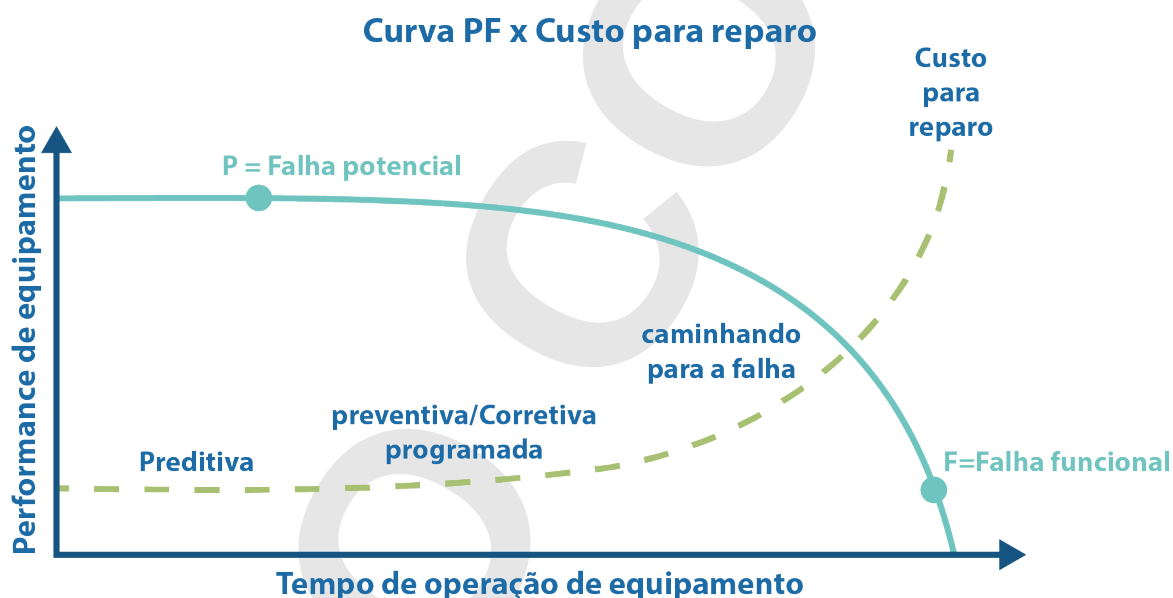
Ajuda a garantir que o sistema final atenda aos critérios de aceitação e satisfaça as expectativas dos usuários.

Agora que sabemos o que são requisitos e por que é muito importante conhecê-los. Vamos conhecer uma representação gráfica a qual contém uma linha de pensamento que expressa, por meio dos requisitos levantados, se está alinhada à expectativa do cliente, a fim de se atingir a satisfação dele de forma mais fácil.

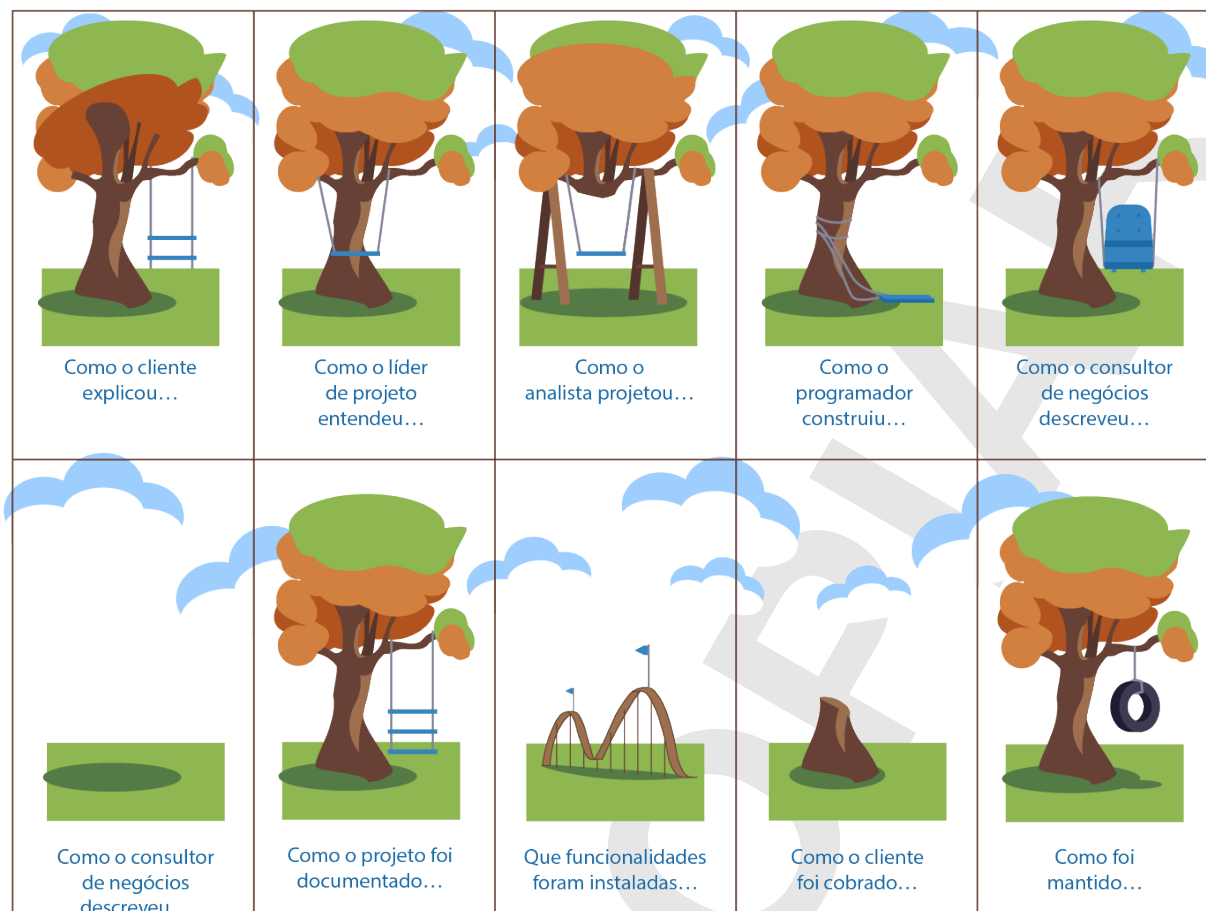


Portanto, está muito mais fácil de atingir a satisfação dele no final do projeto.

E para evidenciarmos ainda mais a importância dos requisitos, segue um gráfico que mostra o custo de correção de um problema no software em diferentes fases do projeto. A afirmação máxima a qual podemos obter – assim que olhamos para ele – é a de que, quanto antes localizarmos os problemas e repará-los menos custo se terá. Portanto, com requisitos bem definidos, desde o início, conseguimos resolver problemas antes mesmo que seja digitada qualquer linha de código.



A seguir, conheça um exemplo de como o levantamento de requisitos pode ser compreendido pelos envolvidos no projeto:



Aqui não foi levantado requisito algum, sendo assim, cada pessoa envolvida no projeto imaginou o software de um modo. Requisitos são importantes para que situações, como essa, não ocorram em nossos projetos.

Agora que temos uma definição clara do que é um requisito e sua importância, podemos nos aprofundar em outros aspectos importantes, tais quais os tipos de requisitos, as técnicas de levantamento e os desafios envolvidos.

A seguir e conheça mais sobre conceitos, definições, e a importância dos requisitos em projetos.

Fundamentos dos Requisitos

Objetivo da aula:

Apresentar os fundamentos dos requisitos de software, essenciais para o sucesso de projetos de automação, especialmente em ambientes rurais.

O que são requisitos?

Requisitos são condições ou capacidades que o sistema precisa atender para resolver um problema ou atingir um objetivo.

Tipos de requisitos:

- **Necessidade:** Ex. controlar irrigação por umidade do solo.
- **Expectativa:** Ex. visualizar dados em tempo real no celular.
- **Restrição:** Ex. funcionar mesmo com internet instável.

Por que requisitos são importantes?

- Tornam o projeto **concreto, direto e mensurável**.
- Evitam funções desconectadas ou sem utilidade prática.
- Permitem **planejamento realista** (tempo, orçamento, equipe).
- Reduzem retrabalho e atrasos.
- Garantem **qualidade e eficácia** nas funções do sistema.

Exemplos práticos de requisitos:

- Medir umidade do solo a cada 30 minutos e acionar irrigação.
- Rastrear animais via colar GPS.
- Alertar sobre velocidade de tratores.
- Controlar irrigação com previsão de chuvas.
- Acesso remoto às câmeras da fazenda pelo celular.
- Alertar se bomba de água ficar ligada por mais de 2 horas.

Requisitos ao longo do projeto:

- **Planejamento:** Definem sensores, interfaces, etc.
- **Implementação:** Guiam o que será codificado.

- **Testes:** Servem como checklist para validar funções.
- **Manutenção e expansão:** Facilitam inclusão de novas funcionalidades.

Riscos de requisitos mal definidos:

- Retrabalho por falta de cobertura de internet.
- Falhas por não prever chuvas ou ausência de GPS.
- Funções sem aplicação prática.
- Sistemas que não atendem à realidade do cliente.

Conclusão:

Requisitos são o **mapa do projeto**.

Com eles, é possível prever problemas, ajustar funções e garantir que tudo ocorra conforme o planejado.

Sem requisitos claros, o projeto vira um jogo de tentativa e erro — e isso custa tempo e dinheiro.

Modelos de documentação

A documentação dos requisitos é fundamental para garantir que todos os envolvidos no projeto compartilhem um entendimento claro do sistema a ser desenvolvido. Diversos modelos de documentos podem ser utilizados para registrar os requisitos, cada um com as próprias características e finalidades.

Veja nos cards a seguir para conhecer os principais modelos de documentos:



Especificação de Requisitos de Software (SRS)

É um documento formal que descreve detalhadamente todos os requisitos do sistema de software. Define o escopo do projeto e as funcionalidades do sistema, servindo como um contrato entre o cliente e a equipe de desenvolvimento.



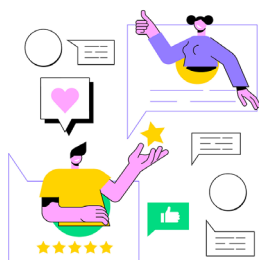
Documento de visão e escopo

É um documento mais conciso o qual descreve os principais objetivos e funcionalidades do sistema, servindo como um guia para o projeto e auxiliando a equipe a manter o foco.



Casos de uso

Descrevem as interações entre o usuário e o sistema, mostrando como o usuário utiliza o sistema para realizar tarefas específicas. São úteis para identificar e documentar os requisitos funcionais do sistema.



Histórias de usuário

São descrições curtas e informais das funcionalidades do sistema, escritas sob a perspectiva do usuário. Amplamente utilizadas em metodologias ágeis de desenvolvimento de software.

A escolha do modelo de documentação de requisitos depende de fatores, como complexidade do projeto e metodologia utilizada, sendo frequentemente benéfica a combinação de diferentes modelos. Independentemente da escolha, a documentação deve ser um processo contínuo e atualizado ao longo de todo o ciclo de desenvolvimento, garantindo uma visão completa e precisa dos requisitos do sistema.

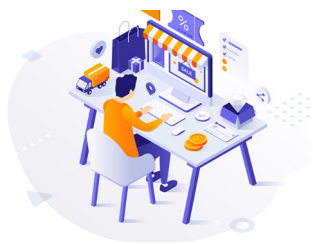
Regras de negócio

As regras de negócio são como leis que regem o funcionamento de uma empresa ou organização. Elas definem de que maneira os processos devem acontecer, quais são as condições e restrições e como as decisões devem ser tomadas.

No contexto do desenvolvimento de sistemas, as regras de negócio têm um impacto significativo nos requisitos, influenciando diretamente o que o sistema deve fazer e como deve se comportar.

Regra de negócio	Definição	Exemplos	
		Regra	Sistema
Definição de funcionalidades	Determinar quais funcionalidades o sistema deve ter.	Cliente com mais de 60 anos tem desconto de 10%.	Deve possuir uma função para calcular automaticamente esse desconto.
Restrições e condições	Impor restrições e condições a quais o sistema deve seguir.	Um produto só poderá ser comercializado se estiver em estoque.	Deve impedir a venda de produtos indisponíveis no estoque.
Cálculos e fórmulas	Envolve cálculos e fórmulas que o sistema deve executar.	Como calcular o valor do frete.	Deve calcular o valor do frete com base no peso do produto e distância para a entrega.
Tomada de decisões	Influenciar a forma como o sistema toma decisões.	Pedidos de compra só podem ser aprovados por um gerente.	Deve exigir a aprovação de um gerente antes de finalizar o pedido.

Com base na imagem a seguir, conheça mais exemplos práticos:



Sistema de e-commerce

Regras de negócio podem definir como:

- calcular o frete;
- métodos de pagamento que são aceitos;
- como aplicar descontos promocionais;
- regras para troca e devolução de produtos.



Sistema bancário

Regras de negócio podem definir como:

- calcular juros;
- limites de crédito para cada cliente;
- validar transações financeiras;
- procedimentos para abertura de contas.

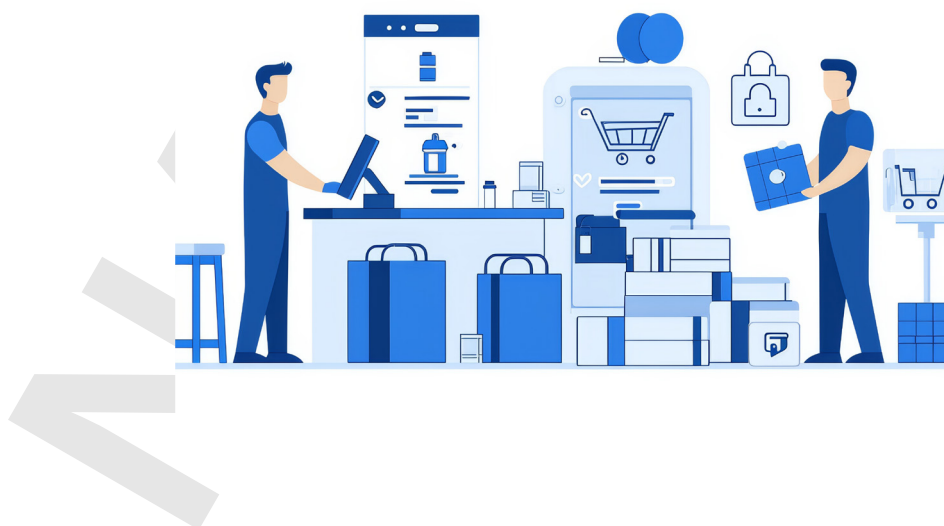


Sistema de gestão de estoque

Regras de negócio podem definir como:

- controlar o estoque de cada produto;
- gerar alertas de reposição;
- calcular o custo de cada item;
- critérios para recebimento e envio de mercadorias.

As regras de negócio são fundamentais no desenvolvimento de sistemas. Elas moldam os requisitos, influenciando funcionalidades, restrições, cálculos e decisões do sistema. Ao entender as regras de negócio, é possível criar sistemas mais eficientes, confiáveis e alinhados aos objetivos da organização.



Vamos supor que você esteja trabalhando para uma loja de e-commerce, desenvolvendo para eles um sistema o qual organiza anualmente todas as vendas e entrada de mercadorias. Abaixo segue alguns exemplos de regras de negócio que podem ser utilizadas dentro desse sistema.



A loja deve divulgar informações claras e detalhadas sobre a empresa, o produto ou serviço; Deve exibir o CNPJ da empresa ou o CPF da pessoa responsável; Deve informar todas as formas de pagamento e o prazo da entrega; O cliente tem direito a cancelar a compra sem necessidade de justificativa, no prazo de sete dias após a entrega do produto ou do serviço.

Restrições

No processo de dar vida a um sistema de software, os requisitos têm a função de um mapa que guia a equipe de desenvolvimento. No entanto, como em qualquer jornada, o levantamento de requisitos pode enfrentar obstáculos e restrições que exigem atenção e soluções estratégicas. Este capítulo sugere diversas restrições possíveis de surgirem durante essa etapa crucial, explorando seus impactos e oferecendo ferramentas para superá-las. A seguir, alguns dos tipos de restrições.



Restrições de tempo

O tempo é um recurso valioso em qualquer projeto. Restrições de tempo podem surgir devido a prazos apertados; necessidade de lançamento rápido do produto ou pressões do mercado. Essas restrições podem levar a decisões precipitadas, coleta incompleta de requisitos, comprometendo a qualidade do sistema.



Restrições de orçamento

O orçamento disponível para o projeto é um fator determinante. Restrições orçamentárias podem limitar a equipe de desenvolvimento, as ferramentas utilizadas, a profundidade da análise de requisitos e a realização de testes abrangentes. É crucial equilibrar as necessidades do sistema com as possibilidades financeiras.



Restrições de recursos humanos

A disponibilidade de profissionais qualificados e experientes é fundamental para o sucesso do levantamento de requisitos. Restrições de recursos humanos — como falta de pessoal, alta rotatividade ou equipes com expertise limitada — podem dificultar a coleta e a análise de requisitos, além de comprometer a comunicação entre as partes interessadas.



Restrições de conhecimento

O levantamento de requisitos exige um profundo entendimento do negócio do cliente, seus processos, suas regras e expectativas. Restrições de conhecimento — como falta de familiaridade com o domínio do problema — podem levar a requisitos incompletos, imprecisos ou desalinhados com as necessidades reais do cliente.



Restrições de comunicação

A comunicação eficaz entre as partes interessadas é essencial para o sucesso do levantamento de requisitos. Restrições de comunicação — como barreiras de linguagem, diferentes níveis de conhecimento técnico, falta de tempo para reuniões ou ferramentas de comunicação inadequadas — podem gerar ruídos, mal-entendidos e comprometer a qualidade dos requisitos.



Restrições tecnológicas

As tecnologias disponíveis para o desenvolvimento do sistema podem impor limitações ao levantamento de requisitos. Restrições tecnológicas – como softwares legados, infraestrutura inadequada, falta de conhecimento sobre novas tecnologias ou incompatibilidade entre sistemas – podem influenciar a definição de funcionalidades, desempenho e outros aspectos do sistema.



Restrições políticas e culturais

Fatores políticos e culturais, no interior da organização do cliente, podem afetar o levantamento de requisitos. Restrições políticas – como jogos de poder, resistência a mudanças ou falta de apoio da alta gerência – podem dificultar a coleta de informações e a tomada de decisões. Restrições culturais – como diferentes valores, crenças ou formas de comunicação, podem gerar conflitos e dificultar o entendimento das necessidades do cliente.

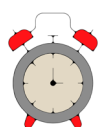


Restrições legais e regulatórias

O sistema de software deve estar em conformidade com leis e regulamentos específicos do setor em que opera. Restrições legais e regulatórias – como leis de proteção de dados, normas de segurança ou regulamentações específicas da indústria – podem influenciar a definição de requisitos, funcionalidades e outros aspectos do sistema.

Superando as restrições

Apesar dos desafios, é possível superar as restrições no levantamento de requisitos. Conheça abaixo algumas estratégias:



Gerenciamento de tempo

Priorizar atividades, definir prazos realistas e utilizar ferramentas de gerenciamento de projetos.

Exemplo: priorizar funcionalidades essenciais para o MVP em aplicativos de entrega com prazos apertados.



Planejamento orçamentário

Definir um orçamento detalhado, priorizar funcionalidades e buscar soluções criativas e eficientes.

Exemplo: utilizar ferramentas de código aberto e priorizar funcionalidades de alto valor em startups com orçamentos limitados.



Alocação de recursos

Contratar profissionais qualificados, investir em treinamento e buscar apoio externo, se necessário..

Exemplo: investir em treinamento e consultoria para superar a falta de profissionais qualificados em hospitais.



Comunicação eficaz

Utilizar diferentes canais de comunicação, promover reuniões regulares e criar um ambiente de confiança e colaboração.

Exemplo: usar ferramentas online e reuniões regulares para alinhar equipes globais em projetos de e-learning.



Gerenciamento de riscos

Identificar e analisar os riscos, desenvolver planos de contingência e monitorar o progresso do projeto.

Exemplo: Implementar criptografia e testes de segurança para mitigar riscos em sistemas bancários online.

Ao reconhecer e abordar as restrições no levantamento de requisitos, as equipes de desenvolvimento podem aumentar as chances de sucesso do projeto, garantindo que o sistema atenda às necessidades do cliente e entregue valor ao negócio.



Restrições de desenvolvimento

- O orçamento (valor disponível para o projeto) não pode ultrapassar R\$ 20.000,00.
- O software não pode utilizar a cor vermelha, por ser a cor do concorrente.
- O software deve ser entregue em até um ano após a assinatura do contrato.

O levantamento de requisitos é um processo colaborativo, iterativo e dinâmico que exige a participação ativa de todos os envolvidos. A atenção constante às restrições, a adaptação ágil das estratégias e a busca conjunta por soluções criativas são os ingredientes essenciais para transformar a visão em um sistema final de sucesso.

Tipos de requisitos

No processo de construção de um sistema de software, os requisitos surgem tal qual uma bússola que orienta cada passo. Eles representam as necessidades, expectativas e restrições as quais moldam o produto final. No entanto, nem todos os requisitos são iguais. Eles se manifestam de diversas formas, cada uma com seu papel essencial na definição do comportamento e das características do sistema.

Os requisitos podem ser classificados de variadas maneiras, dependendo da perspectiva e do objetivo da análise. Algumas classificações comuns incluem:



Requisitos de negócio

Descrevem os objetivos estratégicos da organização e como o sistema contribuirá para alcançá-los.



Requisitos de usuário

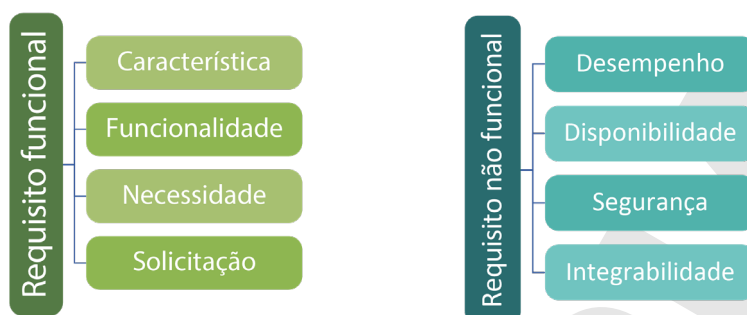
Expressam as necessidades e expectativas dos usuários finais, descrevendo o que eles desejam realizar com o sistema.



Requisitos de sistema

Detalham as funcionalidades e características técnicas do sistema, especificando como ele deve ser construído e implementado.

A seguir, exploraremos os dois principais tipos de requisitos: funcionais e não funcionais os quais juntos fornecem uma visão completa das necessidades do sistema e como atendê-las. A seguir, um esquema que auxilia na diferenciação de cada um.



Vamos supor que você esteja trabalhando para uma loja de e-commerce, desenvolvendo para eles um sistema o qual organiza anualmente todas as vendas e entrada de mercadorias. Abaixo segue alguns exemplos de regras de negócio que podem ser utilizadas dentro desse sistema.

Requisitos funcionais

Os requisitos funcionais definem as ações e operações as quais o sistema deve executar. Eles descrevem o que o sistema faz, suas funcionalidades e comportamentos específicos.

Os requisitos funcionais são o coração do sistema. Para facilitar a compreensão, vamos explorar alguns exemplos detalhados:

- **Navegação de produtos**

O sistema deve permitir que usuários naveguem pelo catálogo de produtos, visualizando informações detalhadas, como descrição completa, preço em diferentes moedas e fotos em alta resolução com zoom.

- **Carrinho de compras**

O sistema deve permitir que usuários adicionem produtos ao carrinho de compras, ajustem quantidades em unidades ou quilos e removam itens com facilidade, exibindo o subtotal em tempo real.

- **Cálculo de pedidos**
O sistema deve calcular automaticamente o valor total do pedido, incluindo impostos (ICMS, IPI), taxas de envio com base no CEP e descontos promocionais, o exibindo detalhadamente para o cliente.
- **Integração de pagamentos**
O sistema deve integrar-se a múltiplos gateways de pagamento (cartão de crédito, boleto, Pix, carteiras digitais) para processar transações financeiras com segurança, utilizando criptografia e autenticação de dois fatores.
- **Notificações**
O sistema deve gerar notificações por e-mail e SMS para o cliente à cada etapa da compra, como confirmação de pagamento, envio do produto com código de rastreamento e previsão de entrega.

Requisitos funcionais para sistema de gerenciamento de projetos:

- **Gerenciamento de projetos**
O sistema deve permitir a criação de projetos com definição de metas SMART, atribuição de tarefas com prazos e responsáveis, e acompanhamento do progresso por meio de gráficos de Gantt e Kanban.
- **Relatórios e gráficos**
O sistema deve gerar gráficos e relatórios personalizáveis que visualizem o status do projeto, o desempenho individual e da equipe, os prazos cumpridos e os custos envolvidos.
- **Colaboração**
O sistema deve permitir o compartilhamento de arquivos em diferentes formatos (PDF, DOCX, planilhas) e a comunicação entre os membros da equipe por meio de chats e comentários em tarefas.
- **Controle de acesso**
O sistema deve permitir o controle de acesso granular de cada membro da equipe, com diferentes níveis de permissão a fim de visualizar, editar e gerenciar projetos e tarefas.

Requisitos funcionais para sistema de gerenciamento de biblioteca:

- **Cadastro de livros**
O sistema deve permitir o cadastro de novos livros com informações detalhadas, como título, autor, editora, ISBN, gênero, sinopse e capa em alta resolução.
- **Cadastro de usuários**
O sistema deve permitir o cadastro de novos usuários com informações completas, como nome completo, endereço, número de matrícula, e-mail e telefone(s) para contato.
- **Empréstimos e devoluções**
O sistema deve permitir a realização de empréstimos e devoluções de livros com controle de prazos, multas por tempo de atraso e histórico de empréstimos do usuário.
- **Relatórios**
O sistema deve gerar relatórios personalizados de livros mais populares por gênero, autores com mais empréstimos, usuários com entregas de livros atrasadas e histórico de empréstimos por usuário.
- **Pesquisa de livros**
O sistema deve permitir a pesquisa de livros por título, autor, assunto, palavra-chave e ISBN, com filtros avançados por gênero, editora e ano de publicação.

Ao definir requisitos funcionais, é importante utilizar uma linguagem clara, precisa e livre de ambiguidades. Cada requisito deve ser específico, mensurável, alcançável, relevante e temporal (SMART). Isso garante que a equipe de desenvolvimento compreenda exatamente o que o sistema deve fazer e evita interpretações equivocadas.

Não funcionais

Os requisitos não funcionais descrevem as características e qualidades do sistema, tais quais desempenho, segurança, usabilidade e confiabilidade. Eles definem de que modo o sistema se comporta, suas restrições e atributos de qualidade.

Os requisitos não funcionais – embora muitas vezes menos visíveis que os funcionais – são essenciais para a qualidade e o sucesso do software.

Analisemos alguns exemplos detalhados.

Requisitos não funcionais para sistema de e-commerce:

- **Desempenho**
Em vez de apenas carregar as páginas em menos de 3 segundos, adicionar um contexto específico, como a página de detalhes do produto deve carregar em menos de 2 segundos a fim de garantir uma experiência rápida de compra.
- **Escalabilidade**
Especificar o tipo de usuários, como o sistema deve ser capaz de lidar com 1000 usuários simultâneos, navegando pelos produtos e 500 realizando transações sem degradação do desempenho.
- **Disponibilidade**
Adicionar um contexto, como o sistema deve estar disponível 24 horas por dia, 7 dias por semana, com no máximo 1 hora de inatividade por mês para manutenção programada, a ser realizada, preferencialmente, durante a madrugada.
- **Segurança**
Especificar os tipos de dados, como o sistema deve criptografar todas as informações sensíveis – como dados de pagamento (número do cartão, CVV) e informações pessoais do cliente (endereço, CPF).
- **Compatibilidade**
Adicionar versões específicas, como o sistema deve ser compatível com as últimas duas versões dos navegadores Chrome, Firefox e Safari.
- **Usabilidade e acessibilidade**
Adicionar versões específicas, como o sistema deve ser compatível com as últimas duas versões dos navegadores Chrome, Firefox e Safari.

Requisitos não funcionais para sistema de gerenciamento de projetos:

- **Mobilidade**
Especificar os sistemas operacionais, como o sistema deve ser acessível em dispositivos móveis, como smartphones e tablets com sistemas operacionais Android e iOS.
- **Confiabilidade**
Adicionar detalhes, como o sistema deve realizar backups automáticos diariamente, armazenando os dados em um servidor externo a fim de garantir a recuperação em caso de falhas.

- **Desempenho**
Especificar as operações, como o sistema deve possuir um tempo de resposta inferior a um segundo para a maioria das operações – como criação de tarefas, atualização de status e geração de relatórios.
- **Conformidade**
Especificar os tipos de dados, como o sistema deve estar em conformidade com a LGPD, garantindo a segurança e a privacidade dos dados dos usuários e projetos.
- **Interoperabilidade**
Especificar as plataformas, como o sistema deve permitir a integração com outras ferramentas, como plataformas de comunicação (Slack, Microsoft Teams) e calendários (Google Calendar, Outlook Calendar).
- **Manutenibilidade**
Adicionar detalhes, como a documentação do sistema deve estar completada e atualizada, incluindo diagramas, manuais e guias de solução de problemas – facilitando, assim, o aprendizado, a utilização e a manutenção do sistema.

[▶ vídeo resumido](#)

Tipos de Requisitos: Funcionais e Não Funcionais

Objetivo da aula:

Compreender os tipos de requisitos mais importantes no desenvolvimento de sistemas de automação, especialmente no agronegócio: funcionais e não funcionais.

Requisitos Funcionais

Descrevem o que o sistema deve fazer para atender às necessidades do cliente.

Exemplos:

- Ativar irrigação automaticamente quando a umidade estiver abaixo de 30%.
- Agendar irrigação por setor da fazenda.
- Registrar localização e trajeto de tratores com GPS.
- Emitir relatórios de manutenção com base em horas de uso.
- Registrar volume de leite ordenhado por vaca ou turno.
- Bloquear ordenha em caso de suspeita de mastite.

Requisitos Não Funcionais

Definem como o sistema deve se comportar, garantindo qualidade, segurança e desempenho.

Exemplos por categoria:

- **Desempenho:** Carregar sistema em menos de 2 segundos em conexão 4G rural.
- **Segurança:** Criptografar dados durante transmissão e armazenamento.
- **Confiabilidade:** Retomar operação após queda de energia.
- **Disponibilidade:** Operar 24/7 com no máximo 30 min de inatividade por mês.
- **Escalabilidade:** Suportar até 500 sensores sem perda de desempenho.
- **Usabilidade:** Interface intuitiva, acessível mesmo para operadores com pouca familiaridade com tecnologia.

Exemplo aplicado: Sistema de climatização de estufas

Funcionais:

- Medir temperatura e umidade.
- Ativar exaustores.
- Enviar alertas de superaquecimento.

Não Funcionais:

- Interface simples para uso com luvas.
- Resposta em até 1 segundo aos sensores.
- Operação offline com armazenamento local.

Como levantar requisitos com qualidade

Técnicas recomendadas:

- **Entrevistas:** Perguntar o que falta e o que esperam dos sistemas.
- **Observação:** Visitar o campo e acompanhar rotinas.
- **Workshops:** Reunir operadores, engenheiros e desenvolvedores para pensar juntos.

Cuidados importantes

- Não ignorar requisitos não funcionais como desempenho e segurança.
- Evitar definições vagas como “rápido” ou “fácil de usar”.

- Detalhar com precisão: ex. "carregar dados em até 3 segundos com conexão instável".

Conclusão

Um bom sistema precisa de equilíbrio entre **o que faz** e **como faz**.

Requisitos bem definidos garantem confiabilidade, produtividade e eficiência — especialmente em ambientes complexos como o agronegócio.

É importante lembrar que os requisitos funcionais e não funcionais estão interligados. Os requisitos funcionais definem o que o sistema deve fazer, enquanto os requisitos não funcionais, definem como deve ser feito. Por exemplo, pelo requisito funcional "o sistema deve permitir o cadastro de novos clientes" pode ser complementado pelo requisito não funcional "o sistema deve garantir que o cadastro seja realizado em menos de 5 segundos".



Agora vamos conhecer uma aplicação do que aprendemos, até o momento, sobre requisitos funcionais:



- O sistema de irrigação deve ser acionado de 8 em 8 horas, caso a umidade do solo esteja abaixo de 30%.
- O sistema de dispensa de alimento para o gado deve liberar o alimento todos os dias às 12 horas e às 18 horas.

Além deles poderíamos ter os seguintes requisitos não funcionais:



- O sistema de irrigação deve finalizar a umidificação em menos de 10 minutos
- O sistema de dispensa de alimentos deve permanecer funcionando sem obstruções em 99% do tempo.

As seguintes restrições poderiam ser levantadas no levantamento de requisitos:



- O sistema de irrigação deve ser suspenso caso esteja chovendo.
- A tela do sistema de dispensa de alimentos deve conter poucos botões a fim de facilitar a utilização por parte do fazendeiro.



Pense nisso

Conseguem imaginar outros requisitos funcionais, não funcionais ou restrições que possam ser aplicadas no contexto?

Recapitulando

Nessa aula, exploramos o conceito fundamental de Requisitos de Software, destacando sua importância para o desenvolvimento de sistemas eficazes e alinhados às necessidades do cliente. Aprofundamos a definição de requisitos, distinguindo entre funcionais, que descrevem as funcionalidades do sistema, e não funcionais, que definem características de qualidade e desempenho. Abordamos também

regras de negócio as quais orientam as políticas da organização, e restrições que limitam as opções de desenvolvimento.

Compreender os diferentes tipos de requisitos – desde o "o quê" do sistema (funcionais) até o "como" (não funcionais), considerando regras de negócio e restrições – é essencial para construir um software completo e de qualidade. Esses conhecimentos os quais formam a base para o sucesso, nos próximos desafios, devem ser aplicados em cada conceito, distinção e tipo de restrição no próximo capítulo, no qual exploraremos técnicas e estratégias avançadas para coletar informações precisas e completas das partes interessadas, construindo uma base sólida para um software que atenda às reais necessidades do cliente.

Na próxima aula, nos aprofundaremos no processo de levantamento de requisitos, explorando técnicas e estratégias para coletar informações precisas e completas das partes interessadas, construindo uma base sólida para um software que atenda às reais necessidades do cliente.

Técnicas e processos de levantamento de requisitos

Apresentação

Nesta aula, vamos explorar o processo de **levantamento de requisitos** e entender por que ele é fundamental para definir com precisão as necessidades dos usuários e demais partes interessadas. Na aula anterior, discutimos o que são requisitos; neste momento, avançaremos para conhecer as principais ferramentas que nos ajudam a identificá-los de forma eficaz.

Vamos iniciar com as **técnicas de coleta de informações** – como briefing, levantamento orientado a pontos de vista, etnografia (observação), entrevistas, brainstorming e prototipação. Veremos de que modo cada uma dessas abordagens pode ser aplicada a fim de identificar tanto **requisitos funcionais** quanto **não funcionais**.

Na sequência, vamos **analisar as etapas do processo** de levantamento: coleta, análise e registro dos requisitos. Por fim, abordaremos a priorização, explorando métodos, como MoSCoW, matriz valor versus esforço e pontuação por critérios, que auxiliam na definição do que deve ser entregue primeiro.

Com esse conjunto de técnicas e estratégias, a equipe de desenvolvimento estará mais bem preparada para compreender o que é realmente essencial no projeto, reduzindo riscos de retrabalho e aumentando as chances de entregar soluções alinhadas às expectativas dos clientes.

Definição

O levantamento de requisitos é o processo de identificar, documentar e validar as necessidades e expectativas dos usuários e partes interessadas em relação a um sistema de software que será desenvolvido. É uma etapa essencial no ciclo de vida do desenvolvimento de software, pois define o escopo do projeto e as funcionalidades que o sistema deve possuir. Por exemplo, em um sistema de gerenciamento de horas de trabalho, um requisito poderia ser: "O sistema deve permitir que os funcionários registrem suas horas trabalhadas diariamente, incluindo o horário de início e término de cada atividade".

Em outras palavras, o levantamento de requisitos é o ato de ouvir os usuários e transformar suas necessidades em especificações claras e detalhadas que guiarão o desenvolvimento do sistema. É como traduzir a linguagem dos usuários para a linguagem dos desenvolvedores, garantindo assim que todos estejam na mesma página e que o sistema final atenda às reais necessidades daqueles que o utilizarão.



Garantia de que o sistema atenda às necessidades dos usuários

Ao entender as necessidades dos usuários, se torna possível construir um sistema que realmente resolva seus problemas e atenda às suas expectativas.



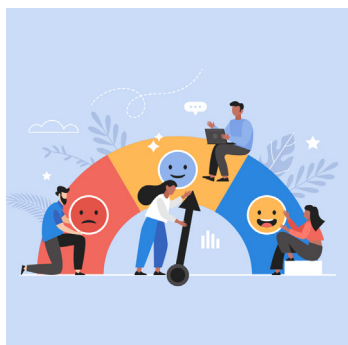
Redução de retrabalho e custos

Um levantamento de requisitos bem executado evita que funcionalidades sejam desenvolvidas de forma inadequada ou que não atendam às necessidades dos usuários, economizando tempo e dinheiro.



Melhora na comunicação entre as partes interessadas

O levantamento de requisitos facilita a comunicação entre os usuários, os desenvolvedores e as outras partes interessadas, garantindo que todos tenham um entendimento comum do projeto.



Aumento da satisfação do cliente

Um sistema que atende às necessidades dos usuários e é entregue dentro do prazo e do orçamento tende a gerar maior satisfação do cliente.

Técnicas

Diversas técnicas podem ser empregadas para levantar requisitos, cada uma com suas vantagens e desvantagens. A escolha da técnica mais adequada depende do contexto do projeto, das características das partes interessadas e da natureza do sistema. Frequentemente, uma combinação de técnicas é utilizada para obter uma visão mais completa.

Briefing

O briefing é o primeiro encontro do detetive com o cliente. É realizada uma reunião inicial para levantamentos de dados, nos quais o cliente apresenta suas necessidades, expectativas e objetivos para o sistema. O briefing é uma oportunidade para o detetive (você!) formular perguntas, esclarecer dúvidas e coletar informações importantes sobre o projeto. Lembre-se sempre de documentar as informações coletadas.

Exemplo Prático: Briefing para o Aplicativo de Lista de Tarefas do Escritório de Advocacia

Veja nos quadros a seguir para conhecer cada etapa do passo-a-passo.

Passo 1

Preparação (antes do briefing)



Definir os objetivos do briefing

- Compreender as necessidades do escritório em relação ao aplicativo.
- Identificar os principais usuários e suas expectativas.
- Definir o escopo inicial do projeto.



Preparar um roteiro de perguntas

- Quais problemas o escritório espera resolver com o aplicativo?
- Quais funcionalidades são essenciais?
- Quem serão os principais usuários?
- Quais são as expectativas em relação à usabilidade e desempenho?



Reunir informações preliminares

- Pesquisar sobre o funcionamento de escritórios de advocacia.
- Analisar aplicativos de lista de tarefas existentes.

Passo 2

Condução do briefing (durante a reunião)



Apresentação e quebra-gelo

- Apresentar-se e explicar o objetivo do briefing.
- Criar um ambiente confortável para o cliente expressar suas ideias.



Apresentação do projeto (pelo cliente)

- Solicitar ao cliente que apresente o escritório e suas atividades.
- Ouvir atentamente e fazer anotações.



Definição do escopo inicial

- Resumir as principais funcionalidades e objetivos do aplicativo.
- Confirmar o escopo inicial com o cliente.



Encerramento e próximos passos

- Agradecer a participação do cliente.
- Informar sobre os próximos passos, como a elaboração do documento de requisitos.

Passo 3

Documentação e acompanhamento (após o briefing)



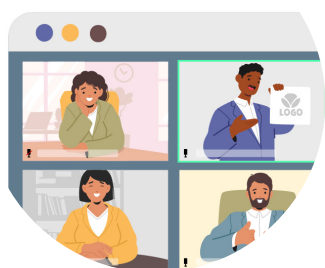
Documentar as informações coletadas

- Organizar as anotações e criar um resumo do briefing.
- Identificar os principais requisitos e expectativas do cliente.



Enviar o resumo para o cliente

- Solicitar a revisão e aprovação do resumo.



Agendar reuniões de acompanhamento

- Manter o cliente informado sobre o progresso do projeto.
- Esclarecer dúvidas e ajustar os requisitos conforme necessário.

Para resumir as etapas e objetivos do briefing observe o esquema a seguir:



Entender as necessidades e requisitos do cliente;



Educar o cliente para que de fato compreenda o que ele está contratando;



Traduzir as expectativas do cliente em entregas mensuráveis;



Planejar o fluxo do projeto dentro do prazo e do orçamento previstos;



Evitar desperdício com tarefas desnecessárias;



Conduzir o projeto de acordo com a estratégia de negócio do cliente.

Leia a seguir o resumo do vídeo em que o professor preenche um briefing entrevistando um cliente.

 **vídeo resumido**

Preenchendo um Briefing com o cliente

Objetivo da aula:

Demonstrar como preencher um **briefing** de forma eficaz, usando perguntas estratégicas para levantar requisitos e entender o problema do cliente antes de propor soluções.

Por que o briefing é essencial?

- O cliente nem sempre sabe exatamente o que precisa.
- O briefing ajuda a traduzir o problema em requisitos funcionais e não funcionais.
- Evita retrabalho, mal-entendidos e soluções inadequadas.

Exemplo prático de briefing

Contexto:

Empresa com problema de organização — excesso de post-its, perda de informações, confusão no escritório.

Informações levantadas:

- **Público:** funcionários entre 30 e 45 anos, incluindo pessoas com baixa visão.
- **Objetivo do sistema:** organizar anotações dos colaboradores.
- **Uso:** em reuniões e durante toda a jornada de trabalho.
- **Requisitos funcionais sugeridos:**
 - Compartilhamento de anotações em tempo real.
- **Requisitos não funcionais sugeridos:**
 - Disponibilidade contínua durante o expediente.
 - Compatibilidade com leitores de tela.

Valores destacados:

- Acessibilidade
- Segurança
- Trabalho em equipe

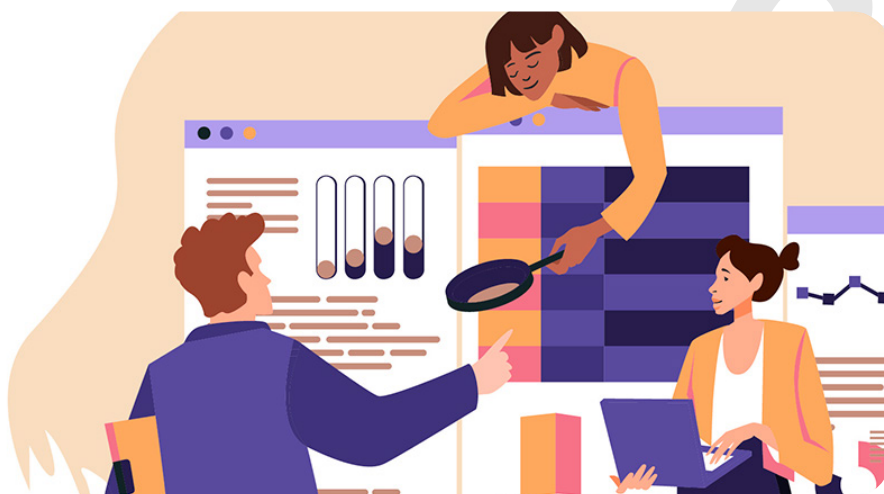
Levantamento Orientado a Pontos de Vista

Assim como um detetive ouve diferentes testemunhas para montar o seu quebra-cabeça, o levantamento orientado a pontos de vista envolve a coleta de requisitos de diferentes partes interessadas. Cada pessoa tem a própria perspectiva sobre o sistema, e é importante considerar todas elas para garantir que o software atenda às necessidades de todos.

Exemplo Prático: levantamento orientado a pontos de vista para o aplicativo de lista de tarefas.

Passo 1

Identificação dos Pontos de Vista (stakeholders)



Advogados :

- Usuários principais do aplicativo os quais criarão e gerenciarão tarefas.
- Interessados em funcionalidades que otimizem a organização e o acompanhamento de casos.

Assistentes jurídicos :

- Responsáveis por auxiliar os advogados na execução das tarefas.
- Interessados em funcionalidades que facilitem a comunicação e a colaboração.

Gerente do escritório :

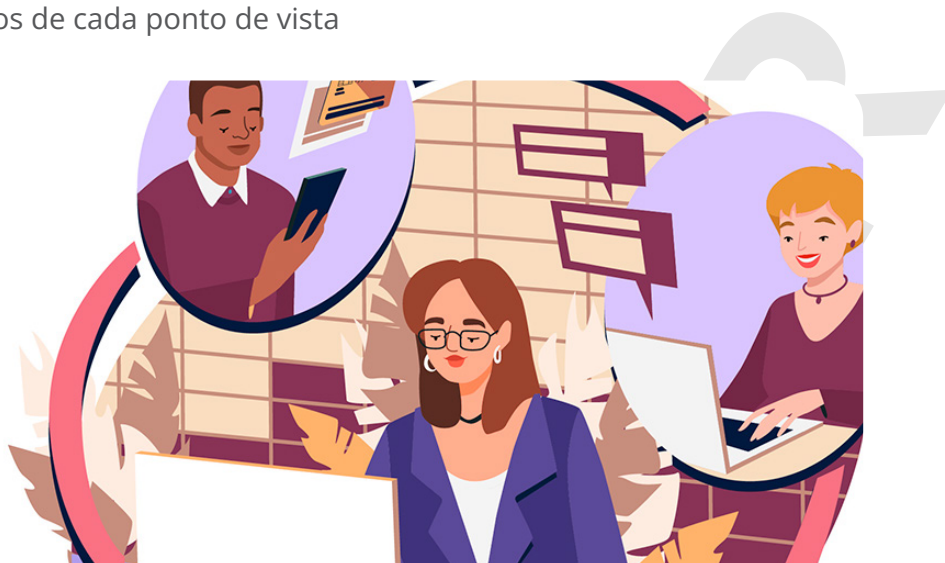
- Responsável por garantir a eficiência e a produtividade do escritório.
- Interessado em funcionalidades que permitam o acompanhamento do desempenho da equipe.

Equipe de TI :

- Responsável pela infraestrutura e suporte técnico do escritório.
- Interessada em requisitos não funcionais, como segurança e desempenho.

Passo 2

Coleta de requisitos de cada ponto de vista



Equipe toda:

- Entrevistas individuais para entender suas necessidades e expectativas.
- Questionários para coletar informações sobre as tarefas que realizam e os problemas que enfrentam.

Advogados:

- Análise de documentos e fluxos de trabalho para identificar as necessidades do dia a dia.

Assistentes jurídicos:

- Observação direta do trabalho dos assistentes para identificar as necessidades do dia a dia.

Gerente do escritório:

- Análise de relatórios e indicadores de desempenho para identificar as necessidades do dia a dia.

Equipe de TI:

- Reuniões para discutir os requisitos não funcionais, como segurança, desempenho e integração com outros sistemas.
- Análise da infraestrutura existente para identificar as necessidades de hardware e software.

Passo 3

Consolidação e análise dos requisitos



- Organizar os requisitos coletados por ponto de vista.
- Identificar os requisitos comuns e os requisitos específicos de cada ponto de vista.
- Analisar os conflitos e as dependências entre os requisitos.
- Priorizar os requisitos com base na importância e na viabilidade.

Passo 4

Validação dos requisitos com os stakeholders



- Apresentar os requisitos consolidados para cada ponto de vista.
- Solicitar feedback e aprovação dos requisitos.
- Realizar ajustes nos requisitos com base no feedback dos stakeholders.

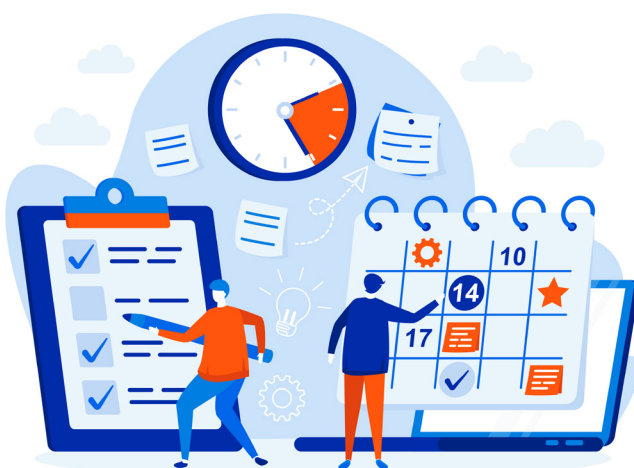
Etnografia (observação)

A etnografia se iguala a um detetive que observa o comportamento dos suspeitos a fim de encontrar pistas. Nessa técnica, o analista observa de que modo os usuários interagem com o sistema ou com processos relacionados ao sistema. A observação pode revelar informações importantes que os usuários podem não mencionar em entrevistas ou questionários.

Exemplo prático: etnografia em um escritório de advocacia.

Passo 1

Planejamento da observação:



Definir os objetivos:

- Compreender como os advogados e assistentes jurídicos gerenciam suas tarefas atualmente.
- Identificar os pontos de dor e as ineficiências nos processos existentes.
- Observar as interações entre os membros da equipe e o uso de ferramentas existentes.

Definir o que observar:

- De que maneira as tarefas são atribuídas e acompanhadas.
- De que forma os prazos são gerenciados.
- Como a comunicação entre os membros da equipe ocorre.
- O uso de ferramentas como agendas, e-mails e planilhas.

Definir o local e o período da observação:

- Observar o trabalho dos advogados e assistentes jurídicos em seus locais de trabalho.

- Realizar observações em diferentes horários do dia a fim de capturar a rotina completa.

Definir o papel do observador:

- Observação participante: o analista interage com os usuários e participa das atividades.
- Observação não participante: o analista observa, de forma discreta, sem sua interferência.

Passo 2

Realização da observação.



Observar as atividades dos usuários:

- Acompanhar o fluxo de trabalho dos advogados e assistentes jurídicos.
- Observar como eles utilizam as ferramentas existentes.
- Prestar atenção nas interações entre os membros da equipe.

Registrar as observações:

- Utilizar anotações, fotos e vídeos para registrar as informações coletadas.
- Documentar os comportamentos, as interações e o uso de ferramentas.

Manter a discrição:

Evitar interferir nas atividades dos usuários.

Garantir que os usuários se sintam confortáveis com a presença do observador.

Passo 3

Análise e Interpretação dos Dados



Organizar as informações coletadas:

- Agrupar as observações por tema ou categoria.
- Criar diagramas e fluxogramas para visualizar os processos.

Identificar padrões e insights:

- Analisar as informações coletadas para identificar os principais pontos de dor e as ineficiências.
- Identificar as necessidades e expectativas dos usuários que não foram mencionadas em entrevistas ou questionários.
- Identificar as melhores práticas que podem ser incorporadas ao novo sistema.

Validar os insights:

- Apresentar os insights para os usuários e solicitar feedback.
- Realizar ajustes nos insights com base no feedback dos usuários.

Entrevistas

As entrevistas são uma das técnicas mais comuns de levantamento de requisitos. O analista conversa com os usuários e as outras partes interessadas para coletar informações sobre suas necessidades e expectativas. As entrevistas podem ser estruturadas, com perguntas pré-definidas, ou não estruturadas, com conversas mais livres.

Exemplo prático: entrevistas para o aplicativo de lista de tarefas.

Passo 1

Preparação da Entrevista



Definir os objetivos da entrevista:

- Entender as necessidades e expectativas do entrevistado em relação ao aplicativo.
- Coletar informações detalhadas sobre suas tarefas e fluxos de trabalho.
- Identificar os problemas e desafios que o entrevistado enfrenta no dia a dia.

Selecionar os entrevistados:

- Advogados, assistentes jurídicos e gerente do escritório.

Definir o tipo de entrevista:

- Entrevistas estruturadas para coletar informações específicas.
- Entrevistas não estruturadas para explorar ideias e opiniões.

Preparar um roteiro de perguntas:

- Perguntas abertas para estimular o entrevistado a compartilhar informações.
- Perguntas fechadas para obter respostas específicas.
- Perguntas de acompanhamento para aprofundar a conversa.

Agendar as entrevistas:

- Escolher um local e horário convenientes para o entrevistado.
- Informar o entrevistado sobre o objetivo da entrevista e o tempo de duração estimado.

Passo 2

Condução da entrevista



Criar um ambiente confortável:

- Começar com uma conversa informal para quebrar o gelo.
- Explicar o objetivo da entrevista e garantir a confidencialidade das informações.

Conduzir a entrevista:

- Utilizar o roteiro de perguntas como guia, mas permitindo que a conversa flua naturalmente.
- Formular perguntas abertas a fim de estimular o entrevistado a compartilhar suas experiências.

Ouvir atentamente e fazer anotações.

- Esclarecer dúvidas e confirmar a compreensão das respostas.

Adaptar o tipo de entrevista:

- Em entrevistas estruturadas, seguir o roteiro de perguntas.
- Em entrevistas não estruturadas, permitir que a conversa explore os tópicos relevantes.

Encerramento da entrevista:

- Agradecer a participação do entrevistado.

Informar sobre os próximos passos, como a análise das informações coletadas.

Passo 3:

Documentação e análise



Transcrever as entrevistas:

- Documentar as informações coletadas de forma clara e organizada.

Analisar as informações:

- Identificar os principais requisitos e expectativas dos entrevistados.
- Comparar as informações coletadas de diferentes entrevistados.
- Identificar os conflitos e as dependências entre os requisitos.

Validar as informações:

- Apresentar os requisitos identificados para os entrevistados.
- Solicitar feedback e aprovação dos requisitos.

Brainstorming

O brainstorming é uma técnica de grupo na qual os participantes geram ideias livremente, sem julgamentos ou críticas. O objetivo é gerar o maior número possível de ideias as quais podem ser analisadas e refinadas posteriormente. O brainstorming é útil para identificar requisitos inovadores e soluções criativas.

Exemplo prático: brainstorming para o aplicativo de lista de tarefas.

Passo 1

Preparação da sessão de brainstorming

Definir o objetivo

- Gerar ideias para funcionalidades inovadoras e criativas para o aplicativo de lista de tarefas.
- Explorar diferentes abordagens para resolver os problemas e desafios dos usuários.

Selecionar os participantes

- Advogados, assistentes jurídicos, gerente do escritório e equipe de TI (se houver).
- Incluir pessoas com diferentes perspectivas e experiências.

Definir as regras

- Todas as ideias são válidas, sem julgamentos ou críticas.
- Incentivar a geração de ideias em grande quantidade.
- Construir sobre as ideias dos outros participantes.
- Manter o foco no objetivo da sessão.

Preparar o ambiente

- Escolher um local confortável e inspirador.
- Disponibilizar materiais – como quadro branco, canetas e post-its.

Passo 2

Condução da sessão de brainstorming

Apresentar o objetivo

- Explicar o objetivo da sessão e as regras de brainstorming.

Aquecimento

- Realizar um breve exercício de aquecimento para estimular a criatividade.

Geração de ideias

- Solicitar aos participantes que gerem ideias livremente.
- Registrar todas as ideias no quadro branco ou em post-its.

- Incentivar a participação de todos os participantes.

Construção de ideias

Incentivar os participantes a construir sobre as ideias dos outros.

- Combinar ideias e explorar novas possibilidades.

Encerramento da sessão

- Agradecer a participação de todos os participantes.
- Revisar as ideias geradas e identificar os principais temas.

Passo 3

Análise e refinamento das ideias

Organizar as ideias

- Agrupar as ideias por tema ou categoria.
- Eliminar as ideias repetidas ou irrelevantes.

Avaliar as ideias:

- Analisar a viabilidade, a relevância e o impacto de cada ideia.
- Priorizar as ideias com base nos critérios definidos.

Refinar as ideias:

- Desenvolver as ideias selecionadas com mais detalhes.
- Criar protótipos ou modelos para visualizar as ideias.

Validar as ideias:

- Apresentar as ideias refinadas para os usuários e solicitar feedback.
- Realizar ajustes nas ideias baseando-se no feedback dos usuários.



Dica

O microcurso do SENAI PLAY com o título “Criatividade e Inovação” explora ferramentas como o brainstorm a fim de transformar ideias em realidade, acesse o conteúdo pelo link abaixo e descubra como liberar seu potencial criativo e impulsionar a inovação.

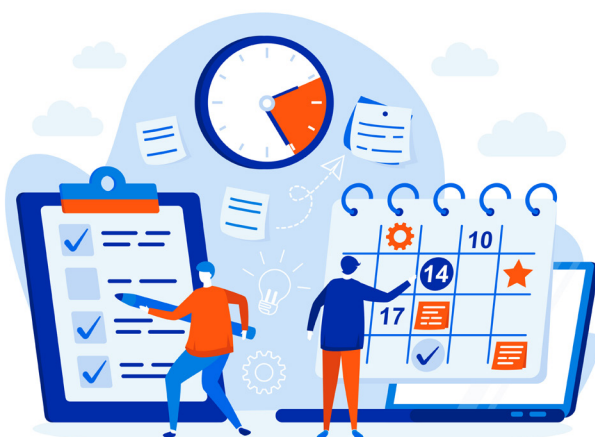
Prototipação

Utilizar a prototipação é como criar um esboço do sistema, um modelo que permite aos usuários visualizar e interagir com as funcionalidades principais antes mesmo de seu desenvolvimento completo. Pense em um arquiteto apresentando uma maquete para o cliente: ela o ajuda a visualizar o projeto final e a fazer ajustes antes do início da construção. Da mesma forma, a prototipação permite que os stakeholders experimentem o software em estágios iniciais, identifiquem problemas de usabilidade, sugiram melhorias e validem os requisitos com mais precisão.

Exemplo prático: prototipação do aplicativo de lista de tarefas.

Passo 1

Planejamento do protótipo



Definir os objetivos:

- Validar os requisitos principais do aplicativo com os usuários.
- Identificar problemas de usabilidade e interface.
- Obter feedback dos usuários a fim de aprimorar o design do aplicativo.

Definir o tipo de protótipo:

- Protótipo de baixa fidelidade: esboços em papel ou wireframes simples para representar a estrutura e o fluxo do aplicativo.
- Protótipo de alta fidelidade: modelos interativos com design visual mais elaborado para simular a experiência do usuário.

Definir as funcionalidades a serem prototipadas:

- Criação e gerenciamento de tarefas.
- Atribuição de tarefas a outros usuários.
- Definição de prazos e lembretes.
- Marcação de tarefas como concluídas.

Escolher as ferramentas de prototipação:

- Ferramentas de desenho e wireframing (por exemplo, Figma, Sketch, Adobe XD).
- Ferramentas de prototipação interativa (por exemplo, InVision, Proto.io).

Passo 2

Desenvolvimento do protótipo



Criar o protótipo:

- Desenhar as telas e interfaces do aplicativo.
- Definir a navegação e o fluxo de interação.
- Adicionar elementos interativos, como botões e links.

Utilizar as ferramentas de prototipação:

- Criar wireframes e protótipos interativos.
- Adicionar animações e transições para simular a experiência do usuário.

Manter o foco nos requisitos:

- Garantir que o protótipo represente as funcionalidades principais do aplicativo.

Utilizar linguagem clara e objetiva para descrever as funcionalidades.

Passo 3:

Validação do protótipo com os usuários



Agendar sessões de validação:

- Realizar sessões individuais ou em grupo com os usuários.
- Apresentar o protótipo e solicitar feedback.

Observar a interação dos usuários

- Observar como os usuários interagem com o protótipo.
- Identificar os problemas de usabilidade e as dificuldades de navegação.

Coletar feedback dos usuários:

- Realizar perguntas abertas a fim de estimular o feedback.
- Registrar as sugestões e os comentários dos usuários.

Realizar ajustes no protótipo:

- Implementar as melhorias sugeridas pelos usuários.
 - Realizar testes adicionais para validar as alterações.
-

Durante o processo de prototipação é importante estar atento às questões a seguir.

- Combinar diferentes técnicas: cada técnica tem suas vantagens e desvantagens, combine diferentes técnicas para obter uma visão completa dos requisitos.
- Envolver os usuários: eles são os especialistas no sistema, mantenha-os envolvidos em todo o processo de levantamento de requisitos.
- Documentar tudo: registre todas as informações coletadas, incluindo entrevistas, observações e ideias geradas em brainstormings.
- Ser flexível: o levantamento de requisitos é um processo iterativo e dinâmico, esteja aberto a mudanças.

Fases

O levantamento de requisitos é uma jornada que transforma ideias nebulosas em sistemas de software concretos. A fim de que essa jornada seja bem-sucedida, é fundamental seguir um roteiro claro e organizado. As fases de coleta, análise e registro são como os marcos que guiam essa jornada, garantindo que cada passo seja dado com precisão e clareza. Cada fase tem seu papel crucial, e a falha em qualquer uma delas pode comprometer o resultado final.

- **Coleta**

A coleta é o ponto de partida, momento de escutar os usuários. É a fase de investigação, na qual buscamos entender suas necessidades, desejos e expectativas. Nesta fase, utilizamos técnicas como entrevistas, questionários, observação e análise de documentos para reunir o máximo de informações relevantes.

Exemplo: Em um projeto de um aplicativo de e-commerce, a coleta de requisitos pode envolver entrevistas com potenciais clientes para entender suas preferências de compra, análise de sites de e-commerce concorrentes e observação do comportamento de compra em lojas físicas.

- **Análise**

Com as informações coletadas em mãos, é o momento de organizá-las e transformá-las em requisitos claros e precisos. A análise é a fase de interpretação, em que buscamos identificar padrões, eliminar ambiguidades e priorizar as necessidades dos usuários.

Exemplo: No projeto do aplicativo de e-commerce, a análise dos dados coletados pode revelar que os clientes valorizam a facilidade

de navegação, a segurança nas transações e a variedade de produtos. Com base nessa análise, podemos definir requisitos como "o aplicativo deve utilizar criptografia de dados" ou "o aplicativo deve oferecer um catálogo diversificado de produtos".

- **Registro**

O registro é a fase de formalização, na qual transformamos os requisitos analisados em documentos que servirão como guia para o desenvolvimento do sistema. É o momento de criar um registro claro e organizado de todas as decisões tomadas durante o levantamento de requisitos.

Exemplo: No projeto do aplicativo de e-commerce, o registro dos requisitos pode resultar na criação de um documento de especificação de Requisitos de Software (SRS), que detalha todas as funcionalidades do aplicativo, os requisitos não funcionais (como desempenho e segurança) e os critérios de aceitação.

A fase de registro é o ponto culminante do levantamento de requisitos, em que todo o conhecimento adquirido se transforma em documentação formal e estruturada. É nesse momento crucial que documentos, como o Documento de Requisitos de Software (DRS) e o Plano de Gerenciamento de Requisitos (PGR) ganham vida, servindo como guias para o desenvolvimento.

!/? Você sabia?

Em resumo, o levantamento de requisitos se desdobra em três fases cruciais: **Coleta**, em que exploramos o universo dos usuários para capturar suas necessidades; **Análise**, momento de transformar dados brutos em requisitos claros e priorizados; e **Registro**, a fase de documentação detalhada, criando um guia sólido para o desenvolvimento do sistema.

Priorização de Requisitos

A priorização de requisitos é uma etapa crucial no processo de levantamento de requisitos, garantindo que os esforços de desenvolvimento sejam direcionados para as funcionalidades mais valiosas e essenciais do sistema. Em um cenário em que os recursos são limitados e as expectativas dos stakeholders são diversificadas, a capacidade de priorizar requisitos de forma eficaz é fundamental para o sucesso do projeto.

!! Importante

Ao incorporar a priorização de requisitos em todas as fases do levantamento deles, as equipes de desenvolvimento podem garantir que seus projetos sejam entregues com sucesso, atendendo às necessidades dos usuários e agregando valor ao negócio.

Técnicas de Priorização de Requisitos

A priorização de requisitos é essencial para garantir que os recursos do projeto sejam utilizados de forma eficiente, focando nas funcionalidades as quais agregam maior valor ao negócio. Existem diversas técnicas que podem ser utilizadas para priorizar requisitos, cada uma contendo suas vantagens e desvantagens.

Veja nos quadros a seguir para conhecer algumas delas.

1. MoSCoW:

Classifica os requisitos em quatro categorias:



- **Must have (deve ter):** Requisitos essenciais para o funcionamento do sistema. Sem eles, o sistema não pode ser considerado completo.
- **Should have (deveria ter):** Requisitos importantes, mas não essenciais. Podem ser implementados em versões futuras do sistema.
- **Could have (poderia ter):** Requisitos desejáveis, mas que representam baixo impacto no sucesso do projeto.
- **Won't have (não terá):** Requisitos que não serão implementados no projeto atual.

Exemplo prático: No aplicativo de lista de tarefas:

- **Must have:** Criação e edição de tarefas, marcação de tarefas como concluídas.
- **Should have:** Atribuição de tarefas a outros usuários, lembretes de tarefas pendentes.
- **Could have:** Relatórios de desempenho, integração com redes sociais.
- **Won't have:** Funcionalidades de gamificação, integração com sistemas de terceiros os quais não são utilizados pela empresa.

Vantagens: Simples de entender e aplicar, facilita a comunicação com os stakeholders.

Desvantagens: Pode ser subjetivo, dependendo da interpretação dos stakeholders.

2. Análise de Valor vs. Esforço:

Avalia o valor que cada requisito entrega para o negócio em relação ao esforço necessário para implementá-lo. De modo que requisitos com alto valor e baixo esforço são priorizados.



Exemplo prático: No aplicativo de lista de tarefas:

- **Alto valor, baixo esforço:** Adicionar um campo com "data de vencimento" às tarefas.
- **Alto valor, alto esforço:** Implementar um sistema de notificações push para lembretes de tarefas.

- **Baixo valor, baixo esforço:** Adicionar um tema visual personalizável ao aplicativo.
- **Baixo valor, alto esforço:** Integrar o aplicativo com um sistema de gestão de projetos complexo.

Vantagens: Permite tomar decisões mais objetivas, focando nos requisitos os quais trazem maior retorno sobre o investimento.

Desvantagens: A estimativa de valor e esforço pode ser imprecisa, dependendo da experiência da equipe.

3. Priorização por Pontos:

Atribui pontos a cada requisito com base em critérios – como importância, urgência e risco. Aqui, os requisitos com maior pontuação são priorizados.



Exemplo prático: No aplicativo de lista de tarefas:

- **Critérios:** importância (1-5), urgência (1-3), risco (1-2).
- **Requisito:** "Lembretes de tarefas pendentes".
- **Pontuação:** importância (5) + urgência (3) + risco (1) = 9.
- **Requisito:** "Relatórios de desempenho".
- **Pontuação:** importância (3) + urgência (2) + risco (1) = 6.

Vantagens: Permite considerar múltiplos critérios de priorização, facilita a comparação entre requisitos.

Desvantagens: A atribuição de pontos pode ser subjetiva, a definição dos critérios pode ser complexa.

Ao utilizar essas técnicas, as equipes de desenvolvimento podem tomar decisões mais embasadas sobre quais requisitos priorizar, garantindo que o software final atenda às necessidades dos usuários e agregue valor ao negócio.



Saiba mais

Benefícios da Priorização:

- Redução de custos e prazos de desenvolvimento.
- Aumento da satisfação dos stakeholders.
- Melhoria na qualidade do software.
- Maior foco nas funcionalidades mais importantes.

Do mesmo modo, é preciso prestar a devida atenção para que a fase de levantamento de requisitos seja bem executada. Muitas vezes pular essa etapa para o desenvolvimento do projeto em si pode parecer uma boa alternativa, mas na realidade, gera a falsa sensação de progresso, caso nossos requisitos não estejam bem definidos.

Requisitos Mal Definidos (Ruins)



1. "O sistema deve ser rápido."

Ambíguo: o que é "rápido"? Para quem?

2. "O site deve ser bonito."

Subjetivo: bonito para quem? Baseado em qual padrão?

3. "O aplicativo deve ter todas as funcionalidades necessárias."

Vago: quais funcionalidades? Para quais usuários?

4. "O sistema deve funcionar em todas as plataformas."

Incompleto: quais plataformas exatamente? Mobile? Web? Windows?

5. "O sistema deve ser fácil de usar."

Sem critério: qual é a definição de "fácil"? Existe alguma métrica de usabilidade?

Requisitos Bem Definidos (Bons)



1. "O sistema deve responder a comandos do usuário em até 2 segundos em 95% das interações."

Objetivo, mensurável e realista.

2. "O layout do site deve seguir as diretrizes do Material Design e ser aprovado pelo cliente X."

Específico e baseado em referência clara.

3. "O aplicativo deve permitir cadastro, login, recuperação de senha e agendamento de consultas."

Lista clara de funcionalidades esperadas.

4. "O sistema deve ser compatível com as versões mais recentes dos navegadores Chrome, Firefox e Safari."

Define exatamente quais plataformas são exigidas.

5. "O sistema deve passar nos testes de usabilidade com no mínimo 80% de aprovação pelos usuários do grupo de teste."

Tem um critério claro e mensurável.

Recapitulando

Nessa aula, mergulhamos no universo do levantamento de requisitos, desvendando sua importância e dominando as técnicas essenciais para transformar necessidades em soluções de software. Aprendemos que o levantamento de requisitos é o alicerce de qualquer projeto de sucesso, garantindo que o sistema atenda às expectativas dos usuários e resolva problemas reais.

Exploramos as principais técnicas de coleta de requisitos, tais quais briefings, pontos de vista, observação, entrevistas e brainstormings, aprendendo a utilizá-las de forma eficaz para obter informações valiosas. Compreendemos a importância das três fases do levantamento de requisitos: coleta, análise e registro, e como cada uma delas contribui para a criação de requisitos claros, precisos e priorizados.

Encerramento

Agora, com o conhecimento adquirido nessa aula, estamos preparados para dar o próximo passo em nossa jornada: Design Thinking. Na aula seguinte, exploraremos de que modo essa metodologia pode nos ajudar com etapas essenciais na descoberta de soluções inovadoras. Problemas podem ser desagradáveis, mas veremos com Design Thinking que a busca da solução destes problemas não precisa ser!



3

Design thinking e solução de problemas

Apresentação

Nesta aula, exploraremos o **Design Thinking** como abordagem de resolução de problemas centrada no ser humano. Conheceremos suas etapas — **Empatia, Definição, Ideação, Prototipação, Teste e Implementação** as quais guiarão o desenvolvimento de soluções alinhadas às reais necessidades dos usuários.

Destacaremos como a empatia aprofundará a compreensão de problemas, a ideação estimulará a geração de ideias criativas e a prototipação/teste permitirá validar rapidamente as propostas, refinando-as antes da implementação final. Por meio de exemplos práticos, evidenciaremos o potencial do Design Thinking em promover inovação, colaboração e promoção de resultados mais assertivos em projetos de software. Veremos que a busca por uma solução pode ser muito interessante e até mesmo descontraída! Unindo resultado e dinâmica.

Introdução ao Design Thinking

Objetivo da aula:

Apresentar os fundamentos do Design Thinking e como essa abordagem ajuda no levantamento de requisitos e na criação de protótipos para soluções mais eficazes.

O que é Design Thinking?

É uma **metodologia criativa e colaborativa** para resolver desafios, centrada no ser humano.

Não é um processo rígido — as etapas podem ser revisitadas em ciclos curtos.

Etapas do Design Thinking

1. Empatia

- Mergulhar na rotina do cliente.
- Conversar, observar e entender suas dores, desejos e expectativas.
- Escuta ativa e conexão com o problema.

2. Definição

- Sintetizar os achados e formular o desafio.
- Identificar ações desejadas como otimização de tempo, automação e gestão de dados.
- Requisitos devem ser claros e objetivos.

3. Ideação

- Gerar o máximo de ideias possíveis, sem julgamentos.
- Criatividade livre, seguida de filtragem posterior.

4. Prototipação

- Criar versões simples e baratas das ideias destacadas.
- Usar ferramentas como Figma para desenhar telas e validar soluções.
- Evita investir em sistemas que ainda não foram testados.

5. Testes

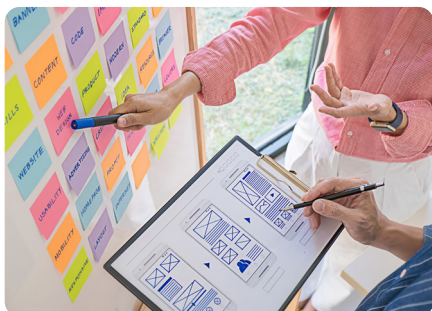
- Compartilhar o protótipo com o cliente.
- Observar uso real, dificuldades e compreensão da solução.

- Ajustar com base no feedback.

Benefícios para o levantamento de requisitos

- Facilita a compreensão profunda das necessidades reais do cliente.
- Gera confiança e acelera a adoção da solução.
- Permite ajustes antes de grandes investimentos.
- Coloca o ser humano no centro, tornando a inovação uma consequência natural.

Aqui está o resumo didático do documento Video07AplicandoaetapadeEmpatia, com linguagem simples, tamanho reduzido, atenção à pontuação e baseado exclusivamente no conteúdo enviado.



Imagine o desenvolvimento do sistema de gestão de eventos acadêmicos que não apenas cria funcionalidades, mas busca entender profundamente as necessidades da instituição de ensino. O desenvolvedor realiza entrevistas com os usuários, analisa como eles controlam os eventos, compreende os desafios enfrentados e coleta de feedback contínuo, até conseguir desenvolver uma solução que realmente atenda às suas necessidades. Essa busca pela compreensão profunda dos problemas e pela criação de soluções centradas no usuário é o que o Design Thinking abrange.

Definição

Design Thinking é uma abordagem de resolução de problemas centrada no ser humano que busca entender as necessidades e desafios dos usuários para criar soluções inovadoras e eficazes. O processo envolve empatia, colaboração e experimentação, sendo amplamente utilizado em áreas como design, desenvolvimento de produtos e serviços, e inovação.

O Design Thinking é composto por cinco fases principais:



Empatia

Entender profundamente as necessidades, desejos e desafios dos usuários por meio de pesquisa e observação.



Definição

Analisar as informações coletadas para definir o problema de maneira clara e centrada no usuário.



Ideação

Gerar uma ampla gama de ideias e soluções criativas a fim de resolver o problema identificado.



Prototipação

Criar protótipos de baixo custo para testar e visualizar as soluções propostas.



Testes

Testar os protótipos com usuários reais, coletando feedback para refinar e melhorar as soluções.



Implementação

Implementar o software, realizando um acompanhamento junto ao usuário a fim de verificar se o processo funcionou corretamente.

A essência do Design Thinking reside na interação contínua, promovendo uma mentalidade de experimentação e aprendizado constante até que se chegue à solução ideal. Aprofundaremos nosso conhecimento agora, abordando as etapas do Design Thinking.

O que é Design Thinking?

Design Thinking nos ajuda na formulação de ideias inovadoras de forma dinâmica e alegre. Quando temos um problema a resolver, o processo de gerar a solução pode se tornar lúdico!



Design Thinking nos ajuda na formulação de ideias inovadoras de forma dinâmica e alegre. Quando temos um problema a resolver, o processo de gerar a solução pode se tornar lúdico!

História do Design Thinking

Quer saber mais sobre a história dessa metodologia que pensa “fora da caixa”?

Acesse o podcast a seguir:

▶ **podcast resumido**

História do Design Thinking

Objetivo do episódio:

Apresentar a origem e evolução do Design Thinking, destacando seus principais nomes e contribuições.

O que é Design Thinking?

- Metodologia voltada para **soluções criativas de problemas complexos**.
- Baseada em **colaboração, empatia e experimentação**.
- Resultado de um **intercâmbio global de ideias** entre designers e estudiosos.

IDEO e a popularização do Design Thinking

- A empresa IDEO foi fundada em 1991 e teve papel central na democratização da metodologia.
- Criou um kit de ferramentas que facilitou o acesso à prática do Design Thinking por designers e não-designers.
- O livro “Design Thinking”, de Tim Brown (CEO da IDEO), lançado no Brasil em 2009, ajudou a difundir o conceito.

Principais nomes ligados ao Design Thinking

- Herbert Simon – primeiro a usar o termo em 1969, no livro As Ciências do Artificial.

- John Dewey
- Buckminster Fuller
- Victor Papanek
- Horst Rittel
- Nigel Cross
- Richard Buchanan
- David e Tom Kelley – irmãos e membros da IDEO.

Conclusão

O Design Thinking é uma metodologia construída ao longo de décadas por diversos pensadores e designers, com foco em **inovação centrada no ser humano**.

A IDEO teve papel essencial em tornar essa abordagem acessível e prática.

As 5 Etapas do Design Thinking

DESIGN THINKING - PROCESSO



Saiba mais sobre a etapa empatia no resumo a seguir:

 **vídeo resumido**

Aplicando a Etapa de Definição

Objetivo da aula:

Demonstrar como aplicar a etapa de empatia da metodologia Design Thinking, usando o exemplo do produtor rural Sr. Antônio, que deseja automatizar sua irrigação.

Por que a empatia é essencial?

- Evita soluções precipitadas e desconectadas da realidade.
- Permite entender dificuldades reais, emoções e rotinas do usuário.
- Ajuda a levantar requisitos funcionais e não funcionais com base em observações concretas.

Exemplo prático: Sr. Antônio

- Produtor de frutas e verduras em 180 ha.
- Irrigação manual com sonda analógica.
- Problemas: esquecimento de medições, desperdício de água, risco de perder certificações.

Queixas relatadas:

- Acordar às 3h para checar pivô.
- Conta de energia alta.
- Risco de perder contrato por falta de laudo.

Preocupações percebidas (não verbalizadas):

- Dúvidas sobre durabilidade de sensores.
- Medo de depender de consultores.
- Falta de conhecimento técnico (ex: API).

Observações da rotina

- Funcionário mede umidade 3x ao dia e anota em caderno.
- Comunicação manual entre equipe.
- Registros inconsistentes em dias de chuva.

Ganhos esperados vs. Dores atuais

Ganhos esperados:

- Redução de 15% no consumo de água.
- Relatórios automáticos.
- Mais tranquilidade no dia a dia.

Dores atuais:

- Sinal 4G fraco.
- Medo de tecnologia complexa.
- Alto custo de sensores.
- Equipe com pouca familiaridade com dashboards.

Requisitos identificados

- Não funcionais:
 - Confiabilidade offline.
 - Interface simples.
 - Manutenção plug-and-play.
 - Exportação de dados em PDF.
 - Retorno sobre investimento em até 18 meses.

Transformando insights em perguntas motivadoras

- Como garantir irrigação autônoma sem internet por 24h?
- Como mostrar economia de água em reais?
- Como permitir que o funcionário reporte falhas com um botão?

Conclusão

A etapa de empatia é fundamental para entender o usuário profundamente, sem julgamentos.

Ela permite levantar requisitos reais e construir soluções que fazem sentido para quem vai usá-las.

Etapas

As etapas na metodologia Design Thinking são absolutamente essenciais para seu sucesso. Elas fornecem uma estrutura, um guia, a qual garante que a equipe aborde um problema de forma sistemática, centrada no usuário e com foco na inovação. Vamos entender cada etapa e verificar por que são importantes dentro dessa metodologia.



Empatia

Essa etapa é a base de todo o processo. Sem uma compreensão profunda das necessidades – desejos, dores e motivações dos usuários – qualquer solução criada corre o risco de ser irrelevante ou ineficaz. A empatia garante que a solução seja verdadeiramente centrada no usuário.

Aplicando a etapa de empatia



O Design Thinking tem o foco na pessoa. Logo, a empatia é um elemento importante para atingir os melhores resultados com a solução final.

Além disso, durante o projeto, é importante definir para quem se está criando a solução. Essa descoberta não é aleatória nem deve se limitar a suposições.

Trabalhando a empatia e realizando pesquisa sobre público-alvo

No resumo do *podcast* a seguir, é tratada a empatia e o público-alvo, pontos chave do trabalho do Design Thinking.

Aplicando a Etapa de Empatia

Objetivo da aula:

Demonstrar como aplicar a etapa de empatia da metodologia Design Thinking, usando o exemplo do produtor rural Sr. Antônio, que deseja automatizar sua irrigação.

Por que a empatia é essencial?

- Evita soluções precipitadas e desconectadas da realidade.
- Permite **entender dificuldades reais, emoções e rotinas** do usuário.
- Ajuda a levantar requisitos funcionais e não funcionais com base em observações concretas.

Exemplo prático: Sr. Antônio

- Produtor de frutas e verduras em 180 ha.
- Irrigação manual com sonda analógica.
- Problemas: esquecimento de medições, desperdício de água, risco de perder certificações.

Queixas relatadas:

- Acordar às 3h para checar pivô.
- Conta de energia alta.
- Risco de perder contrato por falta de laudo.

Preocupações percebidas (não verbalizadas):

- Dúvidas sobre durabilidade de sensores.
- Medo de depender de consultores.
- Falta de conhecimento técnico (ex: API).

Observações da rotina

- Funcionário mede umidade 3x ao dia e anota em caderno.
- Comunicação manual entre equipe.
- Registros inconsistentes em dias de chuva.

Ganhos esperados vs. Dores atuais

Ganhos esperados:

- Redução de 15% no consumo de água.
- Relatórios automáticos.
- Mais tranquilidade no dia a dia.

Dores atuais:

- Sinal 4G fraco.
- Medo de tecnologia complexa.
- Alto custo de sensores.
- Equipe com pouca familiaridade com dashboards.

Requisitos identificados

Não funcionais:

- Confiabilidade offline.
- Interface simples.
- Manutenção plug-and-play.
- Exportação de dados em PDF.
- Retorno sobre investimento em até 18 meses.

Transformando insights em perguntas motivadoras

- Como garantir irrigação autônoma sem internet por 24h?
- Como mostrar economia de água em reais?
- Como permitir que o funcionário reporte falhas com um botão?

Conclusão

A etapa de empatia é **fundamental para entender o usuário profundamente**, sem julgamentos.

Ela permite levantar requisitos reais e construir soluções que fazem sentido para quem vai usá-las.

Mapa de Empatia

Esta é uma excelente ferramenta para aplicar a etapa de empatia no processo de desenvolvimento. Por meio dele, conseguimos conhecer mais profundamente o nosso público-alvo, entendendo não apenas o que pensa e sente, mas também o que vê, ouve e faz em seu dia a dia.

Essa abordagem favorece o trabalho em equipe, permitindo que todos passem a enxergar o cliente a partir da própria perspectiva. E quanto melhor compreendermos quem é o nosso usuário, mais precisos serão os requisitos levantados resultando em um sistema mais alinhado às suas reais necessidades e, portanto, muito mais assertivo.

Mapa de empatia e a criação de personas

Nome: _____ Idade: _____

O que
PENSA E SENTE?

O que
OUVE?

O que
VÊ?

O que
FALA E FAZ?

quais são as **DORES?**

quais são as **NECESSIDADES?**

O mapa de empatia possibilita reunir características que envolvem diferentes públicos-alvo. Ele permite à equipe entender e visualizar características de determinados públicos. Além disso, **auxilia nas tomadas de decisão e na criação de personas**. Interessante, não?

O mapa de empatia nos ajuda a enxergar o mundo pelos olhos do cliente, compreendendo o que ele pensa, sente, vê, ouve e faz. Quer descobrir como aplicar essa ferramenta na prática e criar personas que realmente representem seu público?

Ferramenta Mapa da Empatia e Criando Personas

A Ferramenta Mapa da Empatia:

- Reúne e permite entender **características dos diferentes públicos-alvo**.
- Auxilia nas **tomadas de decisão** e na **criação de personas**.
- Representa um **segmento** de pessoas.
- Formado pelas **características comuns** das pessoas entrevistadas.
- É **mutável**: pode sofrer alterações ao longo do processo.

Para criar o mapa, é preciso:

- Definir o **objetivo** para criar o mapa.
- **Analisar** entrevistas e encontrar **padrões**.
- Reunir o que os entrevistados disseram.
- Identificar e **agrupar** ações e comportamentos.
- Descobrir **motivações, objetivos e desejos** dos entrevistados.
- Identificar o que os entrevistados **sentem**.
- Agrupar **barreiras** mais impactantes.
- Identificar **necessidades** dos entrevistados.

Criando Personas

A persona:

- Permite **visualizar o público** que irá se beneficiar.
- Cria **representações fictícias**, com características realistas.
- Transforma grupos de usuários em **uma "pessoa" com histórias**.
- Possibilita a criação de **perfis**.
- Contribui para o **direcionamento** do projeto.

Para representar a realidade, a equipe deve considerar:

- O **comportamento** da persona.
- As **expectativas** da persona.
- Os **obstáculos** enfrentados.

Deve-se utilizar o pensamento convergente, buscando criar um resumo para cada persona.

Outras informações que podem ser criadas para as Personas:

- Nome e foto
- Sexo
- Idade
- Classe social
- Demais características

Pontos de atenção

Com a ferramenta mapa de empatia e criando personas:

- É possível entender as características do público-alvo.
- Trata-se de características fictícias com representações realistas, como sendo uma pessoa, neste caso, o público-alvo do projeto.

Exemplo de pesquisa com usuários

Observação e entrevistas são outras abordagens as quais podemos aplicar antes do Design Thinking para entendermos e conhecermos melhor nossos usuários. Tendo feito entrevistas e observações, teremos muito mais conhecimento e propriedade para realizar o Design Thinking em grupo.



Entrevista



Observação

Definição



Essa etapa incentiva a criatividade e a geração de ampla gama de ideias. Ao adiar o julgamento e incentivar o brainstorming, a equipe explora possibilidades que talvez não fossem consideradas de outra maneira. A etapa de Ideação garante que a equipe não se prenda a soluções óbvias ou convencionais.

A seguir, saiba mais sobre a etapa definição.

 **vídeo resumido**

Aplicando a etapa de Definição do Problema

Objetivo da aula:

Compreender como definir o problema de forma clara, inspiradora e centrada no usuário, segundo o método do Design Thinking.

O que é a etapa de definição?

- É o momento de refinar os dados coletados na empatia em um ponto focal.
- A definição correta guia todo o processo de inovação.
- Uma definição mal feita pode gerar soluções geniais, mas inúteis.

Exemplo prático: Sr. Antônio

Informações coletadas na empatia:

- Quer parar de acordar de madrugada para verificar irrigação.
- Reclama de contas altas de energia.
- Tem medo de perder certificações.
- Usa celular, mas teme depender de suporte técnico.
- Funcionário Zé Luiz faz medições manuais.
- Sinal 4G instável em algumas estufas.

Temas agrupados:

- Confiança
- Custo
- Rotina operacional
- Conectividade
- Responsabilidade compartilhada

Como criar uma boa pergunta de desafio?

Formato ideal:

Usuário + necessidade + insight

Exemplo:

“Sr. Antônio, produtor rural, precisa garantir irrigação confiável mesmo com 4G instável, pois teme que falhas prejudiquem contratos de exportação.”

Pergunta criativa derivada:

“Como podemos garantir irrigação precisa e segura em uma fazenda com conectividade limitada, sem que o produtor sinta que perdeu o controle?”

Evite perguntas mal formuladas

- Supõem a solução: “Como instalar sensores modernos?”
- Genéricas demais: “Como reduzir o uso de água?”
- Sem foco: “Como resolver o problema do Sr. Antônio?”

Ferramentas para priorização

Método MoSCoW:

- **Must:** essencial (ex: irrigação confiável)
- **Should:** importante (ex: relatórios automáticos)
- **Could:** desejável (ex: economia de energia)
- **Want:** opcional (ex: integração com sistema virtual)

Matriz impacto vs. esforço:

- Alta prioridade: alto impacto + baixo esforço
- Baixa prioridade: baixo impacto + alto esforço

Checklist para uma boa definição

- Tem um usuário específico
- Expressa uma necessidade real e clara
- Traz um insight humano
- É aberta à inovação
- É inspiradora e provocadora

Conclusão

Definir bem o problema é **colocar o usuário no centro** e **dar foco à ideação**.

Toda boa inovação começa com uma **pergunta bem feita**.

Ideação



Essa etapa incentiva a criatividade e a geração de ampla gama de ideias. Ao adiar o julgamento e incentivar o brainstorming, a equipe explora possibilidades que talvez não fossem consideradas de outra maneira. A etapa de Ideação garante que a equipe não se prenda a soluções óbvias ou convencionais.

Saiba mais a seguir sobre a etapa de ideação.

 **vídeo resumido**

Aplicando a etapa de Ideação – Gerando Soluções Inovadoras

Objetivo da aula:

Explorar a etapa de ideação do Design Thinking, que busca gerar o maior número possível de ideias criativas e viáveis para resolver desafios reais do cliente.

O que é ideação?

- Fase de exploração criativa após entender o problema.

- Foco em quantidade e diversidade de ideias, sem julgamentos.
- A ideia perfeita não é o objetivo imediato — ela pode surgir após muitas outras.

Preparando o ambiente para o brainstorming

3 ingredientes essenciais:

1. Segurança psicológica – ninguém julga ideias.
2. Participação diversa – misturar visões técnicas e práticas.
3. Tempo curto e ritmo acelerado – estimular fluidez criativa.

5 regras básicas:

- Não julgar ideias.
- Buscar quantidade, não qualidade.
- Construir sobre ideias dos outros.
- Ser visual (desenhar ajuda).
- Ser ousado (ideias malucas são bem-vindas).

Exemplo prático: Sr. Antônio

Pergunta norteadora:

“Como podemos garantir irrigação precisa e confiável, mesmo com conectividade limitada, sem que o produtor sinta que perdeu o controle?”

Ideias geradas:

- Semáforo de irrigação com LED RGB.
- Botão físico com rádio e energia solar.
- Relatório automático via WhatsApp com emojis.
- Aplicativo com voz de locutor de rádio rural.
- Alerta vibratório no bolso do funcionário.

Construção coletiva:

- Botão com placa solar.
- Alerta via relógio barato.
- Voz personalizada com sotaque local.

Critérios para seleção de ideias

1. Relevância – resolve o problema real.
2. Facilidade de implementação – pode ser testado com pouco esforço.
3. Alinhamento com o contexto – funciona mesmo com pouca internet ou equipe não técnica.

Ideias selecionadas:

- Semáforo de irrigação.
- Botão físico solar com rádio.
- Bot de WhatsApp com voz personalizada.

Validação com usuários

- Sr. Antônio aprova o semáforo.
- Funcionário Zé Luiz sugere voz do locutor da rádio local.
- Reações reais indicam direção certa e ajustes úteis.

Checklist de um brainstorming bem-sucedido

- Gerou **volume** (mínimo 20 ideias).
- Teve participação de **usuários reais**.
- **Sem julgamentos** durante a criação.
- Permitiu **evoluir ideias em grupo**.
- Resultou em 2 ou 3 propostas promissoras.

Conclusão

A ideação, quando feita com abertura e intencionalidade, gera **soluções criativas e aplicáveis**.

No Design Thinking, até o erro é um dado valioso.

Brainstorming



É uma ferramenta e técnica muito conhecida. Pode ser usada em quase todas as etapas do Design Thinking, dependendo da necessidade da equipe.

Neste estágio do projeto, é natural que muitos profissionais queiram definir logo a solução; porém, neste momento, ainda se deve trabalhar o pensamento divergente, considerando muitas ideias, buscando ser criativo e pensar fora da caixa.

Saiba mais a seguir sobre a sessão de brainstorming.

▶ **podcast resumido**

Técnica de brainstorming

Objetivo do episódio:

Apresentar os princípios e passos para realizar uma sessão eficaz de Brainstorming, estimulando a criatividade coletiva.

O que é Brainstorming?

- Técnica para **geração de ideias em grupo**.
- Foco em quantidade, não em qualidade imediata.
- Ideias exageradas ou impossíveis **podem inspirar soluções viáveis**.

Regras fundamentais

- **Não julgar ideias** – nenhuma ideia deve ser descartada ou criticada.
- **Evitar discussões** – não comentar ou tentar validar ideias durante a sessão.
- **Estimular a inspiração** – até ideias absurdas são bem-vindas.
- **Foco na quantidade** – quanto mais ideias, maior a chance de encontrar boas soluções.

Recursos necessários

- Ambiente confortável e acolhedor.
- Notas adesivas e canetas para cada participante.
- Um objetivo claro, visível para todos.

Tempo ideal de sessão

- Entre 15 e 40 minutos.
- Sessões curtas podem limitar a criatividade.
- Sessões longas podem causar cansaço.

Passos para realizar o Brainstorming

1. Definir o tempo da sessão.
2. Estabelecer o objetivo (problema a ser resolvido).
3. Garantir o não julgamento.
4. Estimular a inspiração.
5. Incentivar a quantidade de ideias.
6. Usar notas adesivas e caneta.
7. Evitar discussões durante a sessão.

Conclusão

O Brainstorming é uma técnica simples, acessível e poderosa para **desenvolver soluções criativas**.

Com ambiente adequado e regras claras, a equipe pode gerar ideias que transformam o rumo de um projeto.

Exemplos de brainstormings

Cenário

Equipe de marketing tentando criar uma campanha para um novo sorvete vegano. Problemas observados:

- Um membro lidera a discussão e invalida outras ideias.
- As ideias são julgadas antes de serem exploradas.
- Pouca diversidade de sugestões. Ninguém anota nada.



A seguir, vamos conhecer dois exemplos que demonstram como o ambiente e a forma de conduzir uma reunião podem impactar o resultado de um trabalho em equipe.



No primeiro cenário, vemos uma equipe lidando com a pressão de entregar uma campanha. As ideias surgem, mas são rapidamente interrompidas, julgadas ou descartadas. O clima é engessado, pouco receptivo à inovação. Resultado? Pouca participação, repetição de fórmulas antigas e um time desmotivado.

Mas e se o ambiente fosse diferente? Agora, a mesma equipe está reunida novamente, mesma missão, mesmos talentos, mas com uma abordagem completamente diferente.



Podemos notar que no segundo cenário, a mesma equipe enfrenta o mesmo desafio, mas com uma grande diferença: **boas práticas de colaboração são aplicadas**. O grupo estabelece regras claras, tais quais “sem julgamentos” e “construindo sobre as ideias dos outros”, e criando, assim, um espaço acolhedor e criativo. Com isso, todos se sentem seguros para contribuir. As ideias somam-se, transformam-se e ganham força.

Essas duas cenas ilustram o impacto que a cultura de colaboração pode ter na geração de ideias e no engajamento do time.

Prototipação

Essa etapa transforma as ideias em algo tangível e testável. A prototipagem sendo rápida e de baixo custo permite que a equipe valide as ideias com os usuários e obtenha feedback valioso antes de investir em um desenvolvimento completo. Os protótipos também servem para comunicar as ideias de forma clara e concisa aos interessados.

Aplicando a etapa de Prototipação - Transformando Ideias em Soluções

Objetivo da aula:

Demonstrar como transformar ideias em **protótipos visuais e navegáveis**, usando o Figma, para validar soluções antes de investir em desenvolvimento.

O que é prototipar?

A partir disso, devemos considerar os seguintes critérios:

- Simular o funcionamento de uma solução sem construir o sistema completo.
- Testar rapidamente, receber feedback e ajustar com baixo custo.
- Permite errar barato e aprender rápido.

Exemplo prático: Sr. Antônio

Necessidade:

Automatizar processos da fazenda com uma interface web simples.

Funcionalidades do protótipo:

- Visualizar status da irrigação em tempo real.
- Acompanhar histórico de alertas e medições.
- Acionar manualmente sistemas.
- Receber relatórios para auditoria.

Construção do protótipo no Figma

Passos principais:

1. Acessar figma.com e criar um rascunho.
2. Criar quadro com dimensões de tela padrão (1366x768 px).
3. Adicionar cabeçalho com nome do sistema ("Irriga Fácil") e botões de navegação.
4. Criar elementos visuais:
 - Indicador de umidade atual (ex: 28%).
 - Status do solo (ex: "Solo seco").
 - Botão "Ligar irrigação".

Destaques visuais:

- Letras grandes e cores chamativas para facilitar a leitura no campo.
- Interface pensada para o contexto rural e uso prático.

Validação com usuários

Feedbacks simulados:

- Sugestão de botão com comando de voz.
- Ajuste no tamanho do botão vermelho.

Benefícios da validação:

- Ajustes rápidos e sem custo.
- Evita retrabalho e economiza tempo de desenvolvimento.
- Gera base técnica para desenvolvedores e apresentação para parceiros.

Checklist de um bom protótipo

- Mostra funcionalidades principais.
- Usa linguagem acessível e visual amigável.
- Tem fluxos completos (entrada, navegação, saída).
- Pode ser testado com usuários reais.
- Está organizado com frames e transições claras.

Conclusão

Prototipar é **transformar empatia e criatividade em soluções visuais**.

Não é preciso ser designer — basta entender a dor do usuário e criar telas simples e úteis.

Prototipar é **errar no papel antes de errar no projeto**.

Organizando as informações

Organizar as informações é o momento final da etapa de desenvolvimento em que será necessário o pensamento convergente. A equipe precisará selecionar a ideia a qual será prototipada.

Saiba mais no resumo a seguir, sobre como realizar a organização das informações para a prototipagem.

Organizando as informações

Objetivo do episódio:

Apresentar um processo prático para selecionar, validar e detalhar ideias antes da etapa de prototipação.

Etapas para organizar as ideias

1. Validação interna (com a equipe):

- Reunir todos os integrantes.
- Apresentar o objetivo do projeto, o problema e os requisitos.
- Expor as fichas de ideias em local visível.
- Cada integrante vota com 3 a 5 adesivos nas ideias que melhor atendem aos critérios.
- As 3 ideias mais votadas são analisadas e aprimoradas.

2. Preparação para validação externa:

- Criar nome, slogan e objetivo para cada ideia.
- Opcional: desenvolver uma identidade visual simples (logotipo, cores, tipografia).

3. Validação com o público externo:

- Apresentar o contexto e o problema.
- Mostrar as ideias e pedir que o público vote na mais adequada.
- Coletar feedbacks para melhorias.

Detalhamento das ideias

- Escolher duas funcionalidades de cada ideia.
- Descrever como diferentes personas interagiriam com elas.

Exemplo de funcionalidade:

- Descrever rotina diária >> identificar hábitos prejudiciais >> escrever em post-its >> colar em local visível (ex: geladeira).

Refinamento com a equipe

- Avaliar o passo a passo com perguntas como:
 - Faz sentido?
 - Está organizado logicamente?
 - Falta alguma etapa?
 - Pode ser expandido ou simplificado?

Escolha final para prototipação

- Após melhorias, as ideias são colocadas novamente em votação pela equipe.
- A mais votada será prototipada na próxima etapa.

Tipos de protótipos

Para compreender os tipos de protótipos, é importante entender antes a importância de se criar o protótipo dentro do processo de Design Thinking. O protótipo servirá para testar a solução com os usuários e validar se atender aos requisitos do projeto. É importante que os integrantes da equipe tenham uma visão, além destes pontos, do que buscar com o protótipo. Assim, o profissional deve considerar:

- **Flexibilidade**

Mesmo que a equipe defina uma solução ideal, é importante que o projeto permita mudar e se moldar conforme houver necessidade, além do que foi planejado inicialmente. Por exemplo, o Ifood, que começou como delivery para restaurantes, expandiu-se para também atender a supermercados.

- **Escalabilidade**

Um projeto escalável é aquele que necessita dos mínimos recursos para chegar a diversos lugares do mundo, facilitando sua expansão. Portanto, um projeto, que tenha o máximo de autonomia e independência, poderá facilmente ser expandido. Por exemplo, o TikTok que funciona com as postagens das pessoas em qualquer lugar onde se tem internet.

- **Adaptabilidade**

Deve-se pensar o projeto nos possíveis cenários em que poderá ser utilizado por diferentes públicos. O que pode exigir novas formas de

uso da solução e adaptações por parte do público-alvo. Por exemplo, uma cadeira que pode ser usada para subir e trocar uma lâmpada.

Existem diversas maneiras de se criar um protótipo, o tipo escolhido dependerá da necessidade do projeto. O ideal do protótipo é fornecer à equipe e a todos os envolvidos, tais quais cliente e público, a possibilidade de visualizar com clareza a solução e como ela funcionará.

- **Esboço**

Lugar em que a equipe pode desenhar rapidamente a ideia e o seu fluxo de uso. É um protótipo rápido e de baixo custo, permitindo visualizar seu funcionamento e corrigir erros de modo ágil.

- **Template ou modelo**

Este tipo de protótipo exige que seja feito em alta fidelidade; ou seja, deve se parecer com a solução final, simulando tanto os aspectos funcionais quanto os visuais. Ele servirá para que os envolvidos compreendam a solução como se dará no cenário real.

- **Maquete**

É um protótipo em dimensões menores, possibilitando criar cenários de uso sem sua aplicação em locais físicos e com todos os envolvidos. Este tipo de protótipo proporcionará à equipe a visualização de seu funcionamento sem que haja deslocamentos, além do custo menor que o do template.

Muitas vezes, o esboço é criado possibilitando que a equipe faça alterações rápidas. Logo em seguida, é escolhido o template e/ou o modelo, já que necessitam de mais tempo e orçamento para suas criações.



Saiba mais

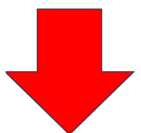
Prototipar é tornar física e real a ideia; é sintetizar todos os estudos e levantamentos realizados em uma solução real, mesmo que básica. É necessário entender que o **protótipo deve permitir testes e validações com todos os envolvidos no projeto, principalmente os usuários.**

O protótipo pode ter níveis de fidelidade em sua representação:



Alta

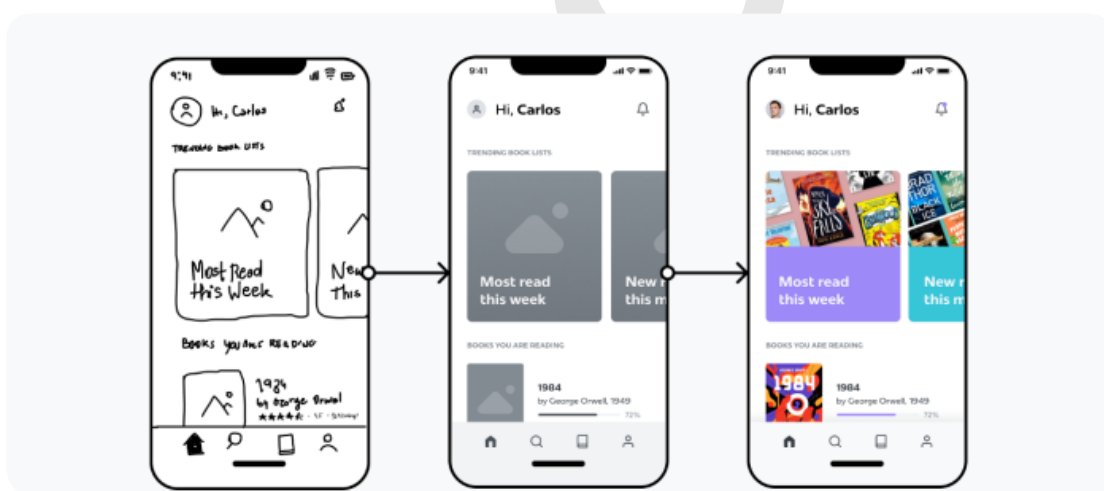
É um protótipo mais próximo do real, como no template ou na maquete. Além das funcionalidades, é possível também validar seus aspectos visuais e seu modo de comunicação com o público.



Baixa

Sendo um protótipo mais simples, como no caso do esboço e até mesmo da maquete. O objetivo é avaliar o conceito da ideia e analisar as funcionalidades entregues.

Aqui temos um exemplo de um protótipo para um aplicativo de leitura. Inicialmente, seu protótipo era apenas rabiscos, mas depois que a ideia foi sendo validada e se verificou se a tela fazia sentido, foi se aperfeiçoando o protótipo até que chegasse ao de alta fidelidade (à direita).



Dica

Para criar o protótipo, a equipe deverá especificar o motivo e o objetivo para essa criação. É este objetivo que determinará o tipo do protótipo, seu nível de fidelidade e o ambiente em que será testado.

Assim, a equipe poderá aplicar os testes, avaliar e, sendo necessário, aperfeiçoá-lo, testá-lo e avaliá-lo novamente, repetindo este ciclo quantas vezes julgar necessárias.

Ferramentas de prototipação

A equipe deve buscar desenvolver o protótipo da forma mais realista possível, tanto nos aspectos funcionais quanto nos visuais. Para isso, o protótipo pode ser uma simulação física ou digital. É válido ressaltar que no mercado ainda existem outras ferramentas que possibilitam criar protótipos digitais, como o Adobe XD e o Marvel.



Sketch

Ferramenta de design vetorial desenvolvida para macOS, projetada especialmente para designers de interfaces.



Figma

Ferramenta de design online que permite criar, compartilhar e testar designs. É uma plataforma colaborativa a qual ajuda equipes de design e de desenvolvimento a construir produtos digitais.

Teste



Essa etapa é fundamental para validar as soluções e obter feedback direto dos usuários. Os testes revelam o que funciona, o que não funciona e o que precisa ser melhorado. O feedback dos usuários direciona as interações e garante que a solução final atenda às suas necessidades e expectativas.

No momento anterior à entrega do produto ao cliente, tudo que é realizado tem o objetivo de chegar à solução final, já validada. Nessa etapa, é necessário que a solução chegue ao final muito bem construída, sem lacunas ou pontos cegos os quais possam gerar um problema posterior.

Por ser uma fase na qual não se pode deixar passar detalhes, é muito comum a realização de testes rápidos com os usuários e melhorar a solução, conforme os feedbacks. Outras ações que são comuns nessa etapa são a realização novamente de sessões de brainstorming, além do uso de outras ferramentas que a equipe julgar necessárias para se alcançar a solução final.

!! Importante

Qualquer falha que a equipe não tenha notado pode acarretar perdas financeiras significativas para o cliente. Portanto, é uma etapa que exige bastante **atenção aos detalhes** e ao **objetivo do projeto**.

Esta é uma etapa em que é possível corrigir pequenas falhas na solução, mas jamais implantar uma nova ideia desde seu início.

▶ podcast resumido

Etapa de teste com os usuários

Objetivo da etapa:

Verificar como o usuário interage com a solução final e validar se ela atende aos requisitos definidos.

Por que testar com usuários?

- Garante que a solução está próxima do ideal antes da entrega.
- Permite identificar melhorias mínimas, já que é uma fase avançada.
- Evita mudanças complexas e caras.

Antes de iniciar os testes:

A equipe deve refletir sobre:

- Quais informações precisam ser coletadas?
- Como será feita a coleta?
- O que se espera dos dados?
- O teste será em ambiente real ou controlado?

Indicadores e Métricas:

- Devem ser definidos antes dos testes.
- Ajudam a acompanhar resultados em curto, médio e longo prazo.
- Permitem entender se a solução está sendo eficiente.

Perguntas para orientar a definição:

- A solução funciona como esperado?

- Está sendo usada como foi projetada?
- Está atingindo o objetivo do projeto?
- Como mensurar isso?

Durante e após os testes:

A equipe deve levantar:

- Quais funcionalidades foram usadas?
- Quantas vezes e por quanto tempo?
- Funcionaram como esperado?
- Quantos usuários foram alcançados?
- Quantos atenderam suas necessidades?
- Quantos tiveram problemas?

Melhorias em tempo real:

- Não é preciso esperar o fim da validação para ajustar.
- Os feedbacks e métricas em tempo real permitem correções rápidas.

Teste de sombra

Teste de sombra é um teste no qual o desenvolvedor do protótipo observa o usuário utilizando o seu protótipo, se o usuário apresentar dificuldade em se localizar e identificar onde as funcionalidades estão, significa que o protótipo precisa ser melhorado.

É importante compreender que, para viabilizar o desenvolvimento da solução, é necessário prestar atenção em:



Usuário

A solução atende às necessidades dos usuários? Quais são os benefícios nítidos?



Prazo

Qual o prazo para criar a solução? A equipe terá tempo suficiente para desenvolver tudo o que está sendo proposto?



Orçamento

Qual o orçamento disponível? É possível desenvolver a solução com o orçamento que foi disponibilizado para o projeto?

Saiba mais a seguir sobre a etapa de teste.

 **vídeo resumido**

Aplicando a etapa de Testes – Transformando Ideias em Soluções

Objetivo da aula:

Apresentar a etapa de testes no Design Thinking, mostrando como validar protótipos com usuários reais para transformar ideias em soluções eficazes.

O que são testes?

- Testar é **observar o uso real** do protótipo.
- Serve para identificar frustrações, confusões, oportunidades de melhoria e validar o fluxo.
- **Errar aqui é barato** — corrigir depois do lançamento pode custar caro.

Como compartilhar o protótipo no Figma

1. Abrir o protótipo.
2. Clicar em “**Compartilhar**” e permitir visualização pública.
3. Gerar e copiar o link.
4. Enviar para usuários reais testarem em computador, celular ou tablet.

Quem deve testar?

- **Usuário principal** (ex: Sr. Antônio, o fazendeiro).
- **Usuário operacional** (ex: funcionário que usará o sistema).
- **Usuário secundário** (ex: engenheira agrônoma que usará os relatórios).

Diversidade de perfis enriquece os testes.

Roteiro de tarefas para teste

1. Verificar nível de umidade no painel.
2. Ativar irrigação manualmente.
3. Consultar histórico da última semana.
4. Gerar relatório em PDF.

Importante: não explicar como fazer — apenas observar.

Coleta de feedback

- Anotar **reações, dúvidas, sugestões e expressões faciais**.
- Fazer **perguntas abertas** após o teste:
 - O que achou mais fácil?
 - Se confundiu em alguma etapa?
 - O que mudaria?

Exemplo de ajustes feitos após testes

- **Cores ajustadas:**
 - Roxo substituído por vermelho para indicar alerta.
 - Verde usado para botão “Ligar irrigação” (ação positiva).
- **Legendas adicionadas:**
 - “Ative a irrigação” para facilitar entendimento.
- **Botão ampliado:**
 - Para facilitar o uso em tablets e ambientes de trabalho.

Checklist de um bom teste

- Testado com usuários reais e diversos.
- Usuário explorou o protótipo sem ajuda.
- Comportamentos e dúvidas foram anotados.
- Protótipo foi ajustado com base nos testes.
- Fluxo está intuitivo e funcional.

Conclusão

Testar é **respeitar o usuário** e aprender com ele.

Design Thinking é um ciclo, não uma linha reta — podemos sempre voltar às etapas anteriores.

Inovar é ouvir, testar e melhorar, com humildade e foco na experiência humana.

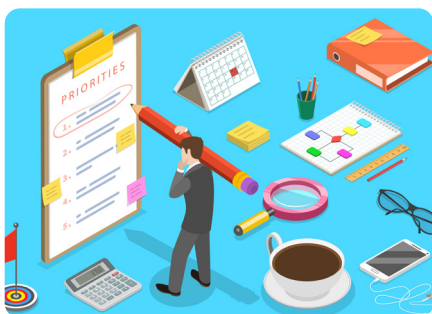
O importante a se fazer logo no início dessa etapa é mapear as funcionalidades da solução, descobrindo assim seu escopo e sua estrutura.

Este mapa da solução deve ter:



Clareza

Para que seja possível entendê-lo rapidamente.



Níveis de prioridade

Das funcionalidades da mesma categoria.



Padronização

Que permita identificar funcionalidades da mesma categoria



Potencial

Para crescer e melhorar, tornando a ideia flexível ao longo do tempo.



Escalabilidade

Permitindo que a ideia se expanda rapidamente pelo mercado.

Entendemos que o Design Thinking, com sua abordagem empática e interativa, se apresenta como uma poderosa ferramenta para a resolução de problemas que é centrada no ser humano. No decorrer de suas etapas bem definidas – desde a imersão nas necessidades dos usuários até a implementação e validação das soluções –, ele fomenta a inovação, a colaboração e o aprendizado contínuo. Ao transformar desafios em oportunidades de criação, o Design Thinking não só entrega produtos e serviços eficazes, como também promove um impacto positivo e duradouro na vida das pessoas, sempre contando com espaço para ajustes e melhorias constantes.

Da ideia à implementação

Segue um exemplo de um fluxo completo de ponta a ponta de Design Thinking.

5. Empatia:

- Entrevistar pacientes e médicos.
- Observar como, atualmente, eles marcam consultas.
- Identificar frustrações com sistemas existentes.

6. Definição:

Percebe-se que os pacientes têm dificuldade em encontrar horários disponíveis e os médicos querem uma agenda automatizada.

7. Ideação:

Propor soluções como: filtro por especialidade, integração com WhatsApp, lembretes por SMS e calendário dinâmico.

8. Prototipagem:

- Criar telas simples do app com fluxo de marcação de consulta.
- Simular funcionalidades principais com ferramentas, como Figma.

9. Testes:

- Usuários testam o protótipo.
- Descobre-se que o botão de agendamento está pouco visível e o login é demorado.

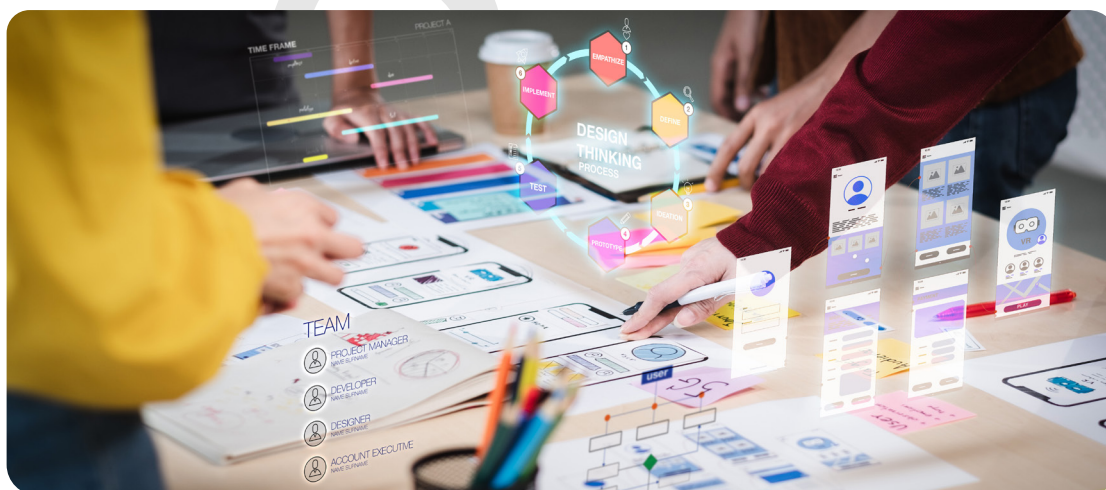
10. Implementação:

- Após ajustes, o app é desenvolvido com Flutter + Firebase.
- Publicação nas lojas e início da operação real.

Encerramento

Nesta aula, estudamos como o Design Thinking se destaca como uma abordagem centrada no usuário que valoriza a empatia e a colaboração para gerar soluções realmente eficazes. Passamos pelas etapas de Empatia, Definição, Ideação, Prototipação e Teste, reforçando a importância de cada uma delas para garantir que as necessidades e desejos do usuário sejam priorizados. Além disso, destacamos a relevância do Mapa de Empatia, da observação e das entrevistas para colher informações cruciais antes de formular as possíveis soluções. Ao aplicar essa metodologia, podemos criar projetos mais inovadores, engajadores e capazes de atender, de maneira mais assertiva, às demandas reais dos usuários.

Nas próximas aulas, estudaremos como fazer o gerenciamento de requisitos da melhor forma, agora que utilizamos Design Thinking para entender mais da realidade do nosso cliente e o que pode ser desenvolvido para auxiliá-lo, vamos aprender a documentar estes requisitos.



4

Gerenciamento e validação de requisitos

Apresentação

Nesta aula, **exploraremos o gerenciamento de requisitos**, aprofundando como **planejaremos, rastreamos, controlaremos e validaremos** as mudanças ao longo de todo o ciclo de vida de um projeto. É crucial termos os requisitos definidos e muito claros, mas eles serão inúteis se não os mantivermos atualizados em conformidade com o projeto! Por isso, **aprenderemos** por que esse processo funciona como um “maestro da orquestra” dos requisitos, alinhando as expectativas dos usuários com as restrições técnicas e de negócios.

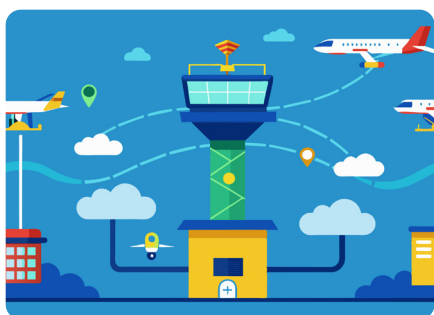
Estudaremos, também, a importância de documentar cada alteração, garantindo rastreabilidade e evitando retrabalho. Por fim, **conheceremos** técnicas de validação, como revisões, além de ferramentas que **ajudarão** no controle de versões, a fim de manter o histórico de cada mudança no projeto. Isso **fortalecerá** a qualidade do sistema, assegurando a entrega de valor real aos usuários e atendendo às necessidades do projeto.

Gerenciamento e validação de requisitos

Definição

O gerenciamento de requisitos é o processo de planejar, rastrear, controlar e validar os requisitos de um sistema de softwares. Envolve a criação de um plano para gerenciar os requisitos ao longo do ciclo de vida do projeto, definir processos para rastrear as mudanças nos requisitos, implementar mecanismos para controlar essas mudanças e realizar atividades de validação, a fim de garantir que os requisitos estejam corretos e completos.

Em outras palavras, o gerenciamento de requisitos é como um “maestro” que coordena a “orquestra” dos requisitos, garantindo que todas as ferramentas estejam em equilíbrio e que o processo ocorra adequadamente.



Garantia da qualidade

Garante que o sistema final atenda às necessidades dos usuários e seja entregue com a qualidade esperada.

Exemplo: a elaboração de um projeto para um sistema de controle de tráfego aéreo. É possível garantir que o sistema atenda a todos os requisitos de segurança e desempenho, evitando falhas que poderiam ter consequências



Redução de riscos

Ajuda a identificar e mitigar os riscos associados às mudanças nos requisitos, evitando retrabalho e custos adicionais.

Exemplo: em um projeto de desenvolvimento de um aplicativo bancário, é possível identificar e mitigar riscos como requisitos conflitantes ou ambíguos, que poderiam levar a erros de programação e a vulnerabilidades de segurança.

Melhora na comunicação

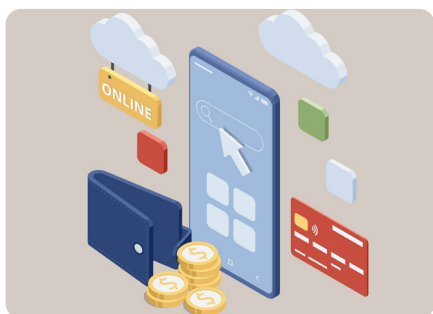
Facilita a comunicação entre as partes interessadas (stakeholders), garantindo que todos tenham um entendimento comum dos requisitos do sistema.



Exemplo: em um projeto de desenvolvimento de um sistema de gestão de projetos, é possível facilitar a comunicação entre os membros da equipe, os clientes e outros stakeholders, garantindo que todos tenham um entendimento comum dos objetivos e dos requisitos do projeto.

Aumento da produtividade

Ajuda a otimizar o processo de desenvolvimento, garantindo que os recursos sejam utilizados de maneira eficiente.



Exemplo: em um projeto de desenvolvimento de um sistema de e-commerce, é possível otimizar o processo, garantindo que os desenvolvedores trabalhem em funcionalidades que realmente agreguem valor ao negócio, evitando retrabalho e desperdício de recursos.

Rastreabilidade

Permite rastrear a origem de cada requisito, facilitando a identificação de impactos em virtude das mudanças e a garantia de que todos os requisitos sejam implementados.

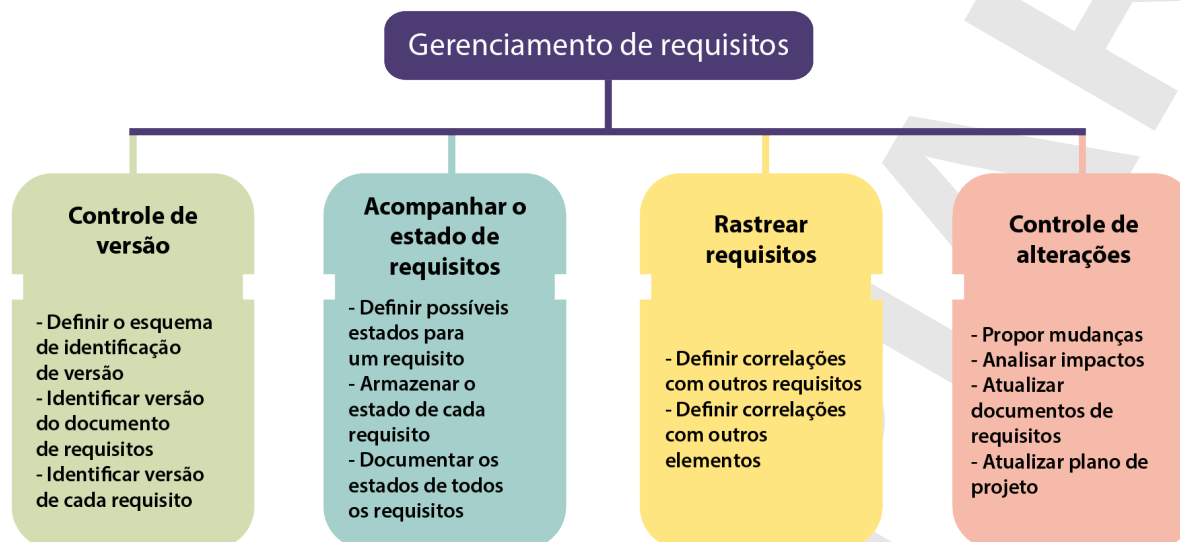


Exemplo: em um projeto de desenvolvimento de um sistema de prontuário eletrônico, é possível rastrear a origem de cada requisito, facilitando a identificação de impactos causados pelas mudanças e a garantia de que todos os requisitos sejam implementados corretamente, conforme as regulamentações da área da saúde.



O que é gerenciamento de requisitos?

Dada a definição de requisitos, agora estudaremos as principais vantagens do gerenciamento de requisitos eficaz.



Saiba mais a seguir, sobre como é possível utilizar a gestão de requisitos da melhor forma, a fim de manter o projeto atualizado e bem documentado.

vídeo resumido

Introdução à Gestão de Requisitos

Objetivo da aula:

Apresentar a importância da **gestão de requisitos** no desenvolvimento de sistemas, com foco em evitar desperdícios, retrabalho e garantir soluções úteis para o cliente.

O que são requisitos?

- **Funcionais:** descrevem o que o sistema deve fazer.
 - Ex: enviar alerta com um clique.
- **Não funcionais:** tratam de desempenho, segurança, usabilidade.
 - Ex: funcionar offline por até 24 horas.

Exemplo prático: Sr. Antônio (produtor rural)

Necessidades levantadas:

- Alertas quando umidade < 30%.
- Painel funcionando sem internet.
- Relatórios automáticos em PDF para auditoria.

Problemas evitados com boa gestão:

- Sistema com dados em tempo real sem 4G.
- Login complexo para celular básico.
- Auditor sem perfil de acesso.

Atividades da gestão de requisitos

1. **Elicitação:** ouvir e observar o cliente.
2. **Análise e negociação:** resolver conflitos e definir o que é viável.
3. **Documentação:** escrever requisitos de forma clara e acessível.
4. **Validação:** revisar com o cliente antes de construir.
5. **Gestão de mudanças:** registrar e controlar alterações ao longo do projeto.

Ferramentas utilizadas

- **Google Docs:** log de requisitos.
- **Figma:** protótipos visuais.
- **Planilhas:** registro de mudanças e impactos.
- **WhatsApp:** validação com fotos, áudios e vídeos.

Boas práticas

- Coletar requisitos com empatia.
- Analisar com foco na rotina real.
- Documentar de forma acessível.
- Validar antes de construir.
- Registrar todas as mudanças.

Conclusão

A gestão de requisitos garante que o sistema **respeite a realidade do cliente**.

O analista atua como **ponte entre o mundo real e o digital**, ouvindo, organizando e validando.

Tudo começa com uma escuta atenta e uma gestão bem estruturada.

Gerente de requisitos



O gerente de requisitos desempenha um papel fundamental, atuando como um elo entre usuários, desenvolvedores e demais partes interessadas. Ele é o responsável por administrar todo o processo de gerenciamento de requisitos, desde o planejamento inicial até a validação final.

Entre as principais funções do gerente de requisitos, podemos destacar:

- Liderar e coordenar as atividades de levantamento, análise, documentação e controle de mudanças dos requisitos;
- Facilitar a comunicação e a colaboração entre as partes interessadas, garantindo o alinhamento das expectativas;
- Priorizar requisitos com base em valor comercial e viabilidade técnica;
- Garantir a rastreabilidade dos requisitos ao longo de todo o ciclo de vida do desenvolvimento do projeto.

Considere a seguinte situação: um projeto de desenvolvimento de um aplicativo móvel de lista de tarefas.

A seguir conheça algumas atribuições do gerente de requisitos:

- **Atribuição 1**
Organiza workshops com os futuros usuários, a fim de entender suas necessidades e expectativas, como a priorização de tarefas por data, a criação de listas compartilhadas e a integração com calendários.
- **Atribuição 2**
Documenta esses requisitos em um documento de especificação detalhado.

- **Atribuição 3**

Prioriza as funcionalidades a serem implementadas em cada versão do aplicativo, definindo quais tarefas serão incluídas na versão inicial e quais serão adicionadas em atualizações futuras.

- **Atribuição 4**

Gerencia as mudanças que surgem ao longo do projeto, como a solicitação de novos campos de informação para cada tarefa ou a necessidade de integração com outros aplicativos de produtividade.

Agora analise outro cenário: o desenvolvimento de um sistema de gestão empresarial para uma rede de lojas.

- **Atribuição 1**

Atua como um negociador entre os diferentes departamentos da empresa, como os departamentos de vendas, de estoque e financeiro, cada um com as próprias necessidades e prioridades, como a geração de relatórios de vendas personalizados, o controle de estoque em tempo real e a integração com sistemas de pagamento online.

- **Atribuição 2**

Garante que todos os requisitos sejam considerados, negociando-os e priorizando-os para assegurar a entrega de um sistema que atenda às necessidades de toda a organização, como a implementação de um Sistema de Gestão de Clientes (CRM) integrado, que permita o acompanhamento das vendas e a personalização do atendimento.

Em suma, o gerente de requisitos é essencial para garantir que o software desenvolvido esteja alinhado com os objetivos de negócio e que atenda às expectativas dos usuários, desde o planejamento inicial, em que ele define como as mudanças de requisitos do aplicativo de lista de tarefas serão gerenciadas, até a validação final, garantindo que todas as mudanças atendam aos critérios de qualidade, como a usabilidade, a segurança e o desempenho.

Gestão de mudanças



No mundo dinâmico do desenvolvimento de softwares, a mudança é a única constante. As necessidades dos usuários evoluem, as tecnologias se transformam e os requisitos inevitavelmente se modificam. Ignorar essa realidade pode levar a projetos obsoletos, sistemas que não atendem às necessidades e às expectativas e, em última análise, levam ao fracasso. É aqui que entra a gestão de mudanças, um processo crucial para garantir que os requisitos se adaptem à nova realidade sem comprometer a qualidade do sistema.

A gestão de mudanças consiste no processo de identificar, avaliar, controlar e documentar as alterações nos requisitos de um sistema de softwares ao longo do ciclo de vida do projeto. Envolve a criação de um mecanismo formal para solicitar, analisar e aprovar mudanças nos requisitos, garantindo que as partes interessadas estejam cientes e que os impactos das mudanças sejam avaliados de maneira adequada.

A importância da gestão de mudanças



Adaptação às necessidades

A gestão de mudanças permite que o sistema se adapte às mudanças nas necessidades dos usuários e do negócio, garantindo que permaneça relevante e útil.

Exemplo: em um aplicativo de banco online, a gestão de mudança permite implementar um sistema de pagamento por aproximação, não disponível anteriormente.



Redução de riscos

A gestão de mudanças ajuda a identificar e mitigar os riscos associados às mudanças nos requisitos, evitando retrabalho, atrasos e custos adicionais.

Exemplo: em um sistema de gestão de estoque para supermercados, é preciso integrar um novo sistema de rastreamento de produtos. A gestão de mudanças ajuda a identificar os riscos dessa integração, os possíveis conflitos com o sistema existente e a implementar medidas para mitigá-los.



Melhora na comunicação

A gestão de mudanças facilita a comunicação entre as partes interessadas, garantindo que todos estejam cientes das mudanças e de seus impactos.

Exemplo: em um projeto de um sistema de prontuário eletrônico para um hospital, a equipe recebe uma solicitação para adicionar um novo campo de informação para registro das alergias dos pacientes. A gestão de mudanças garante que todas as partes interessadas sejam informadas e que o sistema atenda às necessidades de todos.



Rastreabilidade

A gestão de mudanças permite rastrear a origem de cada mudança nos requisitos, facilitando a identificação de impactos e garantindo que todas as mudanças sejam implementadas da maneira correta.

Exemplo: em um sistema de gestão de projetos, um cliente solicita a alteração de um requisito relacionado à geração de relatórios. A gestão de mudanças permite rastrear a origem dessa alteração, identificar os requisitos afetados e garantir que todas as mudanças sejam implementadas.

Processo de gestão de mudanças

A gestão de mudanças em requisitos é um processo crucial para garantir que o software continue atendendo às necessidades do cliente, mesmo com as inevitáveis evoluções e os ajustes que ocorrem ao longo do projeto. Um processo bem

definido garante que as mudanças sejam avaliadas, priorizadas e implementadas de maneira controlada e eficiente.

- **Solicitação de mudança**

O cliente, um usuário ou a equipe de testes, identifica a necessidade de uma alteração e formaliza a solicitação. Por exemplo, em um aplicativo de entrega de comida, o cliente pode solicitar a inclusão de uma opção de pagamento por pix. Essa solicitação deve ser documentada, incluindo a descrição da mudança, sua justificativa e o solicitante.

- **Análise de impacto**

A equipe de desenvolvimento, junto com o gerente de requisitos, avalia o impacto da mudança proposta. Essa análise considera os aspectos técnicos, como o esforço de desenvolvimento e os potenciais riscos, o impacto no cronograma e no orçamento do projeto e as consequências para outras funcionalidades do sistema. No exemplo anterior, a equipe avaliaria o esforço para integrar a função “pagamento por pix”, o tempo necessário para implementar a mudança e se ela afetaria as outras formas de pagamento.

- **Aprovação da mudança**

Com base na análise de impacto, a mudança é submetida a um processo de aprovação. O responsável pela aprovação, geralmente o gerente de projeto ou o cliente, considera os custos e os benefícios da mudança, sua prioridade em relação a outras funcionalidades e o impacto no projeto como um todo. A aprovação pode ser formal, por meio de um documento assinado, ou informal, por meio de uma comunicação por e-mail.

- **Implementação da mudança**

Após a aprovação, a equipe de desenvolvimento implementa a mudança no sistema. Essa etapa envolve a codificação, os testes e a integração da nova funcionalidade ou a correção do problema identificado. No caso do pix, a equipe implementaria a nova opção de pagamento, testaria sua integração com a plataforma de pagamentos e garantiria que não houvesse conflitos com outras funcionalidades.

- **Documentação da mudança**

A mudança implementada é documentada nos artefatos relevantes, como o documento de requisitos, o plano de testes e o código-fonte. Essa documentação garante a rastreabilidade da mudança e facilita a manutenção e a evolução do sistema no futuro. A documentação deve incluir a descrição da mudança, a data de implementação, os responsáveis pela implementação e os testes realizados. No exemplo

do pix, a documentação incluiria a descrição da nova funcionalidade, a data em que foi disponibilizada no aplicativo e os testes realizados para garantir seu correto funcionamento.

Cada etapa do processo de gestão de mudanças tem um papel fundamental para garantir que as alterações nos requisitos ocorram de maneira organizada, transparente e eficiente.

Seguir essas etapas ajuda a minimizar riscos, evitar o retrabalho e manter o alinhamento entre equipe, cliente e demais partes envolvidas. Afinal, mudanças são inevitáveis, mas, quando bem gerenciadas, elas se tornam oportunidades de melhoria, não de caos.

!! Importante

- Sempre utilize mecanismos formais para solicitar e aprovar mudanças nos requisitos.
- Avalie o impacto de cada mudança nos requisitos antes de implementá-las.
- Comunique as mudanças nos requisitos a todas as partes interessadas.
- Documente todas as mudanças nos requisitos de maneira clara e precisa.

Validação de requisitos

A validação de requisitos é o processo de confirmar que os requisitos definidos para um sistema de software são adequados e atendem aos objetivos do negócio e às necessidades dos usuários. Envolve a revisão dos requisitos com os stakeholders para garantir que estejam completos, corretos, consistentes e viáveis.

Essa etapa da validação envolve o processo de confirmar que os requisitos definidos para um sistema de software são adequados e atendem aos objetivos do negócio e às necessidades dos usuários. Envolve a revisão dos requisitos com os stakeholders para garantir que estejam completos, corretos, consistentes e viáveis.



Reduzir erros

Ajuda a identificar e corrigir erros nos requisitos antes que eles sejam implementados no sistema, evitando retrabalho e custos adicionais.



Melhorar a comunicação

Facilita a comunicação entre as partes interessadas, garantindo que todos tenham o mesmo entendimento dos requisitos do sistema.



Aumento de satisfação do cliente

Garante que o sistema final atenda às expectativas dos usuários, aumentando a satisfação do cliente.



Rastreabilidade

Permite rastrear a origem de cada requisito, facilitando a identificação de impactos de mudanças e a garantia de que todos os requisitos sejam implementados da maneira correta.

Técnicas

As técnicas de validação são os métodos que utilizamos para verificar se os requisitos estão corretos, completos, consistentes e viáveis. A importância da validação reside em sua capacidade de identificar e corrigir erros nos requisitos antes que eles sejam implementados no sistema, evitando retrabalho, custos adicionais e insatisfação do cliente.

Existem diversas técnicas que podem ser utilizadas na validação de requisitos, cada uma com suas características e seus objetivos específicos.



Revisões

Reuniões com as partes interessadas para revisar os requisitos, identificar erros, inconsistências e ambiguidades e garantir que todos tenham o mesmo entendimento dos requisitos.

Exemplo: em um projeto de desenvolvimento de um aplicativo de e-commerce, o departamento de marketing identifica que um requisito relacionado à personalização de ofertas para clientes fiéis não está claro o suficiente. A equipe, então, revisa e esclarece o requisito, garantindo que todos os envolvidos tenham o mesmo entendimento.



Protótipos

Criação de modelos interativos para simular o comportamento do sistema e validar os requisitos com os usuários, permitindo identificar problemas de usabilidade e garantir que ele atenda às expectativas dos usuários.

Exemplo: em um projeto de desenvolvimento de um sistema de gestão hospitalar, a equipe cria um protótipo interativo da interface de agendamento de consultas; durante a validação, os médicos identificam que o sistema não permite visualizar a disponibilidade de todos os médicos em um único calendário. A equipe, então, modifica o protótipo, garantindo que o sistema atenda às necessidades dos usuários.



Casos de uso

Descrição das interações entre os usuários e o sistema para identificar os requisitos funcionais e validar se o sistema permite realizar as tarefas desejadas pelos usuários.

Exemplo: em um projeto de desenvolvimento de um aplicativo de banco online, a equipe elabora casos de uso com exemplos de como "realizar transferência bancária" e "consultar extrato". Durante a validação, identifica-se que um caso de uso não considera o usuário ter mais de uma conta bancária, o que poderia gerar erros. A equipe, portanto, modifica o caso de uso para incluir essa possibilidade.



Testes de aceitação

Criação de testes que simulam o comportamento do usuário final para verificar se o sistema atende aos requisitos definidos e se está pronto para ser entregue.

Exemplo: em um projeto de sistema de gestão de projetos, a equipe cria testes que simulam o comportamento do gerente de projetos ao elaborar e atribuir tarefas. Eles identificam que o sistema não permite atribuir tarefas a mais de um usuário. Então, corrigem o sistema e executam os testes novamente, garantindo que atenda a todos os requisitos definidos.

Ferramentas



No contexto da validação de requisitos, as ferramentas representam os softwares e as plataformas que auxiliam as equipes de desenvolvimento a garantir que os requisitos estejam corretos, completos, consistentes e viáveis. A importância dessas ferramentas reside em sua capacidade de automatizar e otimizar o processo de validação, facilitando a identificação de erros, a colaboração entre as partes interessadas e a rastreabilidade dos requisitos ao longo do ciclo de vida do projeto.

Existem diversas ferramentas que podem ser utilizadas na validação de requisitos, cada uma com suas funcionalidades e características específicas.

- **Ferramentas de gerenciamento de requisitos**

Softwares como Jira (gerencia tarefas e projetos), Confluence (cria e organiza documentação) e Visure Requirements (gerencia requisitos de sistemas) permitem criar, rastrear e gerenciar os requisitos do sistema, facilitando a identificação de inconsistências e a rastreabilidade dos requisitos ao longo do ciclo de vida do projeto.

- **Ferramentas de prototipação**

Plataformas como Figma, Adobe XD e Axure RP permitem criar protótipos interativos do sistema para simular o comportamento do sistema e validar os requisitos com os usuários, permitindo identificar problemas de usabilidade e garantir que o sistema atenda às expectativas dos usuários.



Dica

Na plataforma SENAI PLAY, o curso “FIGMA: Prototipação de aplicativos” permitirá criar wireframes (esboços) e layouts para auxiliar no desenvolvimento de softwares e aplicativos. Acesse o link a seguir.

[SENAI Play | Plataforma de cursos online](#)

Para atender aos diferentes momentos e necessidades do processo de validação, há uma variedade de ferramentas disponíveis no mercado. Elas vão desde plataformas

de gerenciamento de requisitos, que auxiliam na organização e na rastreabilidade, até ferramentas de prototipação e testes, que aproximam o sistema da experiência real do usuário.

Cada uma dessas ferramentas contribui para tornar o processo mais preciso, ágil e colaborativo — apoiando o desenvolvimento de soluções que realmente atendam às expectativas dos clientes e usuários finais.

Leia a seguir e conheça exemplos de ferramentas utilizadas em diferentes etapas da validação de requisitos.

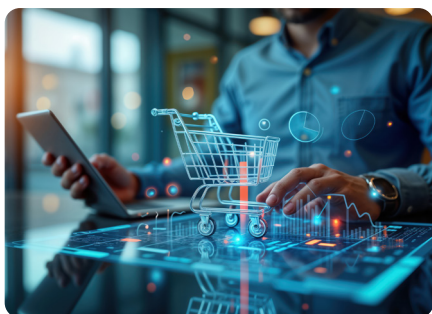
- **Ferramentas de teste**

Softwares como Selenium, JUnit e TestComplete permitem criar testes de aceitação automatizados para verificar se o sistema atende aos requisitos definidos, garantindo a qualidade do sistema e a satisfação do cliente.

- **Ferramentas de modelagem UML**

Ferramentas como o Astah UML, auxiliam na criação de diagramas de casos de uso e outros diagramas UML para modelar o sistema e validar os requisitos com as partes interessadas.

Exemplo:



Em um projeto de um aplicativo de e-commerce, a validação de requisitos pode envolver a criação de protótipos do aplicativo para simular o processo de compra, a criação de casos de uso para descrever como os usuários navegam no aplicativo e a criação de testes de aceitação para verificar se o aplicativo permite realizar compras com sucesso. Além disso, a equipe realiza testes de desempenho para garantir que o tempo de resposta do aplicativo seja inferior a três segundos, atendendo ao requisito não funcional de performance.



Dica

Na plataforma SENAI PLAY, o vídeo “Teste com Selenium” exibe uma atividade de criação de teste automatizado utilizando essa importante ferramenta. Acesse o link para assisti-lo.

[Testes com Selenium](#)

Dicas para o bom andamento no processo de validação de requisitos:



Envolva as partes interessadas em todas as etapas da validação de requisitos.



Utilize diferentes técnicas de validação para garantir que os requisitos sejam validados de maneira completa.



Documente os resultados da validação de requisitos para facilitar a rastreabilidade e a auditoria.

A validação de requisitos é fundamental para o sucesso do projeto. Utilize revisões para identificar falhas, protótipos para simular a experiência do usuário, casos de uso para mapear interações e testes de aceitação para garantir a qualidade final. Cada passo é crucial para construir um software que atenda às expectativas e às necessidades reais.

Gestão de mudanças nos requisitos

Já que estamos falando sobre gestão de requisitos, segue um resumo das principais etapas! Quando aplicadas juntas, elas fazem muita diferença no desenvolvimento de um sistema saudável a longo prazo.



Validação de requisitos

Agora que você já conhece as melhores técnicas de levantamento de requisitos e como gerir todos eles, uma etapa essencial é a validação desses requisitos. Conheça

algumas técnicas para validação de requisitos, as quais serão utilizadas para validar, junto com outras pessoas, se os requisitos que foram levantados realmente fazem sentido.

1. Revisões de requisitos

- Verificação detalhada com a equipe do projeto.
- Garante que os requisitos estejam completos, corretos e consistentes
- Inclui leitura crítica e checklist de qualidade.

2. Prototipação

- Criação de versões simplificadas do sistema.
- Permite a validação visual e funcional dos requisitos.
- Facilita o entendimento de requisitos ambíguos.

3. Entrevistas com stakeholders

- Conversas diretas com usuários e partes interessadas.
- Esclarece expectativas e identifica possíveis erros.
- Ideal para capturar feedbacks qualitativos.

4. Testes de aceitação

- Requisitos são convertidos em cenários de teste.
- O sistema é validado conforme os critérios definidos.
- Reforça a rastreabilidade dos requisitos.

5. Workshops colaborativos

- Sessões interativas com usuários e equipe técnica. Ajuda a resolver conflitos e alinhar expectativas.
- Promove consenso sobre funcionalidades.

6. Feedback contínuo

- Validação constante durante o desenvolvimento.
- Uso de metodologias ágeis (por exemplo, scrum reviews).
- Adapta requisitos com base em entregas parciais.

!! Importante

Importância da validação

- Evita retrabalho e desperdício.
- Garante que o produto final **atenda às reais necessidades** dos usuários.
- Aumenta a **qualidade**, a **satisfação** e o sucesso do projeto.

Enquanto na etapa de validação veremos se os requisitos que definimos realmente fazem sentido, na etapa de controle de versões saberemos como é possível manter salvas as versões dos requisitos dentro do software.

A seguir, confira quão importantes são as etapas de validação e de controle de versões.

▶ vídeo resumido

Validação e Controle de Versões

Objetivo da aula:

Apresentar a importância da **validação dos requisitos** e do **controle de versões** para garantir que o sistema atenda às necessidades reais do usuário e mantenha rastreabilidade ao longo do projeto.

Validação de Requisitos

- Um requisito só é válido quando o **usuário confirma** que ele atende às suas necessidades.
- A validação evita erros e retrabalho, garantindo que o projeto siga o caminho certo desde o início.

Formas de validar:

- **Revisão em grupo:** leitura coletiva dos requisitos.
- **Protótipos navegáveis:** simulação prática no Figma.
- **Casos de uso simulados:** histórias que ilustram o funcionamento.
- **Checklist de validação:** verificar se o requisito é completo, claro e testável.

Exemplo prático:

Requisito inicial: "Sistema deve permitir acionamento remoto da bomba."

Após validação: "Somente o proprietário ou eletricista pode ativar remotamente via código."

Controle de Versões

- Cada alteração nos requisitos gera uma nova versão do sistema.
- Sem controle, há risco de desalinhamento entre cliente, equipe e documentação.

Boas práticas:

- Usar **planilhas** com colunas: código do requisito, descrição, versão, data, responsável, justificativa.
- Ferramentas úteis: Google Docs, Notion, GitHub, Jira, Trello.
- **Não editar diretamente** — sempre criar nova versão.
- Validar toda nova versão antes de implantar.

Exemplo de controle:

- Versão 1.0: alerta por e-mail.
- Versão 1.1: alerta por WhatsApp (justificativa: cliente não usa e-mail).

Prototipação alinhada às versões

- Duplicar frames no Figma com nome da versão.
 - Ex: “Painel de irrigação v1.0” “Painel de irrigação v1.1”
- Garante que o **protótipo reflita os requisitos atualizados**.

Checklist prático

- Requisito aprovado pelo usuário.
- Documentado de forma clara e objetiva.
- Possui código único (ex: RF 03).
- Tem versão registrada.
- Mudanças justificadas e rastreadas.
- Protótipo alinhado com a versão mais recente.

Conclusão

Validação e controle de versões **evitam erros, aumentam a confiança do cliente e garantem que o sistema seja útil e sustentável**.

Assim como uma fazenda precisa de cuidado constante, o desenvolvimento de sistemas exige **atenção aos requisitos** para alcançar o sucesso.

Documentação de requisitos



A documentação de requisitos é a espinha dorsal de qualquer projeto de software bem-sucedido. Ela garante que todos os envolvidos compreendam os objetivos, as funcionalidades e as restrições do sistema, facilitando a comunicação, o desenvolvimento e a validação. Para alcançar esse nível de clareza e precisão, é fundamental seguir normas técnicas e adotar as melhores práticas de documentação.

Normas técnicas



Normas como a IEEE 29148 e a ISO/IEC 29148 oferecem um framework robusto para a engenharia de requisitos. Elas detalham os processos de especificação, validação e gerenciamento de requisitos, abrangendo todo o ciclo de vida do software. Ao seguir essas normas, você garante que seus documentos de requisitos sejam consistentes e rastreáveis. Além disso, a ISO/IEC/IEEE 15288 fornece um modelo de ciclo de vida do sistema e do software que pode ser utilizado para estruturar o processo de desenvolvimento e garantir a conformidade com as melhores práticas da indústria.

Normas IEEE 29148 e ISO/IEC 29148

As normas IEEE 29148 e ISO/IEC 29148 são pilares fundamentais na engenharia de requisitos, oferecendo um guia completo para a especificação, a validação e o gerenciamento de requisitos em projetos de software. Embora compartilhem o mesmo conteúdo técnico, a IEEE 29148 é uma norma americana, enquanto a ISO/IEC 29148 é a versão internacional, garantindo sua aplicabilidade global.

Estrutura e conteúdo abrangente

Ambas as normas abrangem todo o ciclo de vida dos requisitos, desde a elicitação inicial até a validação final.

Conheça a seguir as diretrizes detalhadas que elas fornecem.

Elicitação de requisitos

- Técnicas para identificar e coletar requisitos de diversas fontes, como stakeholders, documentos e sistemas existentes.
- Abordagens para lidar com requisitos conflitantes e ambiguidades.

Especificação de requisitos

- Métodos para documentar requisitos de maneira clara, concisa e precisa, utilizando linguagem natural ou modelos formais.
- Estrutura para criar documentos de requisitos, como o Documento de Requisitos de Software (DRS).

Validação de requisitos

- Técnicas para verificar se os requisitos atendem às necessidades dos stakeholders e se são consistentes com os objetivos do projeto.
- Abordagens para realizar revisões, inspeções e testes de requisitos.

Gerenciamento de requisitos

- Processos para controlar as mudanças nos requisitos ao longo do ciclo de vida do projeto.
- Técnicas para rastrear os requisitos desde sua origem até sua implementação e teste.

Essas diretrizes oferecem uma base sólida e estruturada para lidar com os requisitos de maneira eficaz ao longo de todo o projeto. Ao aplicar essas boas práticas, as equipes aumentam a clareza, reduzem o retrabalho e garantem maior alinhamento com os objetivos do negócio e as expectativas dos usuários.

Para se aprofundar e explorar mais detalhes sobre essas normas e outras referências que orientam o processo de documentação e levantamento de requisitos, confira a recomendação a seguir.



Saiba mais

A Organização IEEE SA Standards Association disponibiliza acesso a todas as normas de sua responsabilidade para consulta. Para conhecer mais sobre essa e outras normas que orientam sobre as documentações para ajudar no levantamento de requisitos, acesse o link a seguir:

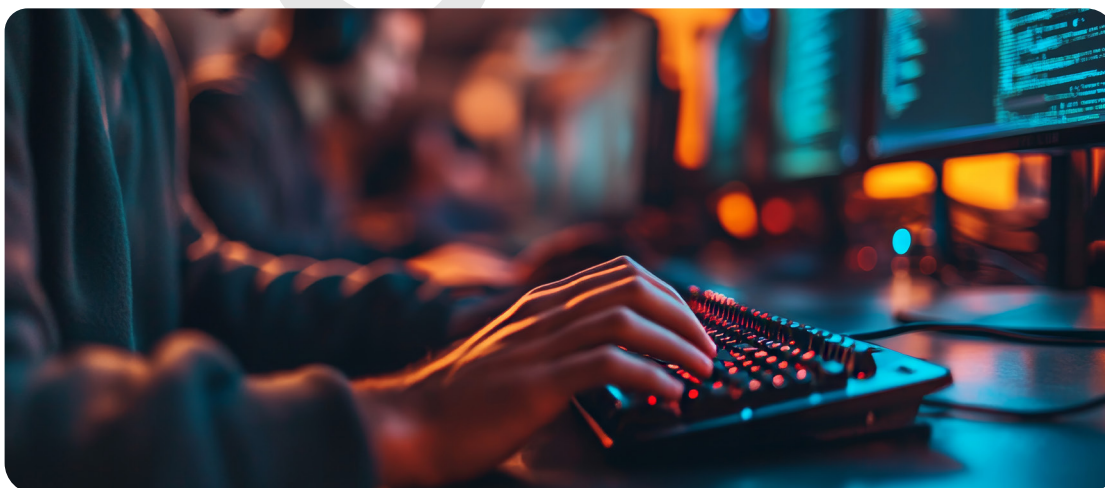
<https://standards.ieee.org/ieee/29148/6937/>

Benefícios da adoção

A adoção das normas IEEE 29148 e ISO/IEC 29148 traz diversos benefícios para os projetos de software. Conheça alguns deles abaixo:

- **Melhoria da qualidade dos requisitos**
As normas garantem que os requisitos sejam completos, consistentes, rastreáveis e testáveis, reduzindo o risco de erros e retrabalho.
- **Aumento da eficiência do desenvolvimento**
A documentação clara e precisa dos requisitos facilita a comunicação entre a equipe de desenvolvimento e os stakeholders, agilizando o processo de desenvolvimento.
- **Redução de custos**
A identificação e a correção de erros nos requisitos no início do ciclo de vida do projeto evitam custos elevados com retrabalho e correções posteriores.
- **Maior satisfação dos clientes**
A garantia de que o software atenda às necessidades dos clientes aumenta a satisfação e a confiança no produto.

Aplicação prática



As normas IEEE 29148 e ISO/IEC 29148 podem ser aplicadas em projetos de software de qualquer porte e complexidade. Elas são especialmente úteis em projetos críticos, em que a qualidade e a confiabilidade dos requisitos são essenciais.

Ao seguir as diretrizes dessas normas, as organizações podem garantir que seus projetos de software sejam desenvolvidos com base em requisitos sólidos e bem definidos, aumentando as chances de sucesso e reduzindo os riscos.

Exemplo de documentação alinhada à IEEE 29148:



Requisitos Funcionais (RF)

RF01: o sistema **deverá** permitir que autores submetam trabalhos em PDF ou DOCX, com tamanho máximo de 5MB.

Critério de aceite: o arquivo enviado aparece na lista de submissões do autor e gera um e-mail de confirmação.

Rastreabilidade: atende à necessidade do usuário USR02 (cadastro de trabalhos).

RF02: o sistema **deverá** atribuir revisores automaticamente com base em palavras-chave dos trabalhos.

Critério de aceite: o revisor logado visualiza apenas trabalhos de sua área (por exemplo, "inteligência artificial").



Requisitos não funcionais (RNF)

RNF01 (Usabilidade): o sistema **deverá** seguir o padrão WCAG 2.1 AA para contraste de cores e leitores de tela.

RNF02 (Desempenho): a página de inscrições **deverá** carregar em menos de dois segundos com 1.000 usuários concorrentes.



Requisitos de Usuário (RU)

USR01: "Como participante, quero filtrar eventos por data para planejar minha agenda."

Traduzido para RF: RF03 (módulo de calendário com filtros).

O exemplo anterior trouxe três pontos importantes a serem notados na aplicação da norma IEEE 29148:

- **clareza:** linguagem direta e verificável (evita ambiguidades);
- **rastreabilidade:** liga requisitos a necessidades de stakeholders (por exemplo, USR01 > RF03);
- **testabilidade:** critérios de aceite mensuráveis (por exemplo, tempo de carregamento).

Norma ISO/IEC/IEEE 15288

A norma **ISO/IEC/IEEE 15288** desempenha um papel crucial na engenharia de sistemas e software, fornecendo um framework padronizado para o ciclo de vida de sistemas. Diferente de normas focadas especificamente em requisitos, como a IEEE 29148, a ISO/IEC/IEEE 15288 adota uma perspectiva mais ampla, abrangendo todos os processos envolvidos no desenvolvimento e na manutenção de sistemas e software.

Escopo e objetivos

A norma ISO/IEC/IEEE 15288 define um conjunto de processos que podem ser aplicados em qualquer tipo de sistema, independentemente de sua complexidade ou seu domínio de aplicação. Ela busca:



Estabelecer um vocabulário comum para a engenharia de sistemas e software.



Fornece um framework para o planejamento, o desenvolvimento, a implementação, a operação e o descarte de sistemas e software.



Facilitar a comunicação e a colaboração entre as partes interessadas em um projeto.



Promover a melhoria contínua dos processos de engenharia de sistemas e software.

Principais processos abordados

A norma ISO/IEC/IEEE 15288 organiza os processos em quatro categorias principais:

- **Processos de acordo**
Definem como os acordos entre as partes interessadas são estabelecidos e gerenciados.
- **Processos organizacionais**
Abrangem os processos de gestão da organização, como gestão de projetos, gestão de riscos e gestão da qualidade.
- **Processos de gerenciamento de projetos**
Definem como os projetos são planejados, executados, monitorados e controlados.
- **Processos técnicos**
Abrangem os processos de engenharia propriamente ditos, como engenharia de requisitos, projeto, implementação, teste e manutenção.

Relevância para a engenharia de requisitos:



Embora a norma ISO/IEC/IEEE 15288 não se concentre exclusivamente em requisitos, ela fornece um contexto importante para o processo de engenharia de requisitos, pois define como os requisitos se encaixam no ciclo de vida do sistema e como eles devem ser gerenciados ao longo do tempo.

Ao seguir as diretrizes da norma ISO/IEC/IEEE 15288, as organizações podem garantir que seus projetos de software sejam desenvolvidos de maneira estruturada e organizada, aumentando as chances de sucesso e reduzindo os riscos.

Benefícios da implementação

Melhora da qualidade dos sistemas e software.

Redução de custos e prazos de desenvolvimento.

Aumento da satisfação dos clientes.

Melhoria da comunicação e colaboração entre os stakeholders.

Implementar a ISO/IEC/IEEE 15288 traz benefícios concretos para o desenvolvimento de sistemas e software. Ao alinhar processos, melhorar a comunicação entre as partes interessadas e promover entregas com mais qualidade para o sucesso dos projetos.

!! Importante

A ISO/IEC/IEEE 15288 não é apenas uma norma, é a bússola que guia o desenvolvimento de sistemas e software, garantindo que cada etapa, desde a concepção até a entrega, seja executada com precisão e qualidade. Ao adotá-la, você não só otimiza processos, mas também constrói a base para projetos de sucesso e clientes satisfeitos.

Exemplo de aplicação da ISO/IEC/IEEE 15288 no desenvolvimento de um sistema de gestão hospitalar.



Desenvolvimento de um Sistema Integrado de Gestão Hospitalar (SIGH) para um hospital de médio porte, com módulos para agendamento de consultas, prontuário eletrônico e gestão de estoque de medicamentos.

1. Processos de acordo

Estabelecimento de contratos e expectativas com os stakeholders.

Exemplo:

- **Acordo com o corpo clínico:** o documento formal define que o sistema deve se integrar aos dispositivos médicos existentes (ex.: aparelhos de ultrassom).

Critério de aceite: assinatura do termo de concordância pelos médicos após validação do protótipo.

2. Processos organizacionais

Gestão da organização e qualidade. Exemplo:

- **Gestão de riscos:** identificação do risco "atraso na integração com o sistema de pagamentos" e do plano de mitigação (contratação de um especialista em APIs).
- **Gestão da qualidade:** checklist semanal para verificar se os requisitos do prontuário eletrônico atendem à LGPD.

3. Processos de gerenciamento de projetos

Planejamento e execução do projeto. Exemplo:

- **Plano de projeto:** cronograma com marcos (milestones) para:
- Entrega do módulo de agendamento em três meses.
- Treinamento dos usuários finais antes da implantação.
- **Monitoramento:** reuniões quinzenais com relatório de progresso (por exemplo, 70% dos requisitos de estoque validados).

4. Processos técnicos

Engenharia do sistema. Exemplo:

- Engenharia de requisitos:
 - **Requisito:** “O sistema **deverá** bloquear o agendamento de consultas se o paciente tiver pendências financeiras.”
 - **Fonte:** necessidade do departamento financeiro (rastreabilidade).
- Projeto do sistema:
 - O diagrama de arquitetura mostra a integração entre os módulos de agendamento e financeiro.
- Implementação:
 - Uso de containers Docker para garantir portabilidade entre ambientes.
- Teste:
 - **Caso de teste:** “Agendar consulta com paciente inadimplente >> Sistema exibe mensagem de bloqueio.”

5. Operação e manutenção

Pós-implantação. Exemplo:

- **Processo de incidentes:** protocolo para registrar falhas no módulo de estoque (por exemplo, medicamentos não atualizados em tempo real).
- **Melhoria contínua:** feedback dos enfermeiros gera um novo requisito: “Alertas para medicamentos próximos do vencimento.”

O exemplo anterior trouxe três pontos importantes a serem notados da aplicação da norma ISSO/IEC/IEEE 15288:

- **Vocabulário comum:** termos como “stakeholder”, “critério de aceite” e “rastreabilidade” seguem a norma.
- **Abordagem estruturada:** todos os processos (técnicos, gerenciais, organizacionais) são cobertos.
- Benefícios tangíveis:

- redução de retrabalho (por exemplo, validação contínua com médicos);
- qualidade garantida (checklists de LGPD).

Estrutura-padrão (modelos de documentação)



Documentar os requisitos de um software de maneira clara, concisa e organizada é crucial para o sucesso do projeto. Uma documentação bem estruturada facilita a comunicação entre as partes interessadas, guia a equipe de desenvolvimento e serve como base para testes e validação. Nesta parte da aula, exploraremos alguns modelos de documentos comumente utilizados no levantamento de requisitos, apresentando suas características, vantagens e desvantagens.

Documento de Requisitos de Software (DRS)

O DRS é o documento essencial que descreve detalhadamente os requisitos do software. Especifica as funcionalidades, as características, as restrições e outras necessidades do sistema, servindo como base para o desenvolvimento, o teste e a validação.



Vantagens

Detalhamento completo, rastreabilidade dos requisitos, base para o desenvolvimento.



Desvantagens

Pode ser extenso e complexo, requer manutenção constante.



Exemplo de conteúdo:

Introdução, descrição geral do sistema, requisitos funcionais, requisitos não funcionais, regras de negócio, restrições, casos de uso, diagramas, glossário.

Exemplo prático de DRS para o sistema de gestão de eventos acadêmicos

1. Introdução

Objetivo: documentar os requisitos do sistema, responsável por gerenciar submissões de trabalhos, inscrições, programação e certificados em eventos acadêmicos.

Escopo: sistema web com módulos para autores, revisores, participantes e organizadores.

2. Descrição geral do sistema

Funcionalidades principais:

- Submissão e revisão de trabalhos.
- Gerenciamento de inscrições e pagamentos.
- Emissão automática de certificados.
- Usuários: autores, revisores, participantes, organizadores.

3. Requisitos funcionais (prioritários)

ID	Descrição	Critério de aceite	Fonte
RF01	O sistema deve permitir a submissão de trabalhos em PDF ou DOCX (até 5MB)	O arquivo aparece na lista do autor + e-mail de confirmação.	Reunião com comitê científico.
RF02	O sistema deve atribuir revisores automaticamente por área de pesquisa.	O revisor logado visualiza apenas trabalhos de sua área (por exemplo "Engenharia").	Norma IEEE 29148 (rastreadibilidade)

4. Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Usabilidade	Interface deve seguir WCAG 2.1 AA (contraste mínimo de 4.5:1 para textos).

RN02	Desempenho	Página de inscrições deve suportar 1.000 usuários simultâneos sem lentidão.
-------------	------------	---

5. Regras de negócio

ID	Categoria	Descrição
RN01	Trabalhos submetidos após o prazo são automaticamente rejeitados.	Sob demanda do organizador (caso justificado).
RN02	Os certificados só são emitidos para participantes com $\geq 75\%$ de presença.	-

6. Restrições

- **Tecnológicas:** o sistema deve ser compatível com navegadores Chrome, Edge e Firefox (versões recentes).
- **Prazos:** implantação piloto em até seis meses.

7. Caso de uso (exemplo simplificado)

Título: submissão de trabalho.

Atores: autor.

Fluxo principal:

- Autor faz login.
- Seleciona "Submeter trabalho".
- Preenche metadados (título, área, coautores).
- Faz upload do arquivo.
- Recebe confirmação por e-mail.

8. Glossário

- **Metadados de trabalho:** título, resumo, palavras-chave, área temática.
- **Revisor:** usuário com permissão para avaliar trabalhos.

Vale destacar algumas vantagens do DRS como detalhamento completo de requisitos com IDs, critérios de aceite e fontes. Sua rastreabilidade, RF02 vinculado à norma IEEE 29148 e suas desvantagens mitigadas, como complexidade controlada, exemplo focado nos requisitos essenciais (evita extensão desnecessária).



Pense nisso

Desafio: Complete o DRS do sistema de eventos acadêmicos com:

- Um **novo requisito funcional** para o módulo de certificados (por exemplo, personalização com logo do evento).
- Uma **regra de negócio** para o pagamento de inscrições (por exemplo, desconto para grupos).

Plano de Gerenciamento de Requisitos (PGR)



Objetivo

Definir como os requisitos serão gerenciados ao longo do projeto, incluindo processos, ferramentas, responsabilidades e estratégias.



Vantagens

Fornece estrutura e controle ao processo, facilita a comunicação e o acompanhamento.



Desvantagens

Requer tempo e esforço para criar e manter, pode ser burocrático se mal aplicado.



Exemplo de conteúdo

Introdução, objetivos, escopo, responsabilidades, ferramentas e tecnologias, processos de gerenciamento de mudanças, métricas e indicadores, plano de comunicação.

Exemplo prático de PGR para o sistema de gestão de eventos acadêmicos – modelo simplificado.

1. Introdução

Objetivo: estabelecer como os requisitos do “EventUA” serão identificados, documentados, rastreados e controlados durante o projeto.

Escopo: aplica-se a todos os requisitos funcionais, não funcionais e regras de negócio do sistema.

2. Responsabilidades

Papel	Atribuições
Product owner	Priorizar requisitos, validar critérios de aceite.
Analista de requisitos	Elaborar documentos (DRS, casos de uso), garantir rastreabilidade.
Comitê de mudanças	Avaliar e aprovar solicitações de alteração de requisitos.

3. Ferramentas e tecnologias

- **Gestão de requisitos:** Jira Software (com plug-in de rastreabilidade).
- **Documentação:** Confluence (para DRS e glossário compartilhado).
- **Comunicação:** Slack (canal #requisitos-eventua).

4. Processo de gerenciamento de mudanças

- Solicitação:** formulário padrão preenchido pelo stakeholder.
- Análise:** impacto técnico e no cronograma avaliado pelo analista.
- Aprovação:** comitê decide em reunião semanal.
- Atualização:** DRS e backlog ajustados (versão documentada).

Exemplo:

Mudança solicitada: "Incluir submissão de vídeos (MP4) para trabalhos."

Decisão: aprovada para a sprint 3, com acréscimo de duas semanas no prazo.

Métricas e indicadores

Métrica	Meta	Frequência
% Requisitos estáveis	≥ 90%	úteis
Tempo médio de aprovação	≤ 5 dias úteis	úteis

5. Plano de comunicação

Stakeholders: reuniões quinzenais (via Zoom) e relatórios no Confluence.

Equipe Dev: atualizações diárias no Slack e revisões na sprint planning.

Vale destacar algumas vantagens do PGR: estrutura clara, papéis e processos definidos evitam ambiguidades. Controle de mudanças: exemplo prático de fluxo para evitar expansão de escopo. As desvantagens mitigadas, como burocracia reduzida e processos enxutos (por exemplo, formulário único para mudanças), também podem ser verificados nesse modelo.



Saiba mais

Desafio: Adapte esse PGR para um sistema de biblioteca universitária, definindo:

- Uma nova responsabilidade (por exemplo, “Bibliotecário chefe valida requisitos de empréstimo”).
- Um indicador relevante (por exemplo, “Taxa de requisitos conflitantes resolvidos por sprint”).

Documento de visão

O documento de visão fornece uma visão geral de alto nível do projeto, descrevendo o propósito do software, seus principais objetivos, o público-alvo e os benefícios esperados. Serve como um guia estratégico para o projeto, alinhando as expectativas das partes interessadas e fornecendo um contexto para os requisitos mais detalhados.



Vantagens

- **Comunicação concisa:**
resume o projeto em 1-2 páginas (ideal para stakeholders não técnicos).
- **Alinhamento:**
exemplo mostra claramente o porquê do sistema (Seções 2 e 3).



Desvantagens

Detalhe limitado: funcionalidades são descritas em alto nível (o detalhamento fica para o DRS).



Exemplo de conteúdo

Nome do projeto, descrição do problema, objetivos, público-alvo, principais funcionalidades, benefícios, riscos e oportunidades.

Exemplo prático de documento de visão para o sistema de eventos acadêmicos

1. Nome do projeto

EventUA – Sistema Integrado de Gestão de Eventos Acadêmicos

2. Descrição do problema

Atualmente, a organização de eventos acadêmicos (como congressos e seminários) enfrenta desafios como:

- **Processos manuais** de submissão e revisão de trabalhos.
- **Falta de integração** entre inscrições, pagamentos e emissão de certificados.
- **Dificuldade de comunicação** entre organizadores, autores e participantes.

3. Objetivos do sistema

- **Automatizar** a gestão de submissões, as revisões e a programação de eventos.
- **Centralizar** inscrições e pagamentos em uma plataforma única.
- **Garantir rastreabilidade** desde a submissão até a emissão de certificados.

4. Público-alvo

Perfil	Necessidades
Autores	Submeter trabalhos, acompanhar avaliações, receber certificados.
Revisores	Avaliar trabalhos, emitir pareceres, gerenciar prazos
Organizadores	Configurar eventos, gerar relatórios, validar certificados.
Participantes	Inscrever-se em atividades, acessar programação, baixar materiais

5. Principais funcionalidades (high-level)

Módulo de submissões: upload de trabalhos com verificação automática de formatação.

Módulo de revisão: atribuição automática de revisores por área temática.

Módulo de certificados: emissão personalizada com assinatura digital.

6. Benefícios esperados

- **Para universidades:** redução de 40% no tempo de organização de eventos.
- **Para participantes:** experiência unificada (do cadastro ao certificado).
- **Para pesquisadores:** maior transparência no processo de revisão por pares.

7. Riscos e oportunidades

Papel	Atribuições	Atribuições
Baixa adesão de usuários iniciais.	Campanha de treinamento para instituições.	Parceria com sociedades científicas.
Atraso na integração com pagamentos.	Contratação de gateway de pagamento confiável.	Expansão para eventos internacionais.

Casos de uso

Os casos de uso descrevem as interações entre o usuário (ator) e o sistema para atingir um objetivo específico. Fornecem uma perspectiva do usuário sobre as funcionalidades do sistema, facilitando a compreensão e a validação dos requisitos.



Vantagens

- **Foco no usuário:**
fluxo centrado nas ações do participante.
- **Base para testes:**
cada passo pode ser transformado em caso de teste (por exemplo, "Testar pagamento recusado").



Desvantagens

Detalhe limitado: Aspectos não funcionais são tratados em outros documentos (por exemplo, RNF no DRS).



Exemplo de conteúdo

Nome do caso de uso, ator, pré-condições, fluxo principal, fluxos alternativos, pós-condições.

Exemplo prático de caso de uso para o sistema de eventos acadêmicos

1. Identificação

Nome do caso de uso: UC03 - Realizar Inscrição em Evento.

Atores:

- **Principal:** participante (usuário cadastrado).
- **Secundário:** sistema de pagamento (integração externa).

2. Pré-condições

O participante **deve** ter uma conta ativa no sistema.

O evento **deve** estar com inscrições abertas.

3. Fluxo principal (6 passos)

Passo	Ação do usuário	Resposta do sistema
1	Acessa a página do evento.	Exibe detalhes (data, programação, preço).
2	Clica em "Inscrever-se".	Solicita a confirmação de dados cadastrais.
3	Confirma os dados e seleciona forma de pagamento (por exemplo, cartão).	Redireciona para gateway de pagamento.
4	Completa os dados de pagamento.	Valida a transação e retorna ao EventUA.
5	-	Envia e-mail de confirmação com comprovante.
6	-	Atualiza o perfil do usuário com "Inscrito".

4. Fluxos alternativos

FA1 - Pagamento recusado

Passo 4 do fluxo principal: se o pagamento falhar:

- Sistema exibe mensagem: "Pagamento não aprovado."
 - Tente novamente ou escolha outra forma."
- Usuário pode:
 - Reinserir dados do cartão.
 - Selecionar boleto bancário.

FA2 - Evento lotado

Passo 2 do fluxo principal: se as vagas esgotarem:

- Sistema exibe: "Inscrições encerradas. Cadastre-se na lista de espera."

5. Pós-condições

Inscrição registrada no banco de dados. Comprovante disponível no perfil do usuário e por e-mail.



Saiba mais

Desafio: Crie um caso de uso para "Avaliar trabalho" (ator: revisor), incluindo:

- Pré-condição: (por exemplo, "Revisor deve ter expertise na área do trabalho").
- Fluxo principal: (3-5 passos).
- Fluxo alternativo: (por exemplo "Conflito de interesses").

Especificação de Requisitos de Software (SRS)

A SRS é um documento formal e detalhado que especifica os requisitos de um sistema de software. É utilizada em projetos complexos e críticos, em que a precisão e a rastreabilidade dos requisitos são essenciais.



Vantagens

- **Rastreabilidade:**
IDs únicos (ex.: SRS-F01) vinculam requisitos a testes.
- **Precisão:**
critérios de aceite mensuráveis (ex.: tempo de resposta).



Desvantagens

Complexidade controlada: exemplo focado nos requisitos críticos (evita sobrecarga).



Exemplo de conteúdo

Introdução, descrição geral do sistema, requisitos específicos, interfaces externas, atributos do sistema, apêndices.

Exemplo prático de SRS para o sistema de eventos acadêmicos baseado na ISSO/IEEE

1. Introdução

Propósito: documentar requisitos completos do sistema "EventUA" para desenvolvimento, validação contratual e auditoria.

Escopo: cobre módulos de submissão, revisão, inscrição e certificação. **Não inclui** integração com redes sociais.

2. Descrição geral

Funcionalidades-chave:

- Submissão de trabalhos com verificação automática de plágio.
- Emissão de certificados com assinatura digital.

Usuários:

Tipo	Acesso
Administrador	CRUD de eventos, relatórios financeiros.
Revisor	Avaliar trabalhos, sugerir melhorias.

3. Requisitos específicos

Requisitos funcionais (exemplo detalhado)

ID	Descrição	Critério de aceite	Prioridade
SRS-F01	O sistema deve validar arquivos submetidos (PDF/DOCX, ≤5MB).	Erro exibido para formatos/tamanhos inválidos.	Alta
SRS-F02	O sistema deve gerar certificados em PDF com QR code para validação.	QR Code redireciona para página de verificação.	Média

4. Requisitos não funcionais

ID	Categoria	Descrição
SRS-NF01	Segurança	Dados de pagamento devem seguir PCI DSS.
SRS-F02	Desempenho	Tempo de resposta ≤2s para 95% das requisições (1.000 usuários concorrentes).

5. Interfaces externas

Gateway de pagamento: integração com API do PagSeguro (métodos: cartão/boleto).

Validação de certificados: consulta via API ao Ministério da Educação (hash + QR code).

6. Atributos do sistema

Confiabilidade: disponibilidade de 99,5% (exceto manutenção programada).

Manutenibilidade: logs detalhados para auditoria de alterações em trabalhos.

7. Apêndices

A.1 Glossário:

- CRUD: Create, Read, Update, Delete.
- PCI DSS: padrão de segurança para dados de cartão.

A.2 Referências:

- Norma ISO/IEC 25010 para qualidade de software.



Saiba mais

Desafio: Complete a SRS para o módulo de “**Revisão por pares**” com:

- **Requisito funcional:** (por exemplo, “O sistema deve ocultar a identidade do autor durante a revisão”).
- **Interface externa:** (por exemplo, “Integração com ORCID para validação de revisores”).

Histórias de usuário

As histórias de usuário são descrições curtas e simples de uma funcionalidade do sistema do ponto de vista do usuário. São comumente utilizadas em metodologias ágeis, facilitando a comunicação e a colaboração entre a equipe e o cliente.



Vantagens

- **Foco no valor:**
todas as histórias partem da perspectiva do usuário (por exemplo, autor, revisor).
- **Flexibilidade:**
os critérios de aceite podem ser refinados a cada sprint.



Desvantagens

- **O baixo detalhamento:** é compensado pelos critérios de aceite (evita ambiguidades).



Exemplo de conteúdo

“Como [tipo de usuário], eu quero [funcionalidade] para que [benefício].”

Exemplo prático de histórias de usuário para o sistema de eventos acadêmicos.

1. Exemplo básico (formato clássico)

Título: Submissão de trabalho

Descrição: **"Como autor, quero enviar** meu trabalho em PDF ou DOCX para que ele seja avaliado pelo comitê científico."^{*}

Critérios de aceite:

- O sistema aceita apenas arquivos PDF ou DOCX (até 5MB).
- Após upload, o autor recebe e-mail de confirmação.
- O trabalho aparece na lista "Submissões pendentes" do autor.

2. Exemplo com detalhamento progressivo

Título: Revisão cega

Descrição: **"Como revisor, quero ver** os trabalhos sem identificar os **autores para que minha** avaliação seja imparcial."^{*}

Critérios de aceite:

- Nomes dos autores e instituições são ocultados.
- Metadados (como agradecimentos) são removidos automaticamente.
- O sistema alerta se algum arquivo vaziar informações de identificação.

3. Exemplo para requisito não funcional

Título: Acesso rápido

Descrição: **"Como participante, quero acessar** a programação em menos de dois **segundos para** que eu não perca palestras importantes."^{*}

Critérios de aceite:

- A página carrega em $\leq 2s$ com 1.000 usuários simultâneos.
- Dados são cacheados após a primeira consulta.

Exemplo de história épica (quebra em subtarefas)

Título: Gestão de certificados

Descrição macro:

"Como organizador, quero gerar certificados **automaticamente para** que reduza o trabalho manual."^{*}

Histórias filhas:

- “Como organizador, quero **importar lista de participantes aprovados** para que eu não precise digitar manualmente.”
- “Como organizador, quero **personalizar o modelo do certificado com logo do evento** para que ele tenha identidade visual.”

!! Importante

A escolha do modelo de documento mais adequado depende do contexto do projeto, da complexidade do sistema e das preferências da equipe. Em muitos casos, uma combinação de diferentes modelos pode ser utilizada para obter uma visão completa e abrangente dos requisitos.

Ferramentas de apoio

Existem diversas ferramentas que podem auxiliar na documentação de requisitos, confira alguns a seguir.

- Ferramentas de modelagem de requisitos (por exemplo, Enterprise Architect, Visual Paradigm).
- Ferramentas de gerenciamento de requisitos (por exemplo, Jira, Confluence).
- Ferramentas de controle de versões (por exemplo, Git, SVN).



Saiba mais

Para conhecer mais sobre o Jira ou o Confluence, acesse os links oficiais dos aplicativos disponibilizados a seguir e conheça as versões gratuitas:

<https://www.atlassian.com/br/software/jira>

<https://www.atlassian.com/software/confluence>

Ao dominar essas normas, práticas e ferramentas, você estará preparado para criar uma documentação de requisitos de alta qualidade, que servirá como base sólida para o desenvolvimento de softwares e garantirá a satisfação dos clientes.

Agora que conhecemos os melhores modelos de documentos e sua estrutura padrão, vamos para uma demonstração prática de como isso pode ser feito!

Estrutura padrão e modelos contextualização

Objetivo da aula:

Apresentar a importância da documentação de requisitos no desenvolvimento de sistemas, com estrutura clara e exemplos práticos aplicados ao contexto agrícola.

Por que documentar requisitos?

- Evita mal-entendidos, retrabalho e falhas.
- Alinha expectativas entre cliente, desenvolvedores, gestores e suporte.
- Define o que o sistema faz e o que não faz.

Estrutura padrão recomendada

1. Identificação do projeto

- Nome e descrição geral.
- Ex: "Sistema de irrigação inteligente – Fazenda São Pedro".

2. Escopo do sistema

- O que está incluído (ex: alertas via WhatsApp).
- O que está excluído (ex: manutenção física dos sensores).

3. Stakeholders (envolvidos)

- Proprietário, operador, auditor, eletricista, equipe de TI.

4. Requisitos funcionais (RF)

- RF 01: Exibir umidade por setor.
- RF 02: Enviar alerta de solo seco via WhatsApp.

5. Requisitos não funcionais (RNF)

- RNF 01: Suporte offline por 24h.
- RNF 02: Interface acessível com um toque.

6. Regras de negócio

- Irrigação só entre 5h e 20h.
- Relatórios assinados toda sexta-feira.
- Alertas após 3 medições abaixo do limite.

7. Modelos e casos de uso

- Ex: “Ativar irrigação” com ator, pré-condição e fluxo principal.
- Diagramas, fluxogramas, protótipos no Figma.

8. Glossário

- Pivô: equipamento de irrigação circular.
- Talhão: divisão da plantação.
- Laudo: relatório técnico exigido por auditoria.

9. Controle de versões

- RF 04 v1.0: alerta por SMS.
- RF 04 v1.1: alerta por WhatsApp (ajuste por preferência do cliente).

Ferramentas para documentar

- Documentos formais (DOC, PDF).
- Wikis online (Notion, Confluence).
- Cards em ferramentas ágeis (Trello, Jira).
- Planilhas organizadas por categoria.

Boas práticas

- Frases simples e objetivas.
- Verbos no infinitivo: exibir, registrar, enviar.
- Evitar ambiguidade: nada de “rápido” ou “bonito”.
- Numerar requisitos para facilitar rastreamento.
- Revisar com o usuário para garantir entendimento.

Conclusão

Documentar requisitos é criar um contrato funcional entre cliente e equipe.

Garante clareza, segurança e alinhamento em todas as etapas do projeto.

É o elo entre **necessidade e solução**.

Controle de versões



No mundo dinâmico do desenvolvimento de softwares, a mudança é a única constante. Requisitos evoluem, funcionalidades são aprimoradas e erros são corrigidos. Nesse cenário, o controle de versões, ou histórico de alterações, emerge como uma ferramenta indispensável, permitindo rastrear, gerenciar e documentar as alterações em um projeto ao longo do tempo.

A importância da documentação das alterações

A documentação das alterações é fundamental para manter a integridade do projeto, facilitar a colaboração entre a equipe e garantir a rastreabilidade das decisões tomadas. Ao registrar cada modificação, é possível:



Reverter para versões anteriores

Em caso de erros ou problemas, é possível retornar a uma versão estável do software.



Identificar e corrigir erros

Ao analisar o histórico de alterações, é mais fácil identificar a origem dos erros e corrigi-los de maneira eficiente.



Facilitar a colaboração

A documentação das alterações permite que os membros da equipe compreendam as modificações realizadas por outros, evitando conflitos e retrabalho.



Garantir a rastreabilidade

Ao documentar as decisões tomadas, é possível justificar as escolhas de design e implementação, facilitando a auditoria e a manutenção do software.



Acompanhar a evolução do projeto

A documentação das alterações permite acompanhar a evolução do projeto ao longo do tempo, identificando tendências e padrões.

Exemplos e aplicações práticas:

- **Levantamento de requisitos**

Ao documentar as alterações nos requisitos, é possível acompanhar a evolução das necessidades dos stakeholders e garantir que o software atenda às suas expectativas.

Por exemplo, ao adicionar um novo requisito, é importante registrar a data, o autor e a justificativa da alteração.

- **Documentação do software**

Ao documentar as alterações no código-fonte, é possível acompanhar a evolução da implementação e garantir que o software seja fácil de manter e evoluir. Por exemplo, ao corrigir um erro, é importante registrar a data, o autor, a descrição do erro e a solução implementada.

- **Gerenciamento de projetos**

Ao documentar as alterações no cronograma e no orçamento do projeto, é possível acompanhar o progresso e identificar desvios.

Por exemplo, ao adiar uma entrega, é importante registrar a data, o motivo e o impacto no cronograma.

Ferramentas de controle de versões



As ferramentas de controle de versão são essenciais para o gerenciamento eficaz de projetos de software, fornecendo rastreabilidade de alterações, colaboração facilitada e a capacidade de reverter para versões anteriores, otimizando o fluxo de trabalho e minimizando os riscos.

A seleção do sistema de controle de versão deve considerar:

- **Tamanho e distribuição da equipe**
O Git é ideal para times distribuídos, enquanto o SVN pode ser mais simples para equipes pequenas e locais.
- **Complexidade do projeto**
Repositórios com muitos binários podem se beneficiar do Git LFS ou SVN.
- **Fluxo de trabalhos**
O Git permite branching e merging flexíveis, útil em metodologias ágeis.
- **Integração com outras ferramentas**
Os GitHub/GitLab facilitam o CI/CD, enquanto o SVN se integra bem com ferramentas enterprise.

Alguns exemplos a seguir:



Git

Sistema distribuído, dominante no mercado, com plataformas como GitHub e GitLab.



SVN

Centralizado, mais simples para controle de arquivos grandes.



Mercurial

Alternativa distribuída similar ao Git, com foco em simplicidade.



Dica

A plataforma SENAI Play traz o curso “GitHub para programação Web”, no qual são abordados versionamento com Git e GitHub e como podem ser aplicados ao controle de versões e de documentos. Acesse o link e aprenda mais.

<https://play.senai.br/curso/github-para-programacao-web>

Ao adotar práticas de controle de versões e documentar as alterações de maneira consistente, as equipes de desenvolvimento podem garantir a qualidade, a rastreabilidade e a evolução eficiente de seus projetos de software.

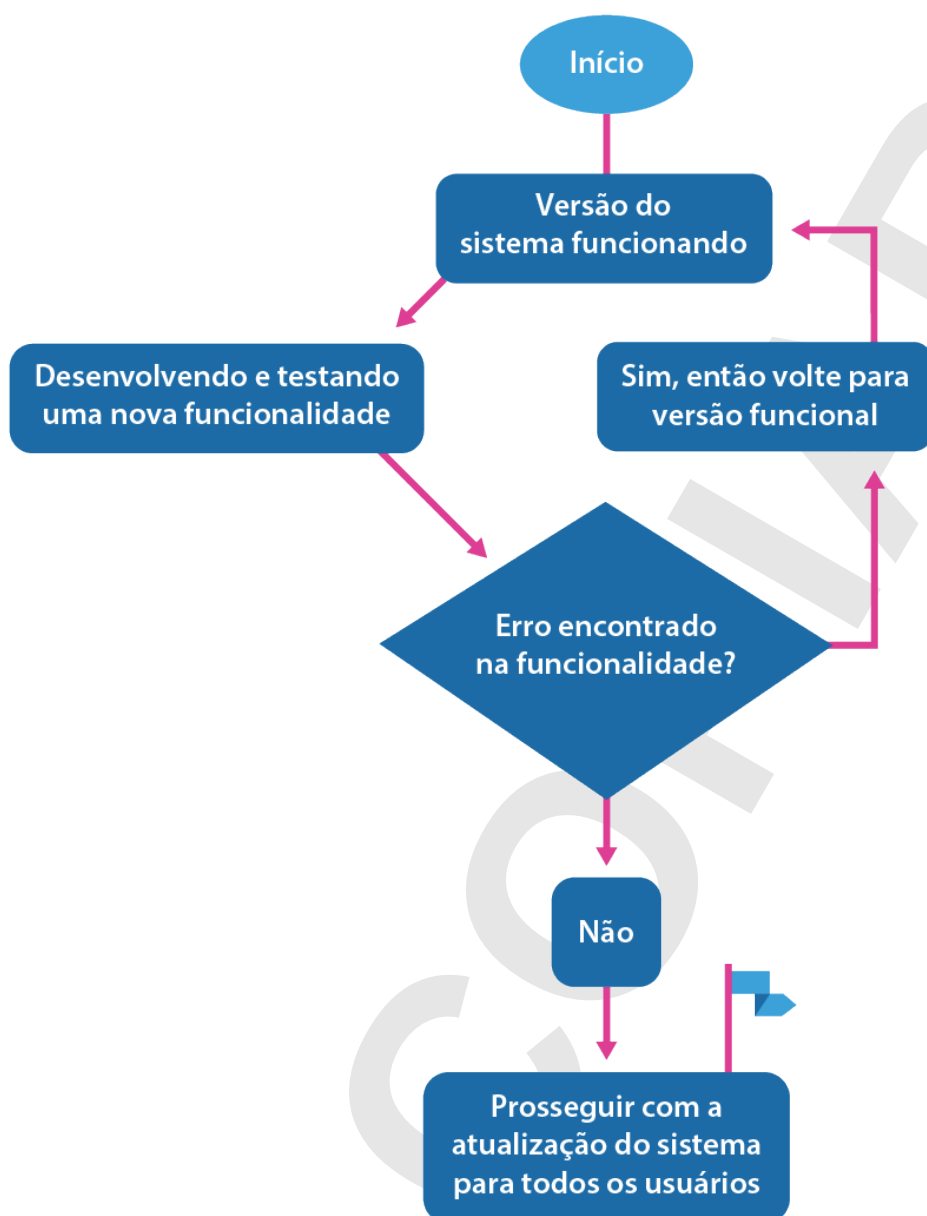


Além do Git e do SVN, que representam os pilares do controle de versões, é importante mencionar que o ecossistema de ferramentas para essa finalidade é vasto e diversificado. Plataformas como Mercurial, por exemplo, oferecem alternativas robustas com os próprios conjuntos de funcionalidades, atendendo a necessidades específicas de projetos.

Além disso, a integração do controle de versões com outras ferramentas de desenvolvimento tem se tornado uma prática cada vez mais comum. Essa integração permite automatizar o processo de construção, o teste e a implantação de softwares, garantindo maior agilidade e confiabilidade nas entregas. Ao explorar essas ferramentas e integrações, as equipes de desenvolvimento podem otimizar seus fluxos de trabalho e garantir a qualidade do software em todas as etapas do ciclo de vida.

Controle de versões do software

Para mostrar a importância do controle de versões de um software, segue um fluxograma com um caso que pode acontecer com softwares já consolidados no mercado.



Recapitulando

Ao longo desta aula, exploramos a importância das normas e dos modelos de documentos no processo de levantamento de requisitos. Compreendemos como a aplicação de normas como IEEE 29148 e ISO/IEC/IEEE 15288 garante a clareza, a consistência e a rastreabilidade dos requisitos, transformando informações abstratas em um guia concreto para o desenvolvimento de softwares. Aprendemos a elaborar documentos essenciais, como o DRS e o PGR, que servem de pilares para a comunicação eficaz entre a equipe de desenvolvimento e os stakeholders, além de dominar o controle de versões, fundamental para manter a integridade do projeto durante suas evoluções.

Sabemos que o mundo do desenvolvimento de softwares está em constante evolução, e novas abordagens surgem para otimizar o processo de criação de softwares. Na próxima aula, mergulharemos no universo das metodologias ágeis, que priorizam a flexibilidade, a colaboração e a entrega contínua de valor. Conheceremos o Scrum, um framework iterativo e incremental que promove a transparência e a adaptação, e o Kanban, um sistema visual que otimiza o fluxo de trabalho e a entrega de valor.

Prepare-se para expandir seus horizontes e descobrir como as metodologias ágeis podem transformar a forma como você aborda o desenvolvimento de softwares, permitindo que entregue produtos de alta qualidade de maneira mais rápida e eficiente a seus clientes.



5

Metodologias ágeis e gestão de projetos

Apresentação

Nesta aula, **exploraremos** como as metodologias **Scrum** e **Kanban** **transformarão** o gerenciamento de projetos e o desenvolvimento de softwares.

Conheceremos os papéis centrais do **Scrum** – Product Owner, Scrum Master e Equipe de Desenvolvimento – e **veremos** como esse framework **possibilitará** entregas incrementais, adaptação constante e colaboração entre todos os envolvidos.

Em seguida, **abordaremos** o **Kanban**, que **fornecerá** uma forma clara de visualizar tarefas.

Aprenderemos, também, a definir prioridades, monitorar progresso e aproveitar gráficos específicos para embasar decisões. Isso **garantirá** que a equipe **mantenha** o foco nas necessidades do projeto, **entregando** valor contínuo ao cliente.

Metodologia SCRUM



Imagine uma equipe de alpinistas, trabalhando em sincronia para escalar uma montanha desafiadora. Eles dividem a jornada em etapas menores, cada uma com um objetivo claro e mensurável, reunindo-se regularmente para avaliar o progresso, ajustar a rota e garantir que todos estejam alinhados. Essa organização, essa colaboração focada em resultados, é o que o Scrum oferece.

Definição



O Scrum é um dos frameworks mais populares no universo das metodologias ágeis. Criado por Ken Schwaber e Jeff Sutherland, foi desenvolvido para ajudar a criar e manter produtos complexos de maneira eficiente. Mais do que um simples processo, o Scrum é uma abordagem estruturada que organiza o trabalho em equipe por meio de papéis, eventos e artefatos. Por meio dessa abordagem, o Scrum garante entregas contínuas e adaptáveis às necessidades do projeto.

Papéis e responsabilidades

No Scrum, os papéis são fundamentais para garantir que o trabalho seja organizado, eficiente e bem direcionado, com foco na entrega contínua de valor. Cada membro da equipe tem responsabilidades específicas, mas todos trabalham de maneira colaborativa para atingir os objetivos do projeto. Vamos explorar em detalhes as responsabilidades de cada um desses papéis.

- **Product owner**

É o responsável por garantir que o produto atenda às necessidades do cliente e aos objetivos do negócio. Ele é a voz do cliente dentro da equipe e tem a responsabilidade de gerenciar o product backlog, que contém as funcionalidades e os requisitos do produto.

Responsabilidades:

- Gerenciar o product backlog.
- Definir as prioridades.
- Garantir a clareza dos requisitos.
- Representar o cliente.

- **Scrum master**

É o facilitador do processo, garante que a equipe siga os princípios e as práticas do Scrum, removendo impedimentos e garantindo que a equipe tenha as condições necessárias para atingir seus objetivos. É um líder servil que ajuda todos os membros a entenderem e a implementarem o Scrum corretamente.

Responsabilidades:

- Facilitar o processo Scrum.
- Remover impedimentos.
- Proteger a equipe.
- Promover a melhoria contínua.
- Mentoria e coaching.

- **Time de desenvolvimento**

Composto pelos profissionais que têm a responsabilidade de criar o produto conforme os requisitos e as prioridades definidos pelo product owner. O time é autônomo e multifuncional, cada membro traz habilidades diversas para que, juntos, possam completar o trabalho necessário.

Responsabilidades:

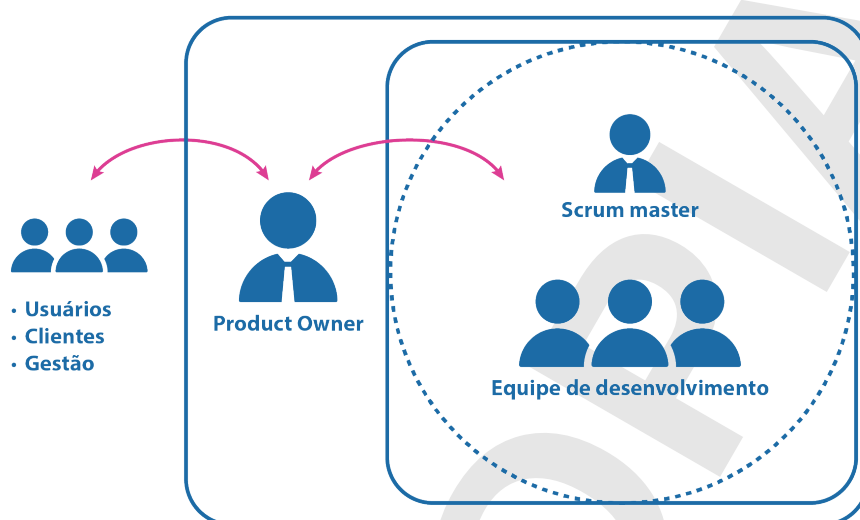
- Desenvolver o incremento.
- Organizar o trabalho.
- Garantir a qualidade do produto.
- Trabalhar em colaboração.

A seguir, você conseguirá visualizar uma prévia de como é o dia a dia de uma equipe que utiliza a metodologia Scrum.

Cada papel contribui para a eficiência e a qualidade da entrega de um projeto. Agora veremos como desenvolver essas responsabilidades aplicadas à gestão de projetos.

Papéis no Scrum

Já tratamos um pouco dos papéis do Scrum. Observe a imagem que mostra como eles se relacionam.



Aplicação da gestão de projetos



No contexto de gestão de projetos, o Scrum permite que as equipes sejam mais ágeis e mais adaptáveis às mudanças. Ele oferece uma estrutura clara, mas com flexibilidade, para que as equipes escolham as melhores práticas para cada situação. A organização do trabalho em sprints (também conhecido como ciclo de desenvolvimento), que é um período fixo e limitado, geralmente acontece entre uma e quatro semanas, durante a qual uma equipe trabalha para concluir uma parte do trabalho de um projeto.

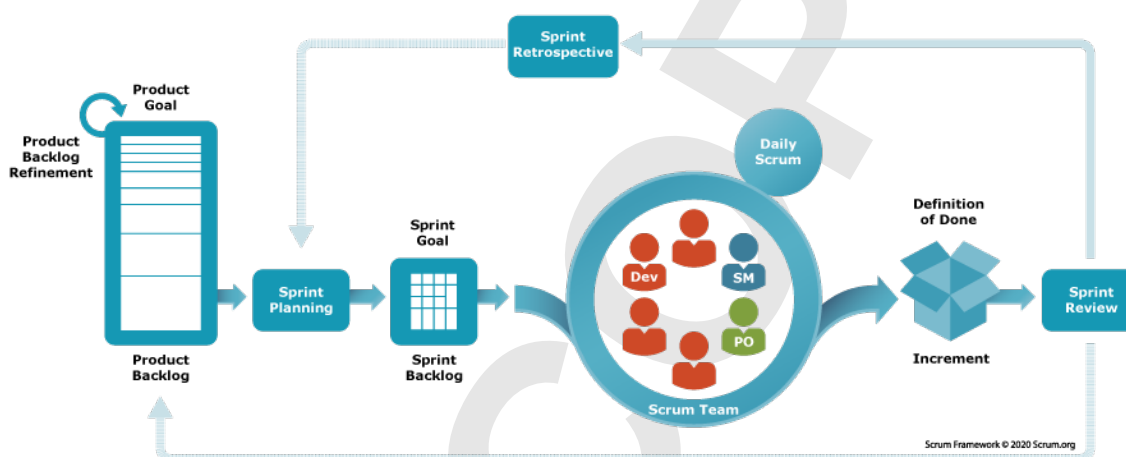


Isso possibilita que a equipe foque em entregas pequenas e incrementais, o que garante um feedback constante, permitindo ajustes rápidos ao longo do processo. Essa abordagem é especialmente valiosa em ambientes de projetos complexos e em constante mudança, como o desenvolvimento de softwares, em que os requisitos podem evoluir rapidamente.

Em resumo, o Scrum, quando aplicado à gestão de projetos, oferece uma abordagem estruturada e ágil que prioriza a colaboração, a adaptação e a entrega contínua de valor. Com papéis claramente definidos e ciclos iterativos e incrementais, o Scrum ajuda as equipes a se manterem focadas nas necessidades do cliente e a se ajustarem rapidamente às mudanças, promovendo um ambiente de trabalho eficiente e altamente produtivo.

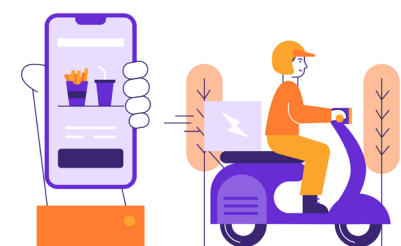
Scrum aplicado à gestão de projetos

Já abordamos como o Scrum pode ser aplicado em projetos, então confira a imagem que mostra um mapa de todas as rotinas recomendadas para um fluxo Scrum ideal. Vamos exemplificar o fluxo com nosso projeto de implantação de um e-commerce.



O product owner é responsável por levantar os requisitos do cliente usando Design Thinking, priorizando as funcionalidades no backlog do produto (uma lista dinâmica e priorizada de tudo o que precisa ser desenvolvido para um produto, incluindo funcionalidades, correções, melhorias e requisitos) e garantindo que as necessidades dos lojistas sejam atendidas. O Scrum master facilitará o processo, garantindo que a equipe siga as práticas do Scrum, removendo obstáculos e promovendo as reuniões diárias de acompanhamento, como o Daily Scrum, para monitorar o progresso e ajustar o que for necessário. A equipe de desenvolvimento, por sua vez, trabalhará nas sprints, dividindo as funcionalidades em tarefas menores e entregando incrementos do produto a cada duas semanas.

Metodologia Kanban



Imagine o desenvolvimento de um aplicativo de entrega de comida, no qual cada funcionalidade, como o cadastro de restaurantes, o gerenciamento de pedidos e o rastreamento de entregas, segue um fluxo contínuo de construção. Cada etapa, desde a análise de requisitos até a implementação e testes, é claramente visualizada, permitindo que a equipe de desenvolvimento identifique gargalos e otimize o processo, a fim de garantir que o aplicativo seja eficiente e atenda rapidamente às necessidades dos usuários. Uma abordagem semelhante, focada em clareza e fluxo, é a metodologia ágil Kanban. Agora, apresentaremos em detalhes o que é essa metodologia e quais são os benefícios de utilizá-la.

Definição

Kanban é uma metodologia de gestão de fluxo de trabalho e de tarefas que se baseia em um sistema visual de organização, originalmente desenvolvido pela Toyota para otimizar sua produção e reduzir desperdícios. No contexto de gestão de projetos, essa metodologia é usada para visualizar, controlar e gerenciar o progresso de tarefas de maneira eficiente. Agora, vamos explorar o processo de criação do quadro-base da metodologia.

Neste resumo do vídeo, observaremos na prática como os processos de metodologias ágeis que estamos estudando podem ser aplicados em um projeto.

Aplicação de Metodologias Ágeis – Scrum e Kanban

Objetivo da aula:

Apresentar as metodologias ágeis Scrum e Kanban, com exemplos práticos e aplicação no contexto de automação agrícola.

Por que usar metodologias ágeis?

- Permitem adaptação rápida a mudanças.
- Evitam retrabalho e entregas tardias.
- Facilitam colaboração com o cliente e visão clara do progresso.

Scrum – Ciclos curtos e organizados

Características:

- Dividido em sprints (1 a 4 semanas).
- Cada sprint tem:
 - Planejamento inicial
 - Reuniões diárias
 - Revisão da entrega
 - Retrospectiva

Papéis na equipe:

- Product Owner (PO): define prioridades.
- Scrum Master: facilita o processo.
- Time de desenvolvimento: executa as tarefas.

Exemplo prático – Fazenda São Pedro:

- Sprint 1: protótipo do painel de irrigação.
- Sprint 2: integração de sensores e bot WhatsApp.
- Sprint 3: testes e ajustes com o cliente.

Kanban – Fluxo visual e contínuo

Características:

- Quadro com colunas:

- A fazer (To Do)
 - Fazendo (Doing)
 - Feito (Done)
- Cada tarefa é um cartão que avança conforme o progresso.
 - Ideal para equipes pequenas e tarefas recorrentes.

Exemplo prático – Kanban da fazenda:

- A fazer: botão de emergência, versão para celular antigo.
- Fazendo: integração com sensores LoRa.
- Feito: protótipo do painel com modo offline.

Criando um Kanban no Trello (passo a passo)

1. Acesse trello.com e crie uma conta.
2. Crie um novo quadro com nome do projeto.
3. Adicione colunas: **A fazer, Fazendo, Feito**.
4. Crie cartões com tarefas.
5. Personalize com:
 - Etiquetas de cor (ex: vermelho para urgência).
 - Prazos e comentários.
 - Anexos (ex: protótipos do Figma).
 - Mencionando colegas com @.

Comparação entre Scrum e Kanban

-

Aspecto	Scrum	Kanban
Organização	Sprints	Fluxo contínuo
Papéis definidos	Sim	Não
Métricas	Velocidade por sprint	Tempo de ciclo

Ideal para

Projetos maiores

Tarefas
recorrentes

Conclusão

Ser ágil é organizar com flexibilidade e propósito, não improvisar.

Scrum e Kanban ajudam a manter clareza, ritmo e melhoria contínua, com o cliente participando do processo.

Criação do quadro-base



A principal característica do Kanban é o uso de um quadro visual em que as tarefas são representadas por cartões que se movem entre diferentes colunas, geralmente refletindo etapas do processo de trabalho (como “A fazer”, “Em progresso”, e “Concluído”). Esse método permite que as equipes vejam rapidamente o que está acontecendo em cada etapa e identifiquem possíveis gargalos ou ineficiências.

- **Visualização do trabalho**

Permite que todos na equipe vejam as tarefas em andamento, as que estão concluídas e as que ainda precisam ser feitas.

- **Limitação de trabalho em andamento (WIP)**

Ajuda a evitar a sobrecarga da equipe, limitando o número de tarefas que podem estar em progresso simultaneamente. Work in progress (WIP) é o termo técnico utilizado para essa medição.

- **Foco na entrega contínua**

Promove uma cultura de entrega frequente e incremental, melhorando a resposta a mudanças nas necessidades dos clientes.

- **Melhoria contínua**

Facilita a identificação de ineficiências e a implementação de práticas que podem melhorar o processo.

Para garantir uma entrega com excelência, visando os pontos citados, existem critérios importantes que devem ser considerados no momento de criação do quadro Kanban.

- **Definir colunas**
Divida o fluxo de trabalho em etapas, como “A fazer”, “Em progresso” e “Concluído”. As colunas podem ser adaptadas com base nas necessidades de seu processo.
- **Identificar tarefas**
Liste todas as tarefas que precisam ser realizadas. Você pode usar cartões físicos ou uma ferramenta digital para representá-las.
- **Estabelecer limites (WIP)**
Defina quantas tarefas podem estar em cada coluna ao mesmo tempo para evitar sobrecarga.
- **Priorizar as tarefas**
Organize as tarefas de acordo com a prioridade e a urgência. Isso pode ser feito manualmente ou por meio de um sistema de pontuação.
- **Escolher uma ferramenta ou método**
Decida se você usará um quadro físico (por exemplo, um quadro branco com post-its), um software de Kanban online ou instalar em seu equipamento.
- **Revisão e ajustes**
Realize reuniões regulares (como as reuniões diárias de stand-up) para revisar o progresso, discutir obstáculos e fazer ajustes conforme necessário.
- **Monitorar e melhorar**
Utilize métricas e feedback para monitorar a eficiência do quadro e do fluxo de trabalho, promovendo melhorias contínuas.

Implementar um quadro-base Kanban eficaz promove maior transparência e colaboração dentro da equipe, facilitando a produtividade e a entrega dos resultados desejados. Continuaremos a aprender mais sobre a metodologia por meio de alguns pontos cruciais que permitem gerenciar o processo de maneira mais eficaz. A seguir, um exemplo de quadro Kanban dentro de um ambiente de trabalho.

Quadro Kanban

Percebemos que o Kanban é uma forma de representar visualmente as tarefas que a equipe do projeto está desenvolvendo. Configure agora um exemplo de como ele pode ser criado. Cada cartão deve ter uma tarefa específica. Se a tarefa for muito grande, deve ser dividida em mais delas.



Identificação do trabalho

A identificação do trabalho no Kanban é essencial para garantir um fluxo organizado e transparente.

- **Cartões Kanban**

Cada item de trabalho deve ser representado por um cartão no quadro Kanban. Esses cartões podem conter informações como:

- título da tarefa;
- descrição;
- responsável;
- prioridade;
- data de início e prazo;
- status (a fazer, em progresso, concluído etc.);
- tags ou categorias.

- **Cores e etiquetas**

Diferencie tarefas por cores ou etiquetas para facilitar a identificação.

- **Swimlanes (raia de separação)**

Divida o quadro em linhas horizontais para categorizar melhor o trabalho. Por exemplo:

- tipos de tarefas (correções, novas features, suporte);
- departamentos ou equipes;
- nível de prioridade.

- **Identificação por código ou ID**

Se for uma equipe grande ou um projeto complexo, é útil adicionar um código único a cada tarefa, como:

- "BUG-102" para erros;
- "DEV-305" para desenvolvimento;
- "DOC-201" para documentação.

- **Avatares ou ícones**

Se a equipe for grande, pode-se usar avatares ou fotos dos responsáveis diretamente nos cartões, para indicar a quem pertence a tarefa.

- **Checklist e subtarefas**

Para tarefas mais complexas, é útil adicionar checklists dentro dos cartões para acompanhar cada etapa.

Exemplos voltados ao nosso projeto:

Vermelho - Tarefas críticas

Exemplo: Corrigir erro crítico de segurança no sistema de login.

Verde - Melhorias ou novas funcionalidades

Exemplo: Implementar nova funcionalidade de geração de relatórios

Azul - Tarefas em andamento

Exemplo: Desenvolver integração com sistema de agenda e palestrantes.

Amarelo - Tarefas em espera

Exemplo: Aguardar aprovação de design para nova tela do usuário

Neste resumo de PDF Mesacast, teremos uma conversa com profissionais que já participaram de projetos de impacto. Conversaremos sobre como a aplicação de metodologias ágeis ajuda no desenvolvimento e na conclusão do projeto e, também, o que a falta dela pode prejudicar.

▶ vídeo resumido

Como a gestão de requisitos pode impactar em um projeto?

Objetivo da aula:

Mostrar como a etapa de **briefing** e a **gestão de requisitos** influenciam diretamente na qualidade, no tempo e no sucesso de um projeto de desenvolvimento de sistemas.

O que é briefing e por que é importante?

- É o **primeiro planejamento estratégico** do projeto.
- Serve para **entender o cliente, o problema e alinhar expectativas**.
- Sem briefing, há risco de criar soluções complexas que não resolvem o problema real.

Como abordar o cliente?

- Pode ser por **entrevistas, reuniões ou workshops**.
- É essencial **envolver quem realmente usa o sistema** (não só gestores).
- Exemplo: desenvolvedor que testou o app de entregas na rua para entender os motoboys.

Importância de entender o público-alvo

- Saber **quem vai usar o sistema** define o nível de complexidade.
- Exemplo: sistema para operários deve ser simples; para jovens, pode ser mais sofisticado.
- Caso real: sistema feito para presidente da empresa não foi usado por não atender à sua rotina de 10 minutos diários disponíveis.

Impacto dos requisitos no tempo de desenvolvimento

- Requisitos mal definidos geram retrabalho e atrasos.
- Desenvolvedor atua como tradutor entre cliente e tecnologia.

- É preciso alinhar prazos reais e considerar mudanças durante o projeto.

Gestão de requisitos

- Não basta anotar: é preciso gerenciar mudanças e novas demandas.
- Requisitos funcionais: o que o sistema faz (ex: cadastrar usuários).
- Requisitos não funcionais: como o sistema faz (ex: carregar tela em até 1 segundo).

Restrições no projeto

- Perguntar ao cliente o que não pode ter no sistema.
- Exemplo: cores, fontes, distância entre logos, regras de segurança e acessibilidade.
- Cada empresa tem políticas internas que precisam ser respeitadas.

Passo a passo para um bom briefing

1. **Ouvir** o cliente com atenção.
2. **Investigar** o problema real.
3. Só depois pensar na solução.
4. Anotar requisitos funcionais, não funcionais e restrições.

Conclusão

Um bom desenvolvedor não é só quem escreve código, mas quem **entende as dores do cliente** e propõe soluções eficazes.

A gestão de requisitos e o briefing bem feitos garantem **qualidade, agilidade e satisfação** no projeto.

Acabamos de visualizar como realizar a identificação do trabalho. A seguir, conheceremos formas de definir as prioridades.

Prioridades do trabalho

Definir a prioridade dos trabalhos no Kanban é essencial para garantir um fluxo eficiente e alinhado com os objetivos da equipe. A seguir, três maneiras de priorizar tarefas:

Utilizar critérios de priorização

Definir critérios claros ajuda a evitar a subjetividade. Algumas abordagens comuns incluem:

Cor vermelha = Urgência vs. Importância



A matriz de Eisenhower pode ajudar a classificar tarefas conforme sua urgência e importância, facilitando a tomada de decisões; é uma ferramenta de gestão de tempo e priorização de tarefas, criada com base na abordagem do ex-presidente dos EUA, Dwight D. Eisenhower.

Cor azul = Valor para o cliente



Itens com maior impacto para os usuários ou clientes devem ter prioridade.

Cor amarela = Riscos e bloqueios



Tarefas que impedem outras atividades devem ser resolvidas primeiro.

Cor preta = Complexidade e esforço



Tarefas pequenas e rápidas podem ser priorizadas para manter o fluxo.

Usar cores e etiquetas para diferenciação: atribuir cores ou tags no Kanban facilita a visualização da prioridade. Exemplo:

Cor vermelha



Cor vermelha - Alta prioridade (bloqueia outros itens, impacto crítico)

Cor amarela



Média prioridade (importante, mas não urgente)

Cor verde



Cor verde - Baixa prioridade (melhoria contínua, sem impacto imediato)

Classificação por níveis de prioridade: outro método eficiente é definir níveis específicos, como:

- P0 – Urgente (necessário resolver imediatamente)
- P1 – Alta prioridade (essencial, mas pode esperar um pouco)
- P2 – Média prioridade (desejável, mas sem urgência)
- P3 – Baixa prioridade (melhoria futura, sem impacto imediato)

Todos os processos citados que são utilizados dentro do Kanban precisam ser acompanhados de alguma forma. Para que isso ocorra, abordaremos o gerenciamento do progresso e do desempenho das tarefas.

Gerenciamento do progresso e do desempenho

O Kanban permite acompanhar o progresso e medir o desempenho do fluxo de trabalho de maneira visual e quantitativa. Para isso, utilizamos métricas, gráficos e boas práticas que ajudam na tomada de decisão e na otimização do processo, as quais são:



Monitoramento visual do fluxo: o próprio quadro-base do Kanban já fornece uma visão clara do progresso. Confira algumas formas de gerenciar:

- **Observar o movimento dos cartões >>** se os cartões estão acumulados em uma etapa, pode haver um gargalo.
- **Usar cores e etiquetas >>** diferenciar tarefas por prioridade, tipo de atividade ou status especial.
- **Reuniões diárias rápidas (stand-up) >>** ajuda a identificar bloqueios e alinhar a equipe.
- Utilizar métricas para avaliação do desempenho.

Exemplo prático: se muitos cartões estão acumulados na coluna “Em andamento” do quadro Kanban do aplicativo de gestão, isso pode indicar um gargalo no processo de desenvolvimento, promovendo atrasos para atender aos prazos estipulados.

Além da visualização do quadro, podemos medir o desempenho com métricas específicas do Kanban:



Lead Time (Tempo de Entrega)

- Tempo total desde a criação da tarefa até a conclusão.
- Quanto menor for o Lead Time, mais eficiente é o fluxo.



Cycle Time (Tempo de Execução)

- Tempo que a tarefa leva para ser finalizada após ser iniciada.
- Ajuda a entender a eficiência da equipe.



Throughput (Produção)

- Número de tarefas concluídas em determinado período (por dia, semana ou mês).
- Ajuda a prever entregas futuras e a medir a capacidade da equipe.



Work in Progress (WIP – Trabalho em Progresso)

- Número de tarefas em andamento ao mesmo tempo.
- Limitar o WIP melhora o fluxo de trabalho e reduz gargalos.

Exemplo de cálculo de métricas para o sistema de gestão de eventos acadêmicos

Vamos aplicar as métricas de desempenho do Kanban ao módulo “Revisão de Trabalhos Acadêmicos” do sistema de gestão de eventos acadêmicos, com dados simulados para uma sprint de duas semanas.

- **Dados do período**
 - Tarefas criadas: 30 trabalhos submetidos.
 - Tarefas concluídas: 24 revisões finalizadas.
 - Tarefas em andamento (WIP): 6 trabalhos (média diária).
 - Lead Time médio: 5 dias (desde a submissão até a aprovação).
 - Cycle Time médio: 2 dias (tempo efetivo em revisão).

- **Cálculo das métricas**

Métrica	Fórmula	Cálculo	Interpretação
Throughput	Tarefas concluídas / Período	24 trabalhos / 10 dias úteis = 2,4/dia	A equipe revisa ~12 trabalhos/seman
Eficiência	$(\text{Cycle Time} / \text{Lead Time}) \times 100$	$(2/5) \times 100 = 40\%$	60% do tempo é espera (por exemplo, fila de revisão)
Capacidade WIP	Throughput $\times$ Cycle Time	$2,4 \times 2 = 4,8$	Ideal: Limitar WIP para ≤ 5 trabalhos simultâneos

- **Análise dos resultados**

Problema identificado:

- Baixa eficiência (40%) >> Grande parte do Lead Time é espera (filas).
- WIP atual (6) acima da capacidade ideal (4.8) >> Gargalo na revisão.

Ações propostas:

- Limitar WIP para 5 trabalhos em revisão simultânea.
- Automatizar triagem inicial (ex.: verificação automática de plágio).
- Adicionar 1 revisor temporário para picos de demanda.

- **Previsão para próxima sprint**

Com as melhorias, estima-se:

- **Cycle Time:** redução para 1,5 dias (ganho de 25%).
- **Throughput:** aumento para 3 revisões/dia (+25%).
- **Lead Time:** queda para 3,5 dias (30% mais rápido).



Saiba mais

Desafio: Calcule as métricas para o módulo de "Inscrições" do sistema de gestão de eventos acadêmicos com os dados a seguir:

- Período: 1 semana (5 dias úteis).
- Inscrições criadas: 120.

- Inscrições concluídas: 100.
- Lead Time médio: 1,2 dias.

Cycle Time médio: 0,5 dia.

Perguntas:

- Qual é o throughput diário?
- Qual é a eficiência do processo?
- Qual é o WIP ideal?



Os gráficos no Kanban transformam dados brutos em insights acionáveis, ajudando a identificar gargalos, prever prazos e otimizar a produtividade. Confira como aplicá-los em projetos:

Cumulative Flow Diagram (CFD) – Diagrama de fluxo cumulativo

- **O que mostrar**

A distribuição de tarefas em cada estágio do fluxo (ex.: “A fazer”, “Em andamento”, “Concluído”) ao longo do tempo.

Gargalos: se uma banda (por exemplo, “Testes”) alarga, indica acúmulo.

- **Campos do gráfico**

Eixo X	Eixo Y	Bubble Size (opcional)
Tempo (dias/ semanas)	Quantidade de tarefas	Cada cor = uma coluna do Kanban (por exemplo, azul = “Em revisão”).

Aplicação prática no sistema de gestão de eventos acadêmicos

- Problema: muitos trabalhos presos em “Aguardando Revisão”.
- Solução: contratar mais revisores ou automatizar triagem inicial.

Gráfico de dispersão (Scatter Plot)

- **O que mostra**

Variação do Cycle Time (tempo de execução) por tarefa.

Padrões: se os pontos estão agrupados (consistência) ou dispersos (imprevisibilidade).

- **Campos do gráfico**

Eixo X	Eixo Y	Bubble Size (opcional)
Throughput	Tarefas concluídas / Período	24 trabalhos / 10 dias úteis = 2,4/dia

- **Variáveis úteis**

Linha de tendência: mostra se o Cycle Time está melhorando ou piorando.

Filtros: por tipo de tarefa (por exemplo, "bug" vs. "feature").

Exemplo: se tarefas de "Integração com pagamentos" têm Cycle Time alto, investigar dependências externas.

Gráfico de throughput

- **O que mostra**

Número de tarefas concluídas por período (dia/semana/mês).

Capacidade da equipe: média histórica ajuda a prever entregas futuras.

- **Campos do gráfico**

Eixo X	Eixo Y	Bubble Size (opcional)
Período (semanas)	Tarefas concluídas	Linha da média móvel (por exemplo, últimos 4 períodos).

- **Aplicação**

Meta: aumentar throughput de 15 para 20 tarefas/semana.

Ação: limitar WIP ou reduzir reuniões operacionais.

Gráfico	Indicação	Variáveis-chave	Ferramentas recomendadas
CFD	Identificar gargalos contínuos	Tempo × Quantidade de tarefas por fase	Kanbanize, Jira, Excel
Scatter Plot	Analisar variação de prazos	Cycle Time × Data × Complexidade	Power BI, Tableau
Throughput	Medir produtividade da equipe	Tarefas concluídas × Período	Trello com plug-ins, GitHub Actions

Exemplo de aplicação no sistema de gestão de eventos acadêmicos:



CFD: revela que 60% dos trabalhos ficam ≥ 3 dias em “Revisão”. Scatter Plot: mostra que revisões de artigos longos (>10 páginas) têm Cycle Time 2× maior. Ação: dividir artigos longos em seções para revisão paralela.



Saiba mais

Desafio: Para o módulo “Inscrições” do sistema de gestão de eventos acadêmicos: Desenhe um CFD hipotético com as colunas “Nova inscrição”, “Pagamento pendente”, “Confirmada”.

Identifique uma variável para um Scatter Plot (por exemplo, tempo de processamento × tipo de pagamento).

Melhoria contínua com o Kanban

Além de monitorar, é importante agir para otimizar o fluxo:

- Revisar periodicamente as métricas para ajustar estratégias.
- Fazer retrospectivas para entender problemas e melhorar o fluxo de trabalho.
- Ajustar os limites de WIP para evitar a sobrecarga da equipe.

O Kanban é uma metodologia eficaz para otimizar o fluxo de trabalho, com o propósito de aumentar a transparência, melhorar a colaboração e maximizar a produtividade. Seu quadro visual facilita a visualização clara das tarefas, permitindo identificar gargalos e ineficiências de maneira rápida. Ao limitar o trabalho em progresso e organizar as tarefas de modo estratégico, o Kanban assegura que a equipe se concentre nas prioridades. Esse foco promove uma entrega contínua e uma melhoria constante, resultando em um gerenciamento de projetos mais ágil, flexível e eficaz, alinhado às necessidades da equipe e dos clientes.



Mapeamento do fluxo de trabalho

Quando utilizamos diferentes cores para os cartões, podemos categorizá-los. Exemplo: cartões roxos são para a equipe de design, cartões azuis para a equipe de back-end e assim por diante.

Uma forma de mensurar bem o trabalho é adicionar no cartão o responsável por aquela tarefa. Dessa forma, quantas tarefas estão ativas por pessoa, sendo possível remanejar as tarefas caso alguém fique sobrecarregado.

Backlog	Development 3		Testing 1	Done
	OnGoing	Done		
F G	E			A
K H	D	C		B
L I				
M J				

Encerramento

Nesta aula, exploramos três metodologias ágeis – Scrum, Kanban e Design Thinking – que transformam a gestão de projetos e o levantamento de requisitos. O Scrum facilita entregas incrementais e adaptáveis, sendo ideal para projetos que exigem flexibilidade e uma resposta ágil às mudanças. O Kanban aprimora o fluxo de trabalho, aumentando a visibilidade e a eficiência, essenciais para gerenciar requisitos em tempo real. Já o Design Thinking, com sua abordagem centrada no usuário, assegura que a solução final atenda às necessidades reais dos usuários, desde a fase de empatia até a implementação.

Ao integrar essas metodologias com modelos de documentação de requisitos, como SRS, documento de visão e escopo, casos de uso e histórias de usuário, os profissionais podem criar uma base sólida para o desenvolvimento de softwares. A gestão eficaz das regras de negócios e restrições, com a definição clara dos requisitos (funcionais e não funcionais), garante que o projeto siga um caminho bem estruturado e alinhado com os objetivos da organização. Ao adotar práticas como briefing, levantamento orientado a pontos de vista, etnografia e entrevistas e brainstorming, as equipes podem aprimorar a comunicação e a colaboração durante todas as etapas do ciclo de vida do software.

Chegamos ao final da Unidade Curricular de Levantamento de Requisitos! Nela, estudamos o que são requisitos, seus tipos e as técnicas para elaborá-los. Aprendemos, também, como podemos documentá-los e trabalhar de maneira colaborativa e dinâmica, prezando por um ambiente de trabalho produtivo e dinâmico, não engessado. Em outro momento, aprenderemos sobre sistemas operacionais, aprofundaremos o estudo sobre os aspectos técnicos de computadores para, depois, construirmos aplicações da internet das coisas utilizando os requisitos que aprendemos nesta Unidade Curricular. Até lá e bons estudos!



REFERÊNCIAS

KERR, E. S. **Gerenciamento de requisitos**. São Paulo. Editora Pearson Education do Brasil. 2015

KIEFER, E. **Levantamento de requisitos: técnicas e Ferramentas para levantamento de Requisitos em desenvolvimento de sistemas** Monografia (Pós-graduação em Gestão de Projetos). – Escola de Engenharia da Universidade Presbiteriana Mackenzie, São Paulo, 2013. Disponível em: <https://adelpha-api.mackenzie.br/server/api/core/bitstreams/d734b431-d2c8-43c4-944c-a87d165069d0/content>. Acesso em 17 jul. 2025.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**. Belo Horizonte: AMGH Editora. 2021.

VAZQUES, C. E.; SIMÕES, G. S. **Engenharia de requisitos: software orientado ao negócio**. 1. Edição. Rio de Janeiro: Brasport, 2016.

BRASIL. Ministério da Educação. **Metodologia de gestão de desenvolvimento ágil de software**. V. 1. Brasília, DF: Ministério da Educação, 2022. Disponível em: https://www.gov.br/mec/pt-br/aceso-a-informacao/institucional/estrutura-organizacional/orgaos-especificos-singulares/secretaria-executiva/stic/documentos/mdsagil_mec_v1-1_dez22.pdf. Acesso em: 16 jul. 2025.

CUNHA, J. A.; FERRARI, L. A. **Metodologias ágeis: uma abordagem prática para o desenvolvimento de software**. São Paulo: Novatec, 2014.

DEALY, M.; LEE, D. **Kanban: mudança evolutiva para o gerenciamento de projetos**. São Paulo: Casa do Código, 2013.

LEWIS, D. **Scrum e metodologias ágeis: aplicação no gerenciamento de projetos de software**. 3. ed. São Paulo: Editora Técnica, 2019.

LIMA, A. Metodologias ágeis: o que são e como aplicar? **Íris| Laboratório de Inovação e Dados**, Ceará, 2022. Disponível em: <https://irislab.ce.gov.br/wp-content/uploads/2022/12/Guia-de-Metologias-Ageis.pdf>. Acesso em: 16 jul. 2025.

METODOLOGIAS Ágeis. Estante de Livros – Senai. Disponível em: <https://estante-delivros.senai.br/view/1E6P7wLghdmAnG45G-VXTgA7XFyC0hMoY>. Acesso em: 16 jul. 2025.

SILVA, J. C.; PEREIRA, A. M. **Metodologias ágeis**: evolução e tendências. Revista Brasileira de Gestão de Projetos, v. 12, 2021.

SUTHERLAND, J. **Scrum**: a arte de fazer o dobro de trabalho na metade do tempo. Rio de Janeiro: Alta Books, 2017.

CUNHA, J. A.; FERRARI, L. A. **Metodologias ágeis**: uma abordagem prática para o desenvolvimento de software. São Paulo: Novatec, 2014.

DEALY, M.; LEE, D. **Kanban**: mudança evolutiva para o Gerenciamento de Projetos. São Paulo: Casa do Código, 2013.

LEWIS, D. **Scrum e metodologias ágeis**: aplicação no gerenciamento de projetos de software. 3. ed. São Paulo: Editora Técnica, 2019.

LIMA, A. **Metodologias ágeis**: o que são e como aplicar?. ÍRIS, Laboratório de Inovação e Dados. Ceará, 2022. Disponível em: <https://irislab.ce.gov.br/wp-content/uploads/2022/12/Guia-de-Metologias-Ageis.pdf>. Acesso em: 17 jul. 2025.

METODOLOGIAS ágeis. São Paulo: Senai. Disponível em: <https://estantedelivros.senai.br/view/1E6P7wLghdmAnG45G-VXTgA7XFyC0hMoY>. Acesso em: 5 mar. 2025.

SCHWABER, K.; SUTHERLAND, J. Scrum: a arte de fazer o dobro de trabalho na metade do tempo. Rio de Janeiro: Alta Books, 2017.

SILVA, J. C. da; PEREIRA, A. M. Metodologias ágeis: evolução e tendências. **Revista Brasileira de Gestão de Projetos**, v. 12, 2021.

SUTHERLAND, J. **Scrum**: a arte de fazer o dobro do trabalho na metade do tempo. Rio de Janeiro: Sextante, 2019.

VEIGA, V.; CASTRO JÚNIOR, J. Metodologia de gestão de desenvolvimento ágil de software. Ministério da Educação, Brasília, v. 1, 2022. Disponível em: https://www.gov.br/mec/pt-br/aceso-a-informacao/institucional/estrutura-organizacional/orgaos-especificos-singulares/secretaria-executiva/stic/documentos/mdsagil_mec_v1-1_dez22.pdf. Acesso em: 17 jul. 2025.

CRÉDITOS

CONFEDERAÇÃO NACIONAL DA INDÚSTRIA – CNI

Antonio Ricardo Alvarez Alban
Presidente

Gabinete da Presidência

Danusa Costa Lima e Silva de Amorim
Chefe do Gabinete – Diretora

Serviço Nacional de Aprendizagem Industrial – SENAI

Antonio Ricardo Alvarez Alban
Presidente do Conselho Nacional

SENAI – Departamento Nacional

Gustavo Leal Sales Filho
Diretor-Geral

SUPERINTENDÊNCIA DE EDUCAÇÃO PROFISSIONAL E SUPERIOR – SUEPS

Felipe Esteves Morgado
Superintendente de Educação Profissional e Superior

Luiz Eduardo Leão
Gerente de Tecnologias Educacionais

Anna Christina Theodora Aun de Azevedo Nascimento

Cyro Visgueiro Maciel

Laíse Caldeira Pedroso

Décio Campos da Silva

Coordenação Geral de Desenvolvimento dos Recursos Didáticos
Nacionais

SENAI – DEPARTAMENTO REGIONAL DE SÃO PAULO

Ricardo Figueiredo Terra
Diretor Regional

Cássia Regina Souza da Cruz
Gerente de Educação

Luiz Carlos de Almeida Filho
Diretor da Escola SENAI de Educação Online

Ana Claudia Neif Sanches Yasuraoka
Melissa Rocha Gabarrone
Coordenação

Ariel Paixão
Desenvolvimento de conteúdo

Laercio Ferreira da Silva
Adriana Valéria Lucena
Allan Marcondes de Oliveira
Getulio Azevedo Alves
Márcio Simião de Queiroz
Rafael Felipe Pérsico
Sueli Diogo
Tatiane da Silva
Escritório de Projetos e Processos

Adriana de Souza Farias
Isabela de Oliveira Crivellari
João Francisco Correia de Souza
Phillipe Rocca Datovo
Ana Julia Carvalheiro Costa
Andrei Souza Alves dos Santos
Annadeives Aparecida Conceição Pita
Catarine Aurora Nogueira Pereira
Clarice da Silva Elias
Cristiane de Barros Rodrigues Favareto
Cristina Yurie Takahashi
Daniel Coronel
Daniel Vieira
Ester Veronica Santos Jardim
Flávia dos Santos Silveira
Jeferson Paiva
Juliana Barbosa Manso
Katya Martinez Almeida
Larissa Gonçalves Ferreira Dias
Leila Fernanda de Oliveira
Maria Eronilsa Nascimento Ciriaco
Mariana Almeida
Rafaelly Domicia de Castro
Raphael Todovertto Soares Silva

Regina Kambara Hirata
Simony Pimentel Santos do Nascimento
Thiago da Silva Ferreira
Victoria Brasil Munhoz
Victoria Mendes Pereira
Design de Aprendizagem

Camila Ciarini Dias
Maurício Rodrigues de Moraes
Anderson Dos Santos Araujo
Beatriz Helena Pedro
Caio Marques Rodrigues
Camilla Tauany Ribeiro Gomes
Ederson Guilherme Antonio Silva
Enrico Moreira Veiga Zilet
Ezequiel Regino Monção
Felipe de Santana Gonçalves
Gisele Gomes da Silva
Iris Toloni
Isabella Ferreira
Jevan Antonio de Souza Rocha
Juliana Rumi Fujishima
Ligia dos Santos Daghes
Luiz Sansone
Marisa Daniela Pereira Araujo
Mateus Eustáquio de Oliveira
Mateus Gonçalves de Melo
Matheus Antônio de Guimarães Elegância
Mhyrella Giovanna Massarente de Oliveira
Thiago da Silva Rosa
Tiago Florêncio da Silva
Willian Ricardo Colombo
Produção Multimídia

Rafael Santiago Apolinário
Aldo Toma Junior
Douglas Lacerda da Conceição
Rolfi Cintas Gomes Luz
Desenvolvimento Tecnológico

